



ספר פרויקט גמר

הנדסאי תוכנה

VoIP Traffic Management

שם הסטודנט: אופק וקנין – 215193814

מנחים: עומר אלוביץ', גל בראון

מקיף ה' דרכא, אשקלון

27.5.2025



תוכן עניינים

4	1. הצעת הפרויקט שאושרה על ידי משרד החינוך
21	2. תקציר/מבוא
21	2.1 הרקע לפרויקט
21	2.2 תהליך המחקר
22	2.3 סקירת ספרות
23	2.4 אתגרים מרכזיים
23	2.4.1 הבעיה איתה התמודד התלמיד
23	2.4.2 הסיבות לבחירת הנושא
23	2.4.3 מוטיבציה לעבודה
24	2.4.4 על איזה צורך הפרויקט עונה? איזה פתרון הפרויקט הזה בא לתת?
24	2.4.5 הצגת פתרונות לבעיה
25	3. מטרות/יעדים
26	4. אתגרים
27	5. מדדי הצלחה למערכת
28	6. רקע תיאורטי/ספרות מקצועית
34	7. תיאור המצב הקיים
35	8. ניתוח חלופות מערכתי
36	9. תיאור החלופה הנבחרת
37	10. אפיון המערכת שהוגדרה /מוצעת
37	10.1 ניתוח דרישות המערכת
38	10.2 מודול המערכת
39	10.3 אפיון פונקציונלי
40	10.4 ביצועים עיקריים
40	10.5 אילוצים
41	11. תיאור הארכיטקטורה
41	11.1 הארכיטקטורה של הפתרון המוצע בפורמט של Top-Down level design
43	11.2 תיאור הרכיבים בפתרון
43	11.3 תיאור תהליכים של מערכת ההפעלה שמתבצעים בפרויקט
45	11.5 תיאור פרוטוקולי תקשורת
45	11.6 שרת-לקוח
46	14. ניתוח ותרשים UML/Use cases של המערכת המוצעת
46	14.1 תיאור ה-UC העיקריים של המערכת
47	14.2 הצגת מקרה (use case) עבור כל הפונקציות העיקריות בפרויקט
67	14.3 מבני נתונים בשימוש
69	14.4 חישוב יעילות האלגוריתם
70	14.5 הקשרים בין היחידות השונות



70	14.6 עץ מדולים
71	Use case Diagram 14.7
71	14.8 רשימת Use cases
72	14.9 תרשים UML
73	Sequence Diagrams 14.9.1
75	Design class Diagram 14.10
81	14.11 תרשים מחלקות
81	14.12 תיאור המחלקות המוצעות
87	15. רכיבי ממשק
88	16. תיכון המערכת
88	16.1 ארכיטקטורת המערכת
88	16.2 תיכון מפורט
89	16.3 חלופות לתיכון המערכת
90	17. תיאור התוכנה
90	17.1 סביבת עבודה
90	17.2 שפות תכנות
92	18. תיאור מסכים
93	19. תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם (Screen flow diagram)
94	20. מה תפקידו של כל מסך / חלון עם צילום מסך של החלון הרלוונטי
96	21. תיאור מסך הפתיחה – מה הוא מכיל והאם משמש נקודת ניווט
97	22. כל מסכי האפליקציה / אתר / מערכת מנהלית, בליווי הסברים
98	23. עבור כל אלמנט תצוגה כדוגמת: כפתור, תיבת טקסט יש להסביר את תפקידם
101	24. הודעות למשתמש (alert) למיניהם
102	25. ממשק משתמש
103	26. קוד התוכנית
108	27. תיאור מסד הנתונים
109	28. מדריך למשתמש
110	29. בדיקות והערכה
111	30. מסקנות
112	31. פיתוחים עתידיים
113	32. בבליוגרפיה



1. הצעת הפרויקט שאושרה על ידי משרד החינוך

תיאור הפרויקט

הפרויקט עוסק בפיתוח מערכת צד שלישי לניהול, עיבוד וגישור תקשורת מדיה בזמן אמת של שמע בין רשתות, שמטרתה היא לספק איכות תקשורת גבוהה ואמינה בתנאי רשת שונים ומשתנים עם ביצוע התאמות לקודקים שונים של מידע עבור כל רשת.

המערכת תקלוט זרמי מדיה ממספר מקורות שונים, תבצע עיבוד והתאמות על הפקטות המתקבלות על פי התקנים המקובלים ב-VoIP, תנאי ויציבות הרשת ועל פי הגדרות המוגדרות מראש בקובץ קונפיגורציה המוגדר על ידי המשתמש. לאחר קליטת המידע ועיבודו המערכת תעביר את המידע בצורה אמינה ליעדם תוך שמירה על שלמות המידע, איכותו ושמירה על סדר חבילות השמע המקורי.

המערכת תתמוך בקודקי שמע שונים של מידע וימומשו בה האלגוריתמים של הקידוד והפענוח של אותם הקודקים, כדי להבטיח העברת מידע בקודקים שונים וכדי לספק תאימות בין מקורות שונים בתקשורת.

בתוך המערכת, ימומשו מנגנונים להתמודדות עם מצבי קצה ברשת העלולים להשפיע על הזרמת המדיה ליעד. מנגנונים אלו יוכלו להתמודד עם מצבים בהם הפקטות מגיעות באיחור או בסדר לא נכון תוך שמירה על עיכוב מינימלי בעיבודם וסידורם ועם שליחה רציפה של הנתונים לאחר סידורם.

למערכת תהיה היכולת לנטר בזמן אמת את איכות התקשורת ותוכל לספק משוב על הנתונים הסטטיסטיים של התקשורת. כתוצאה מכך, יהיה בידי המשתמש את היכולת להסיק את המצב הנוכחי של התקשורת ולבצע התאמות בעת הצורך.



הגדרת הבעיה האלגוריתמית

המערכת, עם קבלת חבילות המידע חייבת לבצע סינון וניתוח של חבילות המידע המתקבלות מן זרמי המידע תוך הבחנה בין חבילות תקינות התואמות לתקן הנקבע בפרוטוקולים RTP/RTCP (RFC3550) לבין פקטות UDP שאינן עומדות בתקן RTP/RTCP. על המערכת לוודא שהפקטות המתקבלות אינן שגויות או פגומות.

לצורך מענה לבעיה זו, ימומש ב-VTM אלגוריתמים לביצוע בדיקה של הפקטה האם היא עומדת בתקנים הנדרשים של פרוטוקול בזמן אמת. המערכת תבצע ניתוח של הפקטה ותבדוק עם הערכים ב-header תואמים למבנה הפקטה. יתרה מזאת, עם בדיקת פקטה הנבדקת בפרוטוקול RTP, תיבדק גם תקינותה ביחס לקודק המידע המצוי בקובץ קונפיגורציה של המשתמש מה שיגביר את מיטביות הסינון של הפקטות.

ייתכן שהמקורות השולחים שמע למערכת תומכים בקודקים שונים שאינם תואמים זה לזה. מצב זה עלול להיות בעייתי לביצוע תקשורת. מידע המועבר על ידי מקור אחד לא יכול להתקבל ולעבור עיבוד ישיר על ידי מקור היעד אם הוא אינו תומך בקודק הנשלח מה שעלול לגרום איבוד השמע.

לצורך מענה לבעיה זו, VTM תתמוך במנגנון המרה - Translator שמבצע התאמה בין הקודקים השונים בזמן אמת. מנגנון ההמרה יקלוט את הפקטה עליה רוצים להמיר, ראשית יפענח את המידע שבפקטה לצורתה הלא דחוסה ולאחר מכן יקודד אותו מחדש לקודק הנתמך על ידי היעד. תהליך זה יבוצע בהתבססות על אלגוריתמי פענוח ודחיסה של מידע ועל פי הגדרות קובץ הקונפיגורציה. עקב ביצוע ההמרה, יהיה ניתן לבצע תקשורת זמן אמת בין מקורות שמע התומכים בקודקים שונים תוך שמירה על איכות התקשורת וביצוע של עיכוב מינימלי בתקשורת.

ברשתות תקשורת זמן אמת, עלולה להתעורר בעיה של שונות בזמני הגעת הפקטות. פקטות הנשלחות ברשת עשויות להגיע ליעדן בזמנים לא סדירים עקב מצבי רשת משתנים, למשל עומסים ברשת, עיכובים שונים או איבוד של פקטות. כתוצאה מכך פקטות עלולות להגיע ליעדן או בתזמונים שונים (הפקטה מגיעה מוקדם מידי או מאוחר מידי) או שבסדר שגוי. כתוצאה מכך, יש פגיעה באיכות השיחה שיכולה לבוא לידי ביטוי בקטיעות, עיוותים בשמע או השמעה לא רציפה של המידע.

לצורך מענה לבעיה זו, ימומש בתוך ה-VTM מנגנון שמטרתו הוא לסדר מחדש את הפקטות בסדרן המקורי ולסדר את העיכובים שנוצרו בין הפקטות – jitter buffer. במנגנון זה הפקטות הנקלטות במערכת ישמרו ב-buffer זמני, כאשר במהלך אגירתם ב-buffer הפקטות יאורגנו מחדש לפי סדרן המקורי ולאחר ביצוע הארגון הפקטות ישלחו ליעדן בצורה רציפה. מנגנון זה מבטיח השמעה חלקה וברורה של המידע אצל היעד גם במצבים בהם הרשת לא יציבה תוך שמירה על עיכוב מינימלי בזמן השליחה.

בנוסף לבעיות שציינו כנאמר, במערכות זמן אמת, נדרש ניטור ומדידת איכות השיחה בין המקורות השמע. איכות השיחה נמדדת על פי מספר פרמטרים, עיכוב שליחת הפקטות, אובדן הפקטות ושונות בזמני ההגעה של הפקטות.

לצורך מענה לבעיה זו, ימומש ב-VTM ניתוח של פקטות בפרוטוקול RTCP שממנה יהיה ניתן להסיק מידע סטטיסטי על איכות התקשורת ולזהות בעיות הנובעות בזמן ביצוע התקשורת. למשל, במקרים בהם שונות זמני ההגעה גבוהה יהיה ניתן להתאים את מנגנון ה-jitter buffer שיגדל באופן דינאמי למניעת קטיעות. ה-VTM גם תיצור לוגים עבור הנתונים



הסטטיסטיים בשביל שיהיה ניתן לזהות אם יש מגמות ותקלות החוזרות על עצמן לצורך בקרה של המערכת.



רקע תיאורטי בתחום הפרויקט

Jitter buffer

זהו מנגנון המהווה מאגר זמני של אחסון פקטות מדיה המושמש במערכות זמן אמת. Buffer זה נועד לטפל בהשפעות הנגרמות עקב מצבים של jitter ברשת, אשר גורמות שיבושים בתקשורת הבאים לידי ביטוי בקבלת הפקטות באיחורים ותזמונים לא צפויים ובסדר שונה ממה שהן נשלחו במקור. במהלך פעילותו, ה-jitter buffer קולט את הפקטות, מסדר אותן בסדר המקורי ושולח אותן בתזמונים זהים, על מנת שאצל נקודת היעד יופחת מצב ה-jitter במהלך השמעת המידע. ישנם 2 סוגים של jitter buffer:

1. **Fixed size jitter buffer**: אלגוריתם בו נקבע גודל קבוע עבור הבאפר, ועל כן מגדיר מספר מקסימלי קבוע של פקטות הניתנות לאחסון. זהו האלגוריתם הכי פשוט לשימוש אך עם זאת הוא אינו מסתגל לתנאי הרשת הנתונה שעלולים להשתנות.
2. **Adaptive jitter buffer**: אלגוריתם המתאים את גודל הבאפר בצורה דינאמית בהתבסס על תנאי הרשת. שינוי הגודל יכול לבוא לידי ביטוי למשל אם ה-latency הוא גבוה אז ניתן להגדיל את גודל הבאפר בכדי להפחית את הסיכויים לאובדן פקטות או שאם תנאי הרשת יציבים אז ניתן להפחית את גודלו כדי להימנע מעיכובים מיותרים בשליחה.

אלגוריתם ה-jitter buffer ימומש כלהלן:

1. המערכת תקלוט את הפקטות מן הסוקטים וישמרו בתוך ה-jitter buffer לאחר ניתוחן.
2. יבוצע מיון בתוך הבאפר על פי השדות timestamp ו-sequence number אשר מופיעים בתוך ה-RTP header.
3. יבחן מצב הרשת והנתונים הסטטיסטיים של השיחה המסופקים על ידי ניתוח פקטות ה-RTCP לשינוי דינמי של גודל הבאפר.
4. העברה של הפקטות בסדר המקורי ובאינטרוולים זהים לצורך המשך עיבודם ושליחתם אל היעד.

QOS

Quality of service הוא מונח המודד את היכולת של הרשת לספק שירותים באיכות גבוהה למשתמשי קצה. המטרה העיקרית שלה היא לתת עדיפות גבוהה יותר ליישומים קריטיים כמו למשל שידורים ותקשורת בזמן אמת. טכנולוגיה זו שואפת לשפר ביצועים, להפחית את ה-latency ולהפחית אובדן פקטות מידע.

ישנם פרמטרים שונים עליהם יש התייחסות בטכנולוגיית QOS:

1. **Packet loss**: מצב היכול לקרות כאשר יש עומס בתעבורת הרשת.
2. **Jitter**: פרמטר המתייחס לשונות הגעת הפקטות, ובא לידי ביטוי במצבים בהם הפקטות נמסרות אל יעדן במרווחי זמן משתנים. פרמטר זה חיוני למערכות זמן אמת מכיוון שהוא עלול לפגוע בקליטת השמע אצל היעד. זמן זה נמדד ב-milliseconds.
3. **Latency**: מדד המתאר את כמות הזמן העוברת בהעברת המידע מן המקור ליעד. במערכות זמן אמת על מדד זה להיות נמוך כדי להימנע מאיכות ירודה בהעברת המדיה. זמן זה נמדד ב-milliseconds.



4. Bandwidth: מדד המתייחס ליכולת השידור של הרשת, כלומר כמה נתונים ניתן להעביר בפרק זמן נתון. מעריכים מדד זה על פי כמות הביטים המקסימלית שניתן להעביר בשנייה.

איכות השירות, באה לידי ביטוי במערכת זאת על ידי קבלת זרמי מידע של פקטות בפרוטוקול RTCP אשר מספק בזמן אמת מידע סטטיסטי על זרמי המדיה עם ביצוע ניטור התקשורת המתקיימת. מידע ה-QOS בא בעיקר לידי ביטוי בקבלה וניתוח פקטות מסוג RR,SR אשר מכילות בתוכן מידע מסוג זה.

Transcoding

תהליך בו מתבצעת המרה של קודק אחד של מדיה דיגיטלית (כגון אודיו ווידאו) לקודק אחר של מדיה דיגיטלית. במהלך תהליך זה, מתבצע לקיחת קובץ המדיה אשר מקודד בקודק מדיה אחד וביטול הקידוד שלו בכדי לשנות את גודל הקובץ או את קצב הסיביות שלו. תהליך זה מתבצע לעיתים קרובות על מנת להגדיל את מספר מכשירים וההתקנים היכולים לבצע תקשורת זמן אמת ולהפוך את הנתונים לנגישים יותר.

במסגרת פרויקט זה, ימומשו אלגוריתמי קידוד והמרה בין 2 קודקים של שמע –

- 1. G711 ALAW** – אלגוריתם המשמש לשידור דיגיטלי של אותות קול אנלוגי דרך הרשת. הוא נמצא בשימוש נרחב במערכות טלפוניה ותקשורת, ומספק דחיסה ופירוק נתונים יעילים ואמינים של נתוני קול. קודק זה הוא מסוג Lossless.
- 2. Speex** – קודק שמע קוד פתוח, המיועד לקידוד דחיסה של אודיו ומוזיקה. קודק מאוד גמיש התומך במגוון רחב של איכות שמע ובקצב סיביות משתנה אשר יכול לקודד ברוחב פס צר וברוחב פס רחב. קודק זה הוא מסוג Lossy. ניתן לבצע שינוי דינאמי בקצב הסיביות על פי מדד ה-bandwidth.

עם התמיכה בפורמטים אלו, VTM תוכל לבצע המרות מהסוג Lossless to Lossy ומהסוג Lossy to Lossless כלומר יתבצעו המרות מקודק דחיסה ללא איבוד מידע לקודק עם איבוד מידע ולהיפך. בביצוע ההמרה בין הקודקים, ימומשו אלגוריתמי קידוד והפענוח של הקודקים המצוינים, כאשר עבור כל מידע היעבור המרה, ראשית יתבצע אלגוריתם הפיענוח של הקודק לקבלת המידע הלא דחוס ולאחר מכן יתבצע אלגוריתם הקידוד של קודק היעד. מנגנון זה ימומש בתוך מנגנון ה-translator אשר יממש את האלגוריתמים כמתואר:

Look up table – מבנה נתונים המשמש כדי ליעל חישובים על ידי חישוב מוקדם ואחסון תוצאות של פעולות מורכבות או חוזרות. במקום לחשב מחדש ערכים במהלך זמן הריצה, המערכת מאחזרת את התוצאה המחושבת מראש מהטבלה, מאפיין המשפר את היעילות ומקטין overhead.

בקודק G711 מתבצע תהליך הקידוד דחיסה של 16 ביט מידע ל-8 ביט ועבור פיענוח מתבצע הרחבה של 8 ביט ל-16 ביט. לשם ייעול אלגוריתם ההמרה והפענוח, אשתמש במבנה נתונים זה באופן הבא:

- ישנם 256 ערכים אפשריים של G711 אשר ניתן לייצג בנתון אחד וישנם 65,536 ערכים אפשריים של מידע לא דחוס (PCM).
- האלגוריתם יגדיר 2 מערכים בגדלים 256 ושל 65,536 של הטיפוסים 8 ביטים לא מסומנים ושל 16 ביטים לא מסומנים בהתאמה.



3. עבור $i=0$ וכל עוד $i < \text{table.size}$:

- a. בצע את האלגוריתם הקידוד והפענוח אשר מקבל את i כפרמטר.
- b. השם את הערך המתקבל באינדקס i בתוך הטבלה.
- c. העלה את i באחד.

כתוצאה מכך, עם קבלת הפקטות, יהיה ניתן לפענח ולקודד את המידע על ידי השמת האיברים מהאינדקסים המתאימים בטבלאות ובכך להימנע מחישובים החוזרים על עצמם בזמן ריצה.

Libspeex library – לצורך ביצוע קידוד ופענוח של הקודק speex, נעשה מימוש של האלגוריתמים באמצעות הספרייה libspeex אשר מספקת יישום של הקודק ובנוסף פונקציות ומנגנונים אשר מסייעים בקידוד ופענוח של מידע. בשונה מן G711, הספרייה מספקת קידוד ופענוח של frames ולא בערכים יחידים של 16 או 8 ביט. VTM תתמוך בקידוד ופענוח ברוחב פס צר של הקודק (narrowband mode). Narrowband הוא מנגנון הנפוץ במערכות VoIP. בנוסף, אלגוריתמי הקידוד והפענוח של הפקטות ימומשו על כל 160 דגימות שמע המהוות 160 ערכים מהטיפוס 16 ביטים מסומנים.



הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות

הפרויקט, יהיה מורכב מהמודולים הבאים:

VTM – Relay server תקבל פקטות ותשלח פקטות דרך שרת הפועל במודל אסינכרוני ופועל על גבי פרוטוקול UDP/IP. השרת יסווג את זרמי המידע לפי השיחה והמקור ממנה הוא קיבל את הפקטות וינתב אותן למשתתפים בשיחה בה הוא נמצא.

קובץ קונפיגורציה – ל-VTM יהיה קובץ מיוחד בה יוגדרו החוקים לפיו VTM תעבוד. בכל פעם ש-VTM תחל בפעילותה, היא ראשית תטעין את קובץ הקונפיגורציה, בו מוגדרים הדברים הבאים:

1. הגדרות לביצוע logging – log severity ואת הקובץ אליו ירשמו הלוגים.
2. המאפיינים של פקטות מכלל המקורות - גודל הפקטה אשר VTM מצפה לקבל.
3. זוג מקורות – עבור כל מקור בזוג יוגדרו הדברים הבאים: כתובת IP, פורט האזנה לקבלת פקטות והפורט אליו הוא שולח למערכת. כמו כן יוגדר עבור כל מקור הקודק בו הוא תומך.

Packet parser – ל-VTM ימומשו אלגוריתמים אשר יבדקו מזרמי המדיה האם הפקטות עומדות בתקן של פרוטוקולים RTCP/RTP. מנגנון זה יבדוק את הפקטות על פי שדות ה-header של הפקטה בשביל לבחון האם הערכים עומדים בקנה אחד עם מבנה הפקטה, ובנוסף עבור פקטות המתקבלות מסוקט המצפה לפקטות RTP על גבי UDP תיבדק גם תקינותו ביחס לקודק המוגדר בקובץ הקונפיגורציה, שם האם ערך ה-payload type נתמך במערכת והאם גודל ה-payload אינו חורג מהגודל הצפוי.

Jitter buffer - מנגנון שיטפל במקרים בהם יש בעיות רשת, כאשר במצבים אלו המנגנון יסדר את הפקטות שנשלחו בתזמונים שונים או בסדר לא תקין בחזרה לסדרן המקורי וישלחו ליעדן בעיכוב הכי מינימאלי שניתן.

Translator – עם הצורך לתמיכה בקודקים שונים ובהמרתן, VTM תכלול מנגנון של translator שתקבל אליה את הפקטות, תבדוק לאיזה קודק צריך להעביר את הפקטה ותבצע את ההמרה המתאימה לפני שליחת הפקטה ליעדה. ה-translator יתמוך באלגוריתמי קידוד ופענוח של הקודקים G711 ALAW ושל Speex.

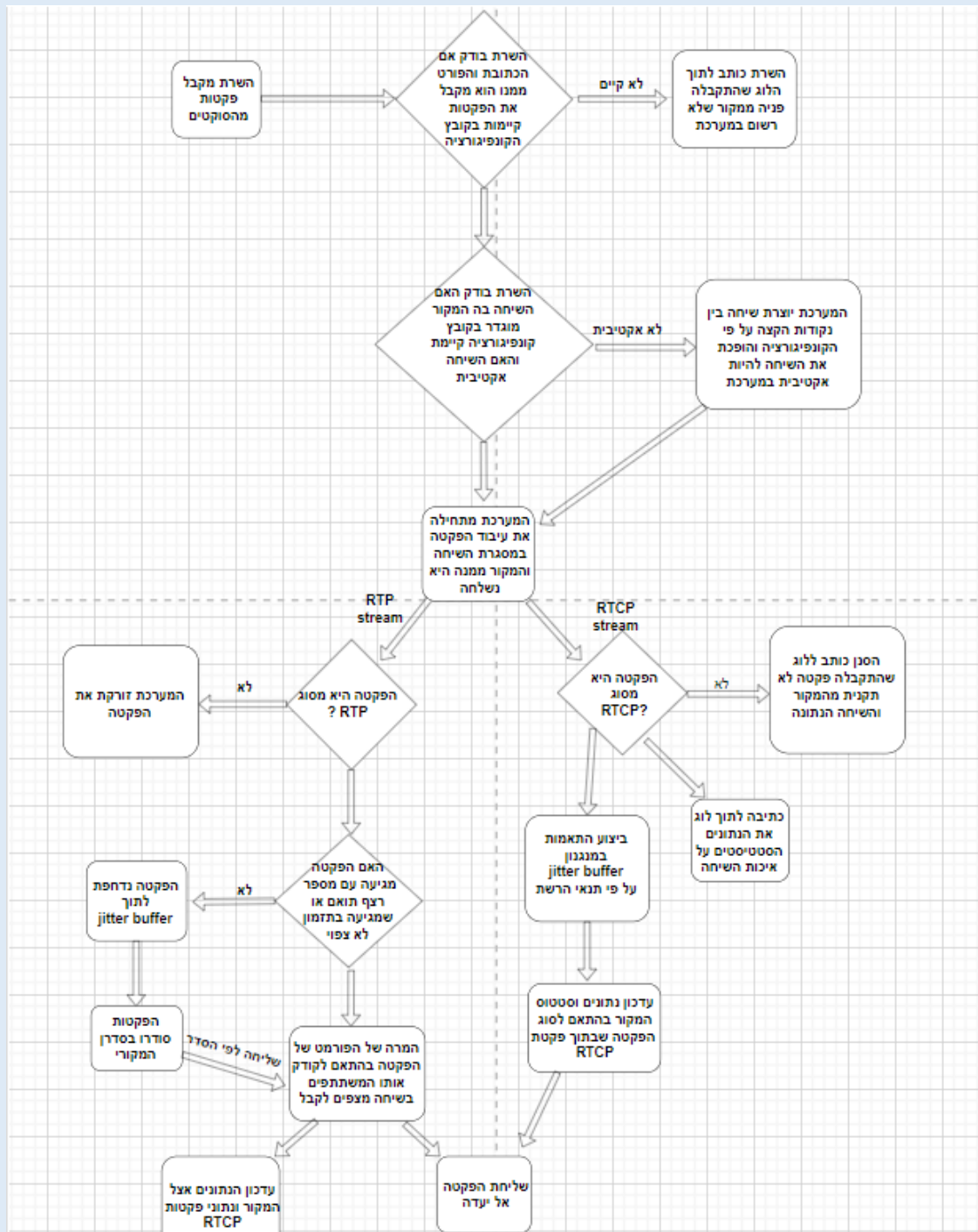


ארכיטקטורה של הפרויקט

ראשית, לפני ש-VTM מבצע את פתיחת הסוקטים והתחלת עיבוד פקטות המידע, ישנם פעולות ראשוניות אותם הוא מבצע עם תחילת הרצתו:

1. טעינת קובץ הקונפיגורציה לבדיקת תקינותו.
2. טעינת טבלאות look up tables של הקודק g711.
3. טעינת logger לביצוע שליחת לוגים בזמן אמת.

עם ביצוע קבלת והעברת המידע דרך VTM, מתקיים התהליך הבא בין המודולים שבמערכת:





הליכים עיקריים בתחום רשתות מחשבים/תקשורת נתונים

קליטת פקטות UDP

VTM מקבל פקטות שנשלחות על ידי מקורות שונים דרך הרשת. הפקטות נשלחות בפרוטוקול שכבת התעבורה UDP, וכתוצאה מכך בין המערכת למקורות לא יוצר חיבור קבוע אלא ישלחו פקטות בנפרד שלא מובטח אמינותם. VTM יפתח 2 סוקטים בתוך ה-relay server המהווים צינורות של מידע בין המקורות למערכת, שני הסוקטים יפעלו על גבי פרוטוקול UDP, סוקט אחד יצפה לקבל פקטות RTP על גבי UDP והסוקט השני יצפה לקבל פקטות RTCP על גבי UDP. הכתובת והפורט שהשרת מאזין הם נקודות הכניסה אל השרת, ואליהם המקורות יוכלו לשלוח פקטות UDP אותם המערכת תוכל לאחסן בתוך זיכרון זמני שנקרא באפר.

ניתוח של פקטות RTP

עבור כל פקטה המתקבלת בסוקט של פקטות RTP על גבי UDP מצופה ממנה שהיא במבנה הבא: RTP header + extension header (optional) + payload. לאחר קבלת פקטה מן סוקט זה, המערכת תנתח את הפקטה על פי הערכים הנמצאים ב-header ותנתח אותה אם היא עומדת לפי תקן פרוטוקול RTP.

המרת פורמט שמע

על פי הגדרות המשתמש בקובץ הקונפיגורציה, VTM משנה את קודק השמע שהתקבל מהמקור כדי להתאים אותו לצרכים של היעד. לשם צורך זה, VTM תתמוך באלגוריתמים של קידוד ופענוח עבור הקודקים G711-ALAW ו-Speex שלשם ההמרה, ראשית ה-payload עובר פענוח לקבלת המידע הגולמי הלא דחוס המקורי ולאחר מכן דחיסתו מחדש בקודק המבוקש.

ניהול jitter buffer

עבור הפקטות המתקבלות דרך הרשת, יכולים לייוצר מצבים בהן פקטות מגיעות באיחור או בסדר לא נכון עקב בעיות תקשורת. לשם התמודדות במצבים אלה, ימומש בתוך VTM מנגנון jitter buffer ששומר את הפקטות זמנית כדי לסדר אותן בסדר הנכון וליצור זרימה רציפה של נתונים. עבור ניהול באפר זה, בנוסף לביצוע בדיקת התקינות של פקטות RTP ביחס לתקני הפרוטוקול על המערכת לבחון את הפקטה ביחס לפקטות הקודמות בזרם המידע – האם שדה sequence number הוא עקבי וגדול באחד מקודמו והאם שדה timestamp הוא הועלה בצורה עקבית ותואמת לקודק השמע.

ניהול RTCP

בסוקט השני אותו פותח השרת, מצופה לקבל פקטות RTCP על גבי UDP לשם ניטור איכות השיחה ולניהול המקורות והשיחות העוברות במערכת. ראשית המערכת תנתח את הפקטה כדי לבחון האם הפקטה היא פקטת RTCP שעומדת בתקנים, ועם הנתונים המועברים בפקטה, המערכת תבצע התאמות לשיפור איכות השידור ותאבחן בעיות שונות העלולות לנבוע מתוך המידע המועבר.



ניהול מקורות ושיחות

עבור כל מקור ושיחה ישמר מידע בתוך המערכת מה שמאפשר ניהול נפרד שלהם. VTM
עבור כל שיחה המנסה להזרים מידע לשרת יבדוק אם הוא מופיע בקובץ קונפיגורציה ולאחר
מכן יבדק אם היא פעילה וקיימת במערכת. עבור כל שיחה ינוהל הקמתה עם השמת נקודות
הקצה המתאימות, העברת הפקטות ליעדן בשיחה עם ביצוע התהליכים התוארו לעיל וסגירת
השיחות מהמערכת בהתאם לנתונים המופיעים בפקטות RTCP.



הליכים עיקריים בתחום מערכות הפעלה

Asynchronous I/O

שיטה לביצוע פעולות קלט/פלט מבלי לחסום את ביצוע התוכנית, כלומר התוכנית יכולה להמשיך בביצוע משימות אחרות באותו הזמן. לאחר סיום פעולת הקלט/פלט מופעל callback לטיפול התוצאה של הפעולה. גישה זו משפרת ביצועים במיוחד בתוכנות הדורשות טיפול במהירות של פעולות קלט/פלט בו-זמנית. תכונה זו באה לידי ביטוי במערכת, במנגנון קבלת הפקטות מן הסוקטים. למערכת תהיה את היכולת לנהל מספר זרמי מידע ממקורות שונים במקביל ביעילות, ללא חסימות מצד מערכת ההפעלה וללא צורך להקצות תהליכון עבור כל מקור – מה שמקטין overhead של המעבד והזיכרון. יתרה מזאת, מאפיין זה יוכל להקטין את ה-latency של התוכנית בכך שהמערכת תוכל לעבד את הפקטות מן המקורות השונים מיד אחרי שהם מתקבלים מהסוקטים במקום שיהיו זרמי מידע שיחסמו מה שיעלה את ה-latency ועלול לפגום באיכות השיחה.

Multithreading

טכניקה המאפשרת לתוכנית להריץ מספר תהליכונים במקביל, ולמעשה מאפשרת לה לבצע מספר משימות במקביל. כל תהליכון הוא יחידת ביצוע נפרדת בתוך התוכנית, ובסביבה מרובה תהליכונים, הם חולקים את אותו שטח זיכרון אך פועלים באופן עצמאי. באמצעות טכניקה זו, ניתן לשפר את ביצועי המערכת על ידי הקצאת תהליכונים שעבור כל שיחה המתרחשת על גבי המערכת, יוקצה תהליכון שיבצע את קבלת הפקטות מזרמי המידע, עיבודם ושליחתם ליעד. יתרון זה בא לידי ביטוי בהורדת עומס מן התהליכון הראשי וחלוקתו בין התהליכונים השונים ובנתינת האפשרות למיקסום הביצועים והשימוש של ליבות ה-CPU של החומרה על גביה תעבוד המערכת.



פרוטוקולי תקשורת הנמצאים בשימוש

פרוטוקול UDP

פרוטוקול תקשורת לא מבוסס חיבור המשמש ברשתות להעברת נתונים בצורה מהירה ויעילה. הוא פועל מבלי ליצור חיבור או להבטיח מסירה אל נקודת היעד, מה שהופך אותו למתאים ליישומים בזמן אמת שבהם ניתנת עדיפות על מהירות שליחת הנתונים על פני האמינות של הנתונים שנשלחו. UDP תומך גם בשידור ובריבוי שידורים, המאפשרים שליחת נתונים למספר נמענים בו זמנית.

פרוטוקול RTP

פרוטוקול שנועד לטפל בתעבורה בזמן אמת כמו שמע ווידאו של האינטרנט. הפרוטוקול מספק אמצעים כדי לפצות על ריצוד וגילוי של פקטות מידע אבודות ולסדר פקטות מידע שהגיעו בסדר שונה בשליחתן, ומאפשר העברת מידע למספר יעדים שונים דרך שידורי IP. משתמשים בפרוטוקול זה עם שימוש בפרוטוקול UDP.

פרוטוקול RTCP

פרוטוקול שנועד לספק מידע ופיקוח על איכות השידור בזמן אמת של שמע ווידאו. פרוטוקול זה מספק משוב על איכות הפצת הנתונים במקרים בהם יש עיכובים או אובדן פקטות, מסייע בסינכרון מספר זרמי מדיה באותו הרשת, ובנוסף מנהל בקרה על העברת הנתונים כמו זיהוי המשתתפים וניהול סיום העברת הנתונים.



פיתוחים עתידיים

יצירת ממשק משתמש ויזואלי עבור ניטור השיחות שה - VTM מנהל. הצגת השיחות ופעילותן באמצעות sound waves עם אינדיקציה של שיחה פעילה או לא.



תיאור טכנולוגיה הנדסה

C++: שפת תכנות בסגנון high-level אשר מהווה superset לשפת C. השפה תומכת בתכנות מסוג מונחה עצמים וכמו כן גם בתכנות פרוצדורלי וגנרי. שפה זו היא שפה רבת עוצמה מאפשרת למפתחים לכתוב קוד עבור יישומים גדולים ופיתוח תוכנה בעלי ביצועים גבוהים.

יתרונות השימוש בשפת C++ להכנת הפרויקט:

1. למערכת זו, חשוב שיהיו ביצועים כמה שיותר מהירים ויעילים בשל עצם העובדה שבין דרישות המערכת היא חייבת לעבד שיחות בזמן אמת. שפת תכנות זו היא שפה מאוד חזקה הידועה במהירותה ויעילותה היחסי בביצועיה ובעבודתה הקרובה לחומרה, ולכן היא עונה על דרישה זו.
2. שפת תכנות זו תומכת בתכנות מסוג מונחה עצמים מה שמאפשר את חלוקת הפרויקט למחלקות. תכונה זו מאפשרת ביצוע של ניהול ותחזוקת הפרויקט בצורה טובה יותר.
3. לשפה זו יש מגוון רחב של ספריות וכלים זמינים.

Cmake – הוא מחולל מערכת בנייה בקוד פתוח המפשט את תהליך ניהול תצורת הבנייה של פרויקטי תוכנה על פני פלטפורמות ומהדרים שונים. Cmake תומך במבני פרויקטים מודולריים, פקודות בנייה מותאמות אישית ותאימות בין פלטפורמות, מה שהופך אותה לבחירה פופולרית עבור יישומי C++ קטנים וגדולים כאחד.

boost – ספריות boost נועדו להיות שימושיות נרחבות וניתנות לשימוש במגוון רחב של יישומים. Boost פותחה כדי לספק תכונות ופונקציות מתקדמות שאינן מסופקות מחדל ע"י הספרייה הסטנדרטית של C++ והיא מכילה בתוכה מגוון רחב של תכונות כגון ספריות לעיבוד מטריצות, לפעולות מתמטיות, לעבודה עם תאריכים וזמנים, ולתקשורת בין מחשבים.

מבין הספריות שהאוסף boost מציעה, בפרויקט נעשה שימוש בספריות הבאות:

1. **Boost.Asio** - ספריית C++ חוצת פלטפורמות לתכנות תקשורת רשתות וקלט-פלט ברמה נמוכה המספקת למפתחים מודל אסינכרוני עקבי. הספרייה מספקת תכונות ופונקציות לעבודה עם תקשורת רשת בפרוטוקולים מתקדמים, למשל TCP, UDP וגם SSL. הספרייה מאפשרת למתכנתים ליצור שרתי רשת ולקוחות מתקדמים בקלות וביעילות.
2. **Boost Log** - ספריית C++ שמטרתה הוא לאחסן בתוך יומן (log) מידע חיוני על ביצועי הפרויקט בזמן ריצה. תכונה זו משמעותית שתהיה לפרויקט, מכיוון שאם יקרו מצבים בהם משהו משתבש, יהיה ניתן להשתמש במידע המאוחסן ביומנים כדי לנתח את התנהגות הפרויקט ולבצע את התיקונים הדרושים.
3. **Boost Property tree** - ספריית C++ המספקת מבנה נתונים המאחסן עץ ערכים מקונן באופן שרירותי המופיע באינדקס בכל רמה על ידי מפתח כלשהו. כל צומת של העץ מאחסן את הערך שלו, בתוספת רשימה מסודרת של תת-הצמתים שלו והמפתחות שלהם. העץ מאפשר גישה נוחה לכל אחד מהצמתים שלו באמצעות



נתיב, שהוא שרשור של מספר מפתחות. מבנה נתונים זה הוא מבנה נתונים מגוון שמותאם במיוחד להחזקת נתוני קונפיגורציה. הספרייה בנוסף מספקת מנתחים ומחוללים למספר פורמטים של מידע, שיכול להיות מיוצג דרך העץ. הפורמטים יכולים להיות JSON / XML / ini.

Speex (libspeex) – ספריית C המספקת יישום של קודק השמע Speex. הספרייה מספקת קידוד ופענוח יעילים של דיבור צר ופס רחב, מה שהופך אותה לאידיאלית עבור יישומי VoIP והזרמת אודיו. Speex מתמקדת בשידור אודיו עם אחזור נמוך וקצב סיביות נמוך, ומציעה רמות דחיסה ואפשרויות איכות שונות. הספרייה קלת משקל, תומכת בדיכוי רעשים, זיהוי פעילות קולית וביטול הד, והיא מותאמת לתקשורת קולית בזמן אמת בסביבות מוגבלות ברוחב פס.

Ini files – קובץ קונפיגורציה פשוט מבוסס טקסט המשמש בדרך כלל לאחסון הגדרות עבור תוכנות או יישומים. הוא מארגן נתונים למקטעים, עם צמדי מפתח-ערך עבור הגדרות בודדות. קבצי INI קלים לקריאה, עריכה וניתוח, מה שהופך אותם בשימוש נרחב למשימות תצורה קלות משקל.



לוחות זמנים

שלב עבודה	תאריך סיום השלב
למידת נושאי VoIP	09.07.2024
למידה והעמקה בשפת C++	17.07.2024
כתיבת RTP+RTCP parser על פי תקן RFC3550	13.08.2024
יצירת קובץ חוקה עבור המערכת	29.08.2024
מימוש Translator עבור קודקים G711 ו-Speex ו-ALAW	1.12.2024
הוספת כתיבת לוגים למערכת	10.12.2024
מימוש שרת relay RTP	10.01.2025
מימוש jitter buffer	01.02.2025
התאמת המערכת למערכת בעלת ריבוי תהליכים	01.03.2025
בדיקות עומסים ו-benchmarking	15.03.2025



טופס הצהרה על מקוריות העבודה

714918	אשקלון	מקיף ה' דרכא אשקלון	644450
סמל שאלון	ישוב	שם המכללה	סמל ביה"ס

אני הח"מ, מצהיר בזאת כי עבודה זו נעשתה על ידי בלבד.
פרויקט הגמר נעשה על סמך הנושאים שלמדתי ובאופן עצמאי / בשיתוף עם חברי לצוות, ונעשתה בהנחיית המורה המנחה.
אני מודע לאחריות שהגנני מקבל על עצמי על ידי חתימתי על הצהרה זו שכל הנאמר בה אמת ורק אמת.

שם התלמיד: אופק וקנין ת.ז. 215193814 חתימת התלמיד:
תאריך: 18.12.2024

אישור המנחה האישי:

הריני מאשר שהפרויקט בוצע בהנחיית, קראתי את העבודה ומצאתי כי היא מוכנה לצורך הגשת פרויקט גמר הנדסאי.

שם המנחה: גל בר-און ת.ז. 033297607 חתימת המנחה:

טלפון נייד: 0523499554 דוא"ל: gal.baron@gmail.com

אישור רכז/ת המגמה:

שם הרכז/ת: גל בר-און ת.ז. 033297607 חתימת הרכז/ת:

תאריך: 19/12/2024



2. תקציר/מבוא

2.1 הרקע לפרויקט

בעידן הדיגיטלי של היום, מערכות תקשורת מבוססות VoIP (Voice over IP) הפכו לסטנדרט בתקשורת שמע בזמן אמת. טכנולוגיה זו מאפשרת שידור שמע על גבי רשתות IP ובכך מחליפה את מערכות הטלפוניה המסורתיות – תוך חיסכון בעלויות, גמישות תפעולית, ויכולת שילוב במערכות תוכנה מודרניות.

עם זאת, טכנולוגיית VoIP רגישה מאוד לאיכות רשת משתנה והיא מציבה אתגרים טכנולוגיים מהותיים. מערכות VoIP עלולות לסבול מירידה משמעותית באיכות השמע עקב תופעות כגון אובדן פקטות (packet loss), שיהוי (latency), שונות בזמני ההגעה של הפקטות (jitter) וביצוע תקשורת בין נקודות קצה באמצעות קודקים שאינם תואמים ביניהם. בעיות אלו מובילות לקטיעות בשיחה, עיוותים בקול, ולעיתים אף להתנתקות. בתנאי רשת מאתגרים – כמו עומסים פתאומיים, חיבורים לא יציבים או סביבות צבאיות – ההשפעה על תקשורת השמע הופכת קריטית.

בשל כך, עולה הצורך בפיתוח מערכת גנרית, עצמאית וגמישה – אשר תדע לגשר, לעבד ולפקח על תקשורת שמע בזמן אמת תוך תמיכה במצבי קצה, ניהול איכות השיחה והתאמה בין קודקים שונים. מערכת כזו תשמש כתשתית טכנולוגית רחבה לפרויקטים נוספים בתחום ה-VoIP, כלומר היא בנויה כך שתוכל לשמש כתשתית לאינטגרציה עם פרויקטים עתידיים בתחום.

2.2 תהליך המחקר

בתחילת דרכי בשנת הפרויקטנטים, התחלתי בללמוד את היסודות של עולם תקשורת הנתונים ועולם ה-VoIP. תחילה, למדתי על העקרונות המרכזיים ותפקידם של הפרוטוקולים RTP ו-RTCP. תוך כדי הכנת הפרויקט צללתי לנושאים מתקדמים יותר בתחום כגון כיצד הפרוטוקולים אלו מסייעים בגילוי מצבים של packet loss ו-jitter ועל הדרכים המסורתיות בהם מתמודדים ומתגברים על בעיות אלו. בנוסף, למדתי על מכשירים ומנגנונים הנתמכים ברמת ה-RTP כמו ה-translator ועל חשיבותו ביצירת רשתות תקשורת מבוססות VoIP. במקביל ללימוד החומר התיאורטי, נדרשתי במהלך הפרויקט להעמיק את הידע שלי בשפת התכנות C++. לשם כך, עשיתי את המעבר מהידע שצברתי במהלך העתודה הטכנולוגית בשפת C לתכנות בשפת C++, ולמדתי את היסודות והכלים שהתווספו בשפת C++. במהלך העבודה על הפרויקט, נחשפתי לכלים יותר ויותר מודרניים בשפה אשר למדתי את מאפייניהם ואת השימוש המקובל בהם. בין היתר, למדתי לעבוד עם ספריות כמו Boost, שסייעו רבות בפיתוח מנגנוני הפרויקט. תרומתה של Boost באה לידי ביטוי במתן שירותים ו-API שהספרייה הסטנדרטית לא מספקת כברירת מחדל, ובאמצעותה מימשתי את מודל התקשורת של המערכת וכן את ניהול העבודה מול קבצים חיצוניים, כגון קובץ הקונפיגורציה המנחה את פעילות המערכת וקובץ הלוגים אליו נכתבות פעולות המערכת.



2.3 סקירת ספרות

בעידן הדיגיטלי, טכנולוגיית ה-VoIP משמשת כאמצעי תקשורת עיקרי עבור ביצוע שיחות קוליות על גבי הרשת. טכנולוגיה זו משמשת במספר תחומים כגון תשתיות אזרחיות, מערכות שידור streaming, ועד לפלטפורמות משחק ותשתיות צבאיות. חדירתה של הטכנולוגיה הפכה אותה למאוד נפוצה לשימוש אך גם מאתגרת מבחינה הנדסית. מערכות אלו נאלצות להתמודד עם תנאים לא אידאליים ברשת IP – דבר הבא לידי ביטוי בבעיות של איכות רשת ה-IP, שונות בקודקים וקשיי סנכרון.

בתנאים של עומס רשת, הפקטות עלולות להגיע בסדר שונה או בעיכוב, מה שגורם לקטיעות והפרעות בשמע. בשביל להתמודד עם בעיה זו, פותחו מספר מנגנונים כאשר jitter buffer ובפרט adaptive jitter buffer הוא אחד מהם והוא נהפך להיות מנגנון ליבה במערכות זמן אמת.

אתגר נוסף שעולה עם השימוש בטכנולוגיה היא אובדן של הפקטות ברשת. מכיוון שאין ביצוע בתקשורת זמן אמת תהליך של retransmission המערכות צריכות לממש מנגנון שיאפשר לגשר על בעיה זו. בשביל להתגבר על בעיה זו, יש בתעשייה מספר גישות שונות שלכולן מטרה משותפת – להשלים בצורה מסוימת את המידע החסר. שיטות שונות לכך יכולות להיות באמצעות פקטות שקט ויכולות להיות באמצעות שיטות מתוחכמות של שחזור דיבור.

שונות בין הקודקים של נקודות הקצה בתקשורת היא גם אתגר שמופיע בשכיחות גבוהה, בעיקר במצבים בהם מערכות/מכשירים שונים רוצים לבצע תקשורת ביניהם. לשם כך, מערכות רבות משלבות תהליך של המרת קודק השמע (transcoding) בשביל לבצע המרה בזמן אמת בין הקודקים.

יתרה מזאת, קיימת בעיה בהערכת איכות שיחת השמע ואיכות מקורות השמע. מערכות בתעשייה משתמשות בפרוטוקול RTCP על מנת לאסוף סטטיסטיקות על השיחה ולהתאים את התנהגות המערכת. עם זאת, שימוש בפרוטוקול RTCP הינו מוגבל לעיתים כיוון שאם נתוני הרשת של מקור השמע אינם טובים, הפרוטוקול מספק את האינדיקציה על כך בלבד. כלומר הפרוטוקול לא מטפל בבעיה עצמה ולא תמיד מאפשר מניעה מוקדמת של הירידה באיכות השמע.

למרות שהעקרונות של שידור שמע בזמן אמת ידועים ונמצאים בשימוש נרחב בתעשייה אתגרים כגון התגברות על בעיות ברשת ה-IP, שונות בקודקים בין מערכות שונות וחוסר תיאום בין נקודות הקצה עדיין מהוות בעיות נפוצות עם השימוש בטכנולוגיה ומהוות בעיות הנדסאיות מובהקות.



2.4 אתגרים מרכזיים

2.4.1 הבעיה איתה התמודד התלמיד

הבעיה המרכזית איתה התמודדתי במהלך הכנת הפרויקט הייתה כיצד לתכנן מערכת אמינה ויעילה שתוכל להתמודד עם מצבי קיצון ברשת, תוך שמירה על תקשורת תקינה בזמן אמת. נדרשתי לפתח מערכת בעלת ביצועים גבוהים, המסוגלת לפעול בצורה אסינכרונית ומקבילית, כך שתוכל להתמודד עם שינויים בלתי צפויים ברשת, כגון אובדן פקטות ו-jitter. בנוסף, תכנון המערכת דרש התחשבות בהגדרות ובדרישות הנקבעות על ידי המשתמש בקובץ הקונפיגורציה, תוך מימוש מנגנונים לזיהוי בעיות רשת ולטיפול בהן בצורה יעילה. שילוב זה בין עמידה בדרישות המשתמש לבין התמודדות עם תנאי רשת מאתגרים היה חיוני כדי להבטיח שמערכת תוכל לספק ביצועים יציבים ושיחה חלקה גם בתנאים לא אידיאליים.

2.4.2 הסיבות לבחירת הנושא

בחרתי בנושא הפרויקט מכיוון שהוא מהווה יסוד חשוב בתחילת דרכי במסגרת ההכשרה שלי כמתכנת עתידי ביחידה. במהלך העבודה על הפרויקט, עסקתי וחקרתי טכנולוגיות מרכזיות שבהן היחידה מתמקצעת. חוויה זו תסייע לי בצעדים הראשונים שלי עם הגיוס ליחידה ותספק לי נחיתה רכה והשתלבות קלה יותר בעבודתי במסגרת פעילותה.

2.4.3 מוטיבציה לעבודה

במהלך לימודי בעתודה הטכנולוגית, התחלתי לחקור ולהעמיק בתחום הנדסת התוכנה. ככל שהתקדמתי בלימודים, המוטיבציה שלי להשתפר גדלה, ושאפתי להתקבל למסלול הפרויקטנטים, מתוך הבנה שזהו המסלול האופטימלי עבורי שיאפשר לי להתפתח מקצועית ואישיותית. לכן, כשקיבלתי את ההודעה על קבלתי למסלול, חשתי סיפוק רב והבנה שהשקעת המאמצים בלימודי השתלמה. במסגרת ביקורי ביחידה, נחשפתי לצוות שבו אשובץ ולמערכות המשמשות את המחלקה בפיתוח ובשימוש היומיומי. סקרנותי בנוגע לאופן העבודה במחלקה ובעיקר בנוגע לפרויקט גמר שאבצע במהלך שנת ההכשרה גברה בי מאוד ונהגתי להעלות שאלות רבות לצוות ולחקור לעומק את התחומים הקשורים הן על הפרויקט והן על אופן פעילות המחלקה. מתוך כך, הבנתי את חשיבות פעילות הצוות עבור חיל האוויר ואת המשמעות של הפרויקט להכשרתי כמתכנת ביחידה. מתוך עניין זה, הפרויקט נתפס בעיניי כהזדמנות ללמוד ולצבור ניסיון מעשי בתחום התוכנה, תוך התמודדות עם אתגרים טכנולוגיים אמיתיים אשר יש להם השלכות פרקטיות ותפעוליות. העבודה על הפרויקט סיפקה לי הזדמנות לממש פתרון לבעיה טכנולוגית אמיתית. בכל שלב ומטלה במהלך הכנת הפרויקט, תמיד שאפתי לא רק לבצע את הדרישות הניתנו במשימות אלא בנוסף להבין לעומק את אופן הפתרון ואם ניתן לבצע לו אופטימיזציה. תמיד חיפשתי דרכים ליעל ולשפר את הארכיטקטורה של הפרויקט מתוך רצון גם ליעל את הפרויקט וגם כדי להעמיק את הבנתי בתחום בו עוסקת המטלה ולהפיק את המירב מן תהליך הפתרון. תחושת הסקרנות והרצון להעמיק את הידע וההבנה שלי היוו המוטיבציה העיקרית לעבודתי בפרויקט והם אלה שהובילו אותי לבצע את העבודה על הצד הטוב ביותר. בכל שלב בהכנת העבודה, ציפיתי לשלב הבא בפרויקט ואיתו למכשולים החדשים שאצטרך להתגבר ולידע הנוסף שאוכל לרכוש במהלכו. בסופו של דבר, ראיתי את העבודה על פרויקט זה לא רק כמשימה אקדמית/מבצעית אלא גם בתור הזדמנות לקבלת כלים חשובים להמשך דרכי בעולם התוכנה ובשירותי ביחידה.



2.4.4 על איזה צורך הפרויקט עונה? איזה פתרון הפרויקט הזה בא לתת?

הפרויקט VTM עונה על הצורך בניהול, עיבוד וגישור יעיל של תקשורת שמע בזמן אמת בין רשתות שונות במיוחד בתנאי רשת מאתגרים ולא יציבים. ברשתות אלה, קיים הצורך בפיתוח מערכת אשר מסוגלת להתמודד עם מצבים מאתגרים ברשת הבאים לידי ביטוי באובדן פקטות, jitter ושיבושים נוספים תוך שמירה על איכות ואמינות השמע ועם הזרמת מידע בצורה מהירה. בנוסף, המערכת נותנת מענה לצורך בהתאמת קודקים שונים בין נקודות קצה שונות, כך שתתאפשר תאימות מלאה בין המשתמשים בפורמטים שונים של שמע ויהיה ניתן לבצע תקשורת שמע בין מכשירים ומקורות שונים. על ידי מימוש מנגנונים לניטור בזמן אמת של איכות התקשורת וסיפוק משוב סטטיסטי, הפרויקט מאפשר למשתמשים לזהות בעיות ולהתאים את התצורה והפעולה של המערכת, ובכך מבטיח חוויית שימוש מיטבית בתקשורת מדיה בזמן אמת.

2.4.5 הצגת פתרונות לבעיה

הפתרונות המוצעים לבעיה הם:

1. פיתוח מודל תקשורת מתקדם לתקשורת שמע בזמן אמת היאפשר ניהול שיחות זרמי שמע ממספר מקורות בו-זמנית, תוך שמירה על ביצועים גבוהים ופעולה אסינכרונית למניעת עיכובים.
2. מימוש מנגנון להמרת קודק שמע אשר יתמוך באלגוריתמים לקידוד ופענוח של קודקים שונים, ויאפשר התאמה בין פורמטים מגוונים של שמע. בכך תתאפשר זרימה תקינה של שמע גם במצבים שבהם נקודות הקצה משתמשות בקודקים שונים.
3. מימוש מנגנון להתמודדות עם תקלות ושיבושים ברשת אשר יזהה פקטות המגיעות בסדר שגוי או בתזמונים לא עקביים (jitter) ויבצע טיפול מהיר לסידור ועיבוד תקין שלהן, תוך שמירה על עיכוב מינימלי.
4. הכללת מנגנון חסימה של פקטות UDP שאינן עומדות בתקנים של פרוטוקולי RTP/RTCP כדי למנוע תקשורת משובשת.
5. יצירת ממשק המאפשר למערכת לפעול בהתאם להגדרות קובץ קונפיגורציה מוגדר מראש, מה שיאפשר התאמה לצרכים של המשתמש.
6. יישום מנגנון לשליחת לוגים בזמן אמת, לצורך מעקב אחר פעילות המערכת, ניתוח אחר ביצועים, וביצוע תחזוקה והתאמות ביעילות.



3. מטרות/יעדים

- פיתוח מנגנון להתמודדות עם מצבי קצה ברשת – כגון פקטות שמע שמגיעות באיחור, בסדר שגוי או ברווחי זמן לא עקביים. המנגנון יבטיח סידור מחדש של הפקטות לפי הסדר הנכון, שמירה על רציפות ואיכות השמע תוך מזעור של עיכובים ושמירת חוויית תקשורת תקינה.
- מימוש אלגוריתמים לקידוד ופענוח שמע המאפשרים המרה בין קודקים שונים, כגון G711 Alaw -i לצורך הבטחת תאימות מלאה בין נקודות קצה שאינן מתקשרות באותו קודק שמע. תמיכה בקודקים שונים מבטיחה תאימות בין מערכות שונות וצמצום אובדן איכות שמע במהלך ביצוע התקשורת.
- פיתוח תשתית תקשורת המבוססת על עקרונות של עבודה אסינכרונית ולא חוסמת במטרה לאפשר עיבוד ושידור של מספר שיחות מקבילות בו-זמנית.
- מימוש מנגנון לניתוח הפקטות המתקבלות באמצעות פרוטוקולי RTP/RTCP מעל UDP לצורך בקרה על איכות השידור. המנגנון יזהה פקטות שגויות או לא תקינות, ויסנן אותן על מנת לשמור על תקשורת יציבה, רציפה וללא שיבושים.
- אפשרות להגדרת קונפיגורציית המערכת באמצעות קובץ קונפיגורציה המאפשר למשתמש לקבוע פרמטרים כגון כתובות רשת, פורטים, קודקים ופרטי שיחות – ללא צורך בשינוי בקוד המקור.
- יצירת מנגנון תיעוד שוטף של פעילות המערכת אשר תתעד את פעילות המערכת בזמן אמת, כולל שגיאות, זרמים פעילים ומדדי איכות. מידע זה יסייע באיתור תקלות, ביצוע תחקור, ושיפור ביצועים לאורך זמן.



4. אתגרים

במהלך הכנת הפרויקט, נתקלתי באתגרים שונים שהתמודדות איתם תרמה רבות להתפתחותי האישית והמקצועית:

- 1. למידה עצמית:** במהלך הכנת הפרויקט, הייתי צריך ללמוד נושאים שונים אשר לא הכרתי לפני תחילתו. מעבר לידע התיאורטי שהייתי צריך לרכוש, נדרשתי גם ללמוד כיצד לכתוב קוד רחב היקף ויעיל. תחילה, דרישה זו הייתה מאתגרת עבורי, שכן לא התנסיתי בעבר בפרויקט מסוג כזה, אך ככל שהתקדמתי, פיתחתי דרכי חשיבה ועבודה יעילות שאפשרו לי להתמודד עם האתגרים. אמצעים אלו באו לידי ביטוי בלמידה של כתיבת קוד נקי, ניצול המאפיינים של שפת התכנות לטובת הפרויקט ושימוש בעקרונות תכנות מונחה עצמים לארגון הפרויקט למודולים בעלי תחום הגדרה ואחריות ברורים.
- 2. התמודדות עם חוסר ניסיון בפרויקטים בתוכנה:** בהיעדר ניסיון קודם בכתיבת פרויקטים, תחילת הפרויקט הייתה יותר מורכבת עבורי. לפני שהתחלתי את הפרויקט, הניסיון והידע שלי בתחום הנדסת תוכנה התבסס על החומר הנלמד במסגרת העתודה הטכנולוגית. לכן כשהתחלתי את הפרויקט, לא היה לי ניסיון מעשי או ידע בניהול פרויקטים. תחילה, יחד עם המנחה המבצעי, חילקנו את הפרויקט למשימות קטנות יותר, ותכננו ארכיטקטורות ראשוניות למשימות הראשונות בפרויקט. עם הזמן, צברתי ביטחון וניסיון בכתיבת קוד ובהובלת הפרויקט, דבר שהתבטא בשיפור מתמיד של הארכיטקטורה וביכולת לעבוד באופן עצמאי יותר.
- 3. ניהול זמן:** במהלך השנה, השילוב בין לימודים במסגרת העתודה, עבודה על הפרויקט, ועיסוקים אישיים, דרש ממני לפתח יכולות מתקדמות של ניהול זמן. עם ביצוע הפרויקט קבעתי לעצמי ביחד עם הליווי מהיחידה יעדים וזמנים בהם הייתי צריך לעמוד במסגרת הפרויקט. פעולה זו של קביעת לוחות זמנים, הקלה עליי בתכנון הפרויקט ואפשרה ביצוע מאורגן וממוקד של הפרויקט.
- 4. תיעוד הפרויקט:** אתגר משמעותי נוסף היה תיעוד והסבר הפרויקט, שכלל כתיבת הצעת פרויקט, ספר פרויקט, ותיעוד הקוד. תחילה, לא הייתה לי הבנה מעמיקה בנושאים אלו, ולכן למדתי כיצד לתעד פרויקט תוכנה בצורה נכונה, לרבות חשיבותו למשתמשים עתידיים. אחד הלקחים החשובים ביותר שצברתי במהלך העבודה הוא הערך של תיעוד מדויק ומפורט, והאופן שבו הוא משפיע על תחזוקת הפרויקט ושימוש בהמשך.



5. מדדי הצלחה למערכת

- ✓ המערכת מסוגלת לבצע המרה יעילה בין קודקים שונים כגון Speex-i G711 Alaw בהתאם לאלגוריתמי הקידוד והפענוח. ההמרה מתבצעת תוך שמירה על איכות שמע גבוהה, ובכך מבטיחה תאימות מלאה בין נקודות קצה שונות
- ✓ המערכת מזהה מצבי קיצון ברשת באמצעות ניטור רציף של תעבורת הנתונים והבחנה בשיבושים או שינויים חריגים. היא מגיבה מיד למצבים אלה, מה שמבטיח העברת שמע חלקה וללא השהיות או איבודי מידע משמעותיים, ובכך מספקת תקשורת רציפה וברורה גם בתנאי רשת קיצוניים.
- ✓ המערכת נבדקה תחת עומס ועמדה בהצלחה בהפעלת מספר זרמי שמע ושיחות מקבילות, תוך ניצול מיטבי של משאבי המעבד והזיכרון. תהליך זה מבוצע ללא חסימות מיותרות או ירידה באיכות ההאזנה.
- ✓ ניתן להגדיר את אופן פעולת המערכת דרך קובץ קונפיגורציה, מה שמאפשר לבצע שינויים והתאמות ללא צורך בשינוי בקוד המקור.
- ✓ המערכת פועלת על פי תקני VoIP מוכרים, תוך שימוש בפרוטוקולים RTP, RTCP. עמידה בתקן מבטיחה אינטגרציה עם רכיבי תקשורת נוספים, יציבות בפעולה, והפחתת סיכון לתקלות.
- ✓ במהלך הריצה, המערכת מייצרת לוגים מפורטים אודות פעולתה, כולל אירועים חריגים, סטטיסטיקות שיחה ופרטי מערכת. מידע זה מאפשר מעקב בזמן אמת, וכן תחקור ושיפור ביצועים לאורך זמן.



6. רקע תיאורטי/ספרות מקצועית

PCM

Pulse Code Modulation היא שיטה המשמשת להמרת אותות אנלוגיים לרצף של ערכים דיגיטליים, כך שניתן יהיה לאחסן, לעבד או לשדר את האות בצורה דיגיטלית – מבלי לאבד מידע משמעותי.

המרת האות האנלוגי לדיגיטלי מתבצע ב-3 שלבים:

1. **Sampling**: בשלב זה, המערכת מודדת את ערך האות האנלוגי במרווחי זמן קבועים. תדירות הדגימה, הנמדדת ביחידות הרץ (Hz), מציינת כמה פעמים בשנייה מתבצעת המדידה. למשל, קצב דגימה של 8kHz משמעותה היא 8000 דגימות שמע בשנייה.
2. **Quantization**: לאחר ביצוע הדגימה, כל ערך נמדד מעוגל לרמות בדידות קבועות מראש. פעולה זו נדרשת מכיוון שאותות אנלוגיים הם רציפים, אך בעולם הדיגיטלי ניתן לעבוד רק עם מספר סופי של ערכים. רמת הדיוק מוגדרת על ידי עומק הסיביות (bit depth). למשל, עומק של 16 ביט מאפשר 65,536 רמות ייצוג שונות. ככל שעומק הסיביות גדול יותר, איכות השמע טובה יותר, אך גם נדרש יותר מקום לאחסון הנתונים.
3. **Encoding**: כל ערך בדיד שקיבלנו משלב 2, מקבל ייצוג בינארי. מספרים בינאריים אלו יכולים לשמש לאחסון, עיבוד או שידור של האות באמצעים דיגיטליים.

PCM הוא הבסיס לכל תהליך עיבוד השמע במערכת. הפרויקט עוסק בקידוד, דחיסה ושידור של אותות שמע על גבי רשתות IP, וכל אות שמע שעובר במערכת – מקורו בפורמט PCM.

Codec

כפי שתואר בחלק הקודם, שיטת PCM ממירה אותות שמע אנלוגיים לערכים דיגיטליים. עם זאת, קבצי שמע בפורמט PCM דורשים נפח אחסון גדול וצריכת רוחב פס גבוהה. כתוצאה מכך, מתעורר קושי בשימוש בפורמט PCM כאשר מדובר בשידור מדיה על גבי רשתות IP ככלל, ובתקשורת VoIP בפרט. כדי להתגבר על קושי זה, נעשה שימוש בקודקים.

קודק הוא רכיב תוכנה או חומרה המבצע קידוד (Compression) ופענוח (Decompression) של נתוני מדיה דיגיטליים, כגון שמע או וידאו. תפקיד הקודק הוא לדחוס את המידע לגודל קטן יותר על מנת לאפשר שידור מהיר ויעיל יותר ברשתות תקשורת או לצורך חיסכון במשאבי אחסון, מבלי לפגוע משמעותית באיכות המידע.

תהליך פעולת הקודק מחולק לשני שלבים עיקריים:

- **קידוד**: המרת האות הדיגיטלי הגולמי לפורמט דחוס באמצעות אלגוריתם קידוד. המטרה היא להקטין את נפח הנתונים תוך שמירה על איכות שמע גבוהה ככל האפשר.
- **פענוח**: תהליך הפוך בו מפענחים את הנתונים הדחוסים ומשחזרים מהם את שמע הקרוב ככל האפשר למקור.

חשיבות השימוש בקודקים בתקשורת זמן אמת:

- **שיפור יעילות העברת הנתונים ברשתות IP**: קודקים מאפשרים להעביר שמע באיכות גבוהה תוך חיסכון משמעותי ברוחב פס, מה שקריטי במיוחד בסביבות IP עם מגבלות רשת.



- **חיסכון בשטח אחסון:** תכני מדיה דחוסים תופסים פחות מקום, מאפשרים אחסון יעיל יותר ומפחיתים את עלויות המשאבים.

סוגים של קודקים:

1. **Lossless Codecs:** מבצעים דחיסה מבלי לאבד מידע כלל. מתאימים למצבים בהם יש חשיבות לשחזור מדויק של המידע המקורי, אם כי קובצי הפלט גדולים יחסית.
2. **Lossy Codecs:** מבצעים דחיסה תוך ויתור מבוקר על מידע "פחות משמעותי" בעיני האוזן האנושית. גישה זו מאפשרת חיסכון קיצוני בנפח הקובץ ומותאמת במיוחד לשידור בזמן אמת.

מאפיינים עיקריים של קודקים הנמצאים בשימוש במערכת:

1. **Sampling rate:** קצב דגימת האות האנלוגי. נמדד ביחידות הרץ (Hz). ככל שהקצב גבוה יותר הייצוג הדיגיטלי של אות השמע מדויק יותר.
2. **Bitrate:** כמות הביטים המועברים בשנייה. נמדד ביחידות ביטים לשנייה (bps). ככל שה-bitrate גבוה יותר משפר את איכות השמע אך דורש יותר רוחב פס.

במסגרת פרויקט זה נעשה שימוש בשני סוגי קודקים עיקריים, שכל אחד מהם מייצג גישה שונה לדחיסת שמע:

1. **G711:** קודק המשמש לשידור דיגיטלי של אותות קול אנלוגי דרך הרשת. הוא נמצא בשימוש נרחב במערכות טלפוניה ותקשורת, ומספק דחיסה ופירוק נתונים יעילים ואמינים של נתוני קול.
קודק זה הוא מסוג Lossless אשר פועל בקצב סיביות קבוע של 64kbps.
קודק טלפוני קלאסי, אשר מבצע דחיסה באמצעות טכניקת **Companding**. מתאים במיוחד לרשתות שבהן רוחב הפס אינו מהווה מגבלה חמורה.
עבור קודק זה קיימות 2 גרסאות:
a. **A-law:** תקן בו נעשה שימוש באירופה ובאסיה.
b. **U-law:** תקן בו נעשה שימוש בארצות הברית, יפן ואזורים נוספים.
בפרויקט זה נעשה שימוש בגרסת A-law.
2. **Speex:** קודק שמע קוד פתוח, המיועד לקידוד ודחיסה של אודיו ודיבור.
קודק מאוד גמיש התומך במגוון רחב של איכות שמע ובקצב סיביות משתנה אשר יכול לקודד ברוחב פס צר וברוחב פס רחב.
קודק זה הוא מסוג Lossy אשר פועל בקצב סיביות משתנה בטווח של 2kbps עד 24kbps.
קודק זה מבצע את הדחיסה באמצעות טכניקת **Code Excited Linear Prediction (CELP)**.



Compadding

הקודק G711 פועל על פי שיטה הנקראת Compadding – שילוב של המילים Compressing (דחיסה) ו-Expanding (הרחבה). עיקרון זה מאפשר קידוד יעיל של אותות שמע בפורמט דיגיטלי.

שיטה זו עובדת ב-2 שלבים: בצד השידור אות השמע עובר דחיסה לא לינארית המפחיתה את טווח הדינמיקה של עוצמת הקול, בזמן שבצד הקליטה מתבצעת הרחבה לא לינארית המשחזרת את האות המקודד בקירוב למקורו.

עיקרון השיטה מבוסס על תכונה מערכת השמיעה האנושית: האוזן האנושית רגישה יותר לשינויים בעוצמות נמוכות מאשר לשינויים בעוצמות גבוהות. לדוגמה, שינוי קטן בעוצמת דיבור שקט מורגש מאוד באוזן, לעומת שינוי באותה מידה בעוצמת דיבור רם – שכמעט ואינו מורגש.

לכן, שיטת Compadding מקצה דיוק גבוה יותר בקידוד של אותות חלשים לעומת קידוד של אותות חזקים יותר אשר מקודדים בצורה גסה יותר. ניתן להקטין את מספר הסיביות הנדרש לקידוד כל דגימת שמע, מבלי לפגוע בתחושת האיכות של המשתמש.

ביצוע פעולת הקידוד מבוססת על פונקציית לוגריתם אשר משמשת לדחיסה של ערכי האות הקולי בצד השידור. פונקציה זו היא אינה לינארית – ככל שערכי הקלט שמתקבלים בה נמוכים יותר (עוצמת אות השמע קטנה יותר) כך הם נדחסים ברמת דיוק גבוהה יותר בזמן שערכי קלט חזקים יותר (עוצמת אות השמע גבוהה יותר) נדחסים ברמת דיוק נמוכה יותר. כתוצאה מכך, היא מאפשרת לשמר רמות דיוק גבוהות במקום שבו זה קריטי, ולוותר על דיוק מיותר במקום שבו זה כמעט ולא מורגש.

CELP

הקודק Speex מבוסס על אלגוריתם קידוד קול בשם Code Excited Linear Prediction. אלגוריתם CELP הוא שיטת קידוד לדיבור שמטרתו היא לבצע דחיסה של הנתונים ולאפשר לשדר אותם ברשת ברוחב פס נמוך תוך שמירה על איכות שמע גבוהה ככל האפשר.

לאלגוריתם זה יש מספר עקרונות עליהם הוא מבוסס:

1. **Source-Filter Model: CELP** מתבסס על רעיון שמדמה את הדרך בה הקול האנושי נוצר. הקול האנושי מיוצר ממקור (מיתרי הקול) שמפיק צליל גולמי וממערך של מסננים (הפה, הלשון, האף) אשר מעצבים את הצליל. בהתאם לכך, האלגוריתם מפריד בין המידע המייצג את מקור האות, לבין המידע הספקטרלי המעצב אותו.
2. **Linear Prediction (LPC): CELP** משתמש בחיזוי לינארי כדי לחזות את הדגימה הבאה על סמך דגימות העבר. כתוצאה מכך הקודק לא משדר את הקול עצמו אלא את ההפרש בין אות הקול הנוכחי לבין המידע שחוזר, מה שחוסך עיבוד מידע.
3. **Pitch Prediction**: מאחר שהדיבור האנושי כולל תבניות מחזוריות (במיוחד בתנועות קוליות), האלגוריתם מזהה מחזורים בקול ומשתמש בהם לחיזוי הדגימות הבאות, תוך שיפור הדחיסה והפחתת שגיאות.
4. **Innovation Codebook**: כמובן שלא כל צליל ניתן לחיזוי מלא. עבור כל אות קול שהאלגוריתם לא מצליח לזהות, האלגוריתם בוחר את "חדשנות" מתוך ספריית תבניות מוכנה מראש (codebook). תוספת זו נועדה לשחזר את מרכיבי הקול שאינם ניתנים לחיזוי ישיר באמצעות LPC או pitch prediction.



5. **Noise Weighting**: האלגוריתם בנוסף לדחיסת המידע, מנסה גם לכוון את מיקום הרעש לתדרים שהאוזן האנושית פחות רגישה אליהם.
6. **Analysis-by-Synthesis (AbS)**: במקום לבדוק רק איך לקודד את הקול בצורה מתמטית, האלגוריתם גם בודק כל אפשרות של קידוד לפי איך היא תישמע לאחר פענוח. זוהי לולאה סגורה שבה כל אפשרות קידוד נבחנת לפי כמה טוב היא משחזרת את הקול. כך מתקבלת איכות שמע גבוהה – מבלי לשדר את כל המידע.

Transcoding

- תהליך שבו מדיה דיגיטלית (כגון שמע או וידאו) מומרת מקידוד בקודק אחד לקודק אחר. בתהליך זה, המידע עובר שני שלבים עיקריים:
1. פענוח (Decoding): המרה מהפורמט הדחוס לייצוג גולמי ולא דחוס PCM.
 2. קידוד מחדש (Encoding): המרה מחדש של המידע לפורמט הדחוס של קודק היעד.
- מנגנון זה חיוני לגישור בין מקורות שמע שמשתמשים בקודקים שונים, ומאפשר קיום תקשורת תקינה בין התקנים ומערכות שונות.
- יתרה מזאת, למנגנון זה יש תפקיד משמעותי ונוסף – התאמה של קצב הסיביות ליכולת הרשת. כאשר ישנה תקשורת בין קודק בעל רוחב פס גבוה לבין קודק חסכוני, תהליך ה-Transcoding למעשה מבצע הקטנה/הגדלה של קצב הסיביות כך שהפלט יוכל להתאים ליכולות המערכת או רשת היעד.
- בפרויקט, ממומש עבור כל קודק הנתמך על ידו, אלגוריתמי הקידוד והפענוח שלהם.
- עבור G711 A-Law תהליך הקידוד והפענוח מתבצע על ידי מימוש וחיפוש בתוך טבלאות Lookup tables (LUT). שיטה זו מאפשרת חישוב מהיר של ערכי ההמרה בין PCM ל-G711 ולהיפך עם צריכת משאבים מינימלית - חיוני במיוחד בשידור קול בזמן אמת.
- טבלאות אלה מכילות בתוכן את כל הערכים האפשריים שאפשר לייצג ב-8 ביט ערך Alaw ואת כל הערכים האפשריים שאפשר לייצג ב-16 ביט ערך PCM.
- עבור speex, תהליך הקידוד והפענוח מתבצע על ידי שימוש בספריית קוד פתוח libspeex. זוהי ספרייה שנכתבה בשפת C המספקת יישום של קודק השמע Speex.
- הספרייה מספקת קידוד ופענוח יעילים של דיבור צר ופס רחב, מה שהופך אותה לאידיאלית עבור יישומי VoIP והזרמת אודיו. הספרייה מספקת יישום מלא של אלגוריתם CELP.
- בפרויקט, הספרייה שולבה על ידי עטיפה (Wrapping) של הפונקציות של libspeex על ידי מחלקה ייעודית, תוך הקפדה במימוש על עקרונות RAIL ויישום של Rule of 5 - לניהול נכון של משאבי זיכרון ותחזוקה בטוחה של המשאבים. באמצעות שילוב זה, מתאפשר שימוש בטוח ונוח בספרייה חיצונית בסביבת פיתוח מודרנית של C++.



QoS

Quality of Service הוא מונח המודד את היכולת של הרשת לספק שירותים באיכות גבוהה למשתמשי קצה. המטרה העיקרית של מנגנוני QoS היא לתת עדיפות גבוהה יותר ליישומים קריטיים כמו למשל שידורים ותקשורת בזמן אמת. טכנולוגיה זו שואפת לשפר ביצועים, להפחית את ה-latency ולהפחית אובדן פקטות מידע.

ישנם פרמטרים שונים עליהם יש התייחסות בטכנולוגיית QoS:

1. **Packet loss**: מצב של אובדן פקטות המתרחש כאשר יש עומס בתעבורת הרשת.
2. **Jitter**: פרמטר המתייחס לשונות הגעת הפקטות, ובא לידי ביטוי במצבים בהם הפקטות נמסרות אל יעדן במרווחי זמן משתנים. פרמטר זה חיוני למערכות זמן אמת מכיוון שהוא עלול לפגוע בקליטת השמע אצל היעד. זמן זה נמדד ב-milliseconds.
3. **Latency**: מדד המתאר את כמות הזמן העוברת בהעברת המידע מן המקור ליעד. במערכות זמן אמת על מדד זה להיות נמוך כדי להימנע מאיכות ירודה בהעברת המדיה. זמן זה נמדד ב-milliseconds.
4. **Bandwidth**: מדד המתייחס ליכולת השידור של הרשת, כלומר כמה נתונים ניתן להעביר בפרק זמן נתון. מעריכים מדד זה על פי כמות הביטים המקסימלית שניתן להעביר בשנייה.

הפרויקט מנתח באופן רציף את מדדי ה-QoS באמצעות ניתוח פקטות RTCP ומבצעת התאמות דינמיות כדי להבטיח זרימת שמע רציפה ואמינה גם בתנאי רשת לא יציבים.

Jitter buffer

זהו מנגנון המהווה מאגר זמני של אחסון פקטות מדיה המשמש במערכות זמן אמת. באפר זה נועד לטפל בהשפעות הנגרמות עקב מצבים של jitter ברשת, אשר גורמות שיבושים בתקשורת הבאים לידי ביטוי בקבלת הפקטות באיחורים ותזמונים לא צפויים ובסדר שונה ממה שהן נשלחו במקור.

במהלך פעילותו, ה-buffer קולט את הפקטות, מסדר אותן בסדרן המקורי ושולח אותן בתזמונים זהים, על מנת שאצל נקודת היעד יופחת מצב ה-jitter במהלך השמעת המידע.

ישנם 2 סוגים של jitter buffer:

1. **Fixed size jitter buffer**: אלגוריתם בו נקבע גודל קבוע עבור הבאפר, ועל כן מגדיר מספר מקסימלי קבוע של פקטות הניתנות לאחסון. זהו האלגוריתם הכי פשוט לשימוש אך עם זאת הוא אינו מסתגל לתנאי הרשת הנתונה שעלולים להשתנות.
2. **Adaptive jitter buffer**: אלגוריתם המתאים את גודל הבאפר בצורה דינאמית בהתבסס על תנאי הרשת. שינוי הגודל יכול לבוא לידי ביטוי למשל אם ה-latency הוא גבוה אז ניתן להגדיל את גודל הבאפר בכדי להפחית את הסיכויים לאובדן פקטות או שאם תנאי הרשת יציבים אז ניתן להפחית את גודלו כדי להימנע מעיכובים מיותרים בשליחה.

בהכנת הפרויקט, נעשה מימוש עבור Adaptive jitter buffer, כאשר במהלך פעילות הבאפר נעשה שימוש במספר טכניקות נוספות:



- **Delay histogram:** במהלך פעילותה, המנגנון שומר בתוכו את זמני ההגעה של הפקטות לבאפר. באמצעות שמירה וניתוח של התפלגות זמני ההגעה של הפקטות, מחושב באופן שוטף ערך ההשהיה האופטימלי הדרוש לבאפר באופן שמקטין את אובדן הפקטות מבלי ליצור השהיות מיותרות.
- **Hysteresis:** מנגנון ייצוב אשר מונע תגובות קיצוניות לשינויים קיצוניים וזמניים ב-jitter. כאשר מתרחשת קפיצה פתאומית במספר הפקטות המאוחרות, מתווסף מקדם עונשין (Penalization Term) שמאט את קצב השינויים בגודל ההשהיה. מטרתו היא לשמור על חוויית שמע יציבה גם כאשר תנאי הרשת אינם יציבים.
- **Interpolation:** כאשר לא ניתן להוציא פקטה מתוך הבאפר בזמן נתון, למשל עקב הקטנת השהיית הבאפר או עיכוב לא צפוי, מופעל מנגנון Interpolation. מנגנון זה מזהה את כמות השמע שעל המערכת להשלים באופן סינתטי לצורך סנכרון המערכת עם ה-timestamps של הפקטות מבלי להשאיר חורים בשמע אצל המאזינים.

באמצעות ה-jitter buffer, המערכת יודעת להתמודד עם שינויים קיצוניים ב-QoS, לאזן בין השהיית שמע מינימלית לבין שמירה על רציפות השיחה, ולאפשר תקשורת קולית חלקה גם כאשר תנאי הרשת משתנים בצורה דינמית.

PLC

בתקשורת זמן אמת מבוססת רשת, כמו שיחות שמע ב-VoIP, אין אפשרות להמתין להעברה חוזרת של פקטות שאבדו. כל עיכוב או אובדן פקטות פוגע ישירות ברציפות השיחה, ויוצר אצל המשתמש תחושות של "קפיצות" או "חורים" בשמע.

כאשר קצב הגעת הפקטות נעשה לא יציב, או שמתרחש אובדן חלקי או מלא של פקטות, הנגן בצד המקבל נתקל בקטעים חסרים – מה שמוביל לשיבושים, רעשים לא טבעיים והשהיות שפוגעות קשות באיכות ואמינות התקשורת.

לכן, בשביל להתגבר על בעיה זו, קיים המנגנון Packet Loss Concealment. מטרתו של המנגנון הוא להשלים על אובדן הפקטות באמצעות יצירה של פקטות סינתטיות שישמר את רצף השמע בצורה אמינה ורציפה.

בפרויקט נעשה מימוש של מנגנון זה - כאשר מזהה חוסר בפקטת שמע בנקודת זמן מסוימת, המערכת יוצרת פקטת שקט סינתטית במקום הפקטה החסרה. באמצעות שימוש ב-PLC ויצירת פקטות שקט, המערכת מצליחה לשמר זרם שמע יציב ורציף גם בתנאי רשת בעייתיים תוך שמירה על תחושת שיחה טבעית.

חשוב לציין כי פקטות של שקט הן לא פקטות חסרות תוכן, אלא נתוני שמע עם ערכים מאופסים שמונעים "הפרעות" בשמע.



7. תיאור המצב הקיים

במצב הנוכחי בחיל האוויר, ניהול תעבורת שמע מתבצע באמצעות רכיב רשת ייעודי לסיון פקטות שמע, אשר מיועד לטיפול באותות אנלוגיים בלבד.

רכיב זה, מסוגל לסנן פקטות פגומות, אך אינו מותאם למערכות תקשורת דיגיטליות מודרניות הפועלות בפרוטוקולי זמן אמת.

השימוש ברכיב הקיים יוצר מספר בעיות משמעותיות:

- ☒ נדרש לבצע המרה כפולה של האותות: מדיגיטלי לאנלוגי לטובת הסיון, ואז בחזרה לדיגיטלי לצורך המשך השידור.
- ☒ תהליך זה יוצר עיכובים מיותרים ומכביד על התקשורת.
- ☒ חוסר ההתאמה שלו לפרוטוקולי זמן אמת, עלול לגרום אף לחסימה של פקטות תקינות, ולפגוע ישירות באיכות השיחה – גם כאשר אין שיבושים בזרם השמע.
- ☒ הרכיב הקיים מאפשר טיפול בעד 30 ערוצי תקשורת בלבד.
- ☒ הגדלת כמות הערוצים דורשת הוספת ממירים וחומרה נוספת, מה שמוביל להגדלת העלויות ולמורכבות תחזוקתית שהולכת וגוברת עם הגדלת השימוש.
- ☒ הרכיב אינו כולל מנגנונים להתמודדות עם בעיות רשת דוגמת packet loss/jitter/transcoding מה שמחייב שימוש נוסף בפתרונות חיצוניים.



8. ניתוח חלופות מערכת

בחלק זה נשווה בין המערכת המוצעת VTM, לבין הרכיב הקיים הנמצא בשימוש המתואר בחלק של "תיאור המצב הקיים":

יתרונות	הרכיב הקיים	VTM
	- פתרון קיים ומוטמע בשטח. - מסוגל לנתח פקטות ברמת נמוכה יותר (אנלוגית מאשר דיגיטלית).	- מותאם לתקשורת מבוססת IP מודרני. - תומך בפרוטוקולי זמן אמת. - מבצע עיבוד דיגיטלי מלא ללא המרות כפולות. - גמישות להתרחבות ולשדרוג.
חסרונות	- אינו מותאם לסביבות זמן אמת. - דורש המרה מאנלוגי לדיגיטלי וחזרה, מה שיוצר עיכובים. - מוגבל במספר ערוצי השמע שהוא יכול לטפל בהם. - עלול לסנן פקטות שמע חוקיות עקב חוסר הבנה של פרוטוקולי זמן אמת.	צורך בכוח עיבוד גבוה לעיבוד בזמן אמת.
הזדמנויות	שימוש במערכת קיימת כחלק מפרויקטים מעורבים אנלוגי-דיגיטלי.	- הרחבה לשימוש עבור עשרות/מאות ערוצי תקשורת ללא תלות ברכיבי חומרה חיצוניים. - שילוב פשוט בסביבות תקשורת מודרניות, מבצעות עתידיות. - שיפור חוויית משתמש באיכות שיחה בזמן אמת.
איומים	- טכנולוגיה שאינה מתאימה למגמת השוק. - בעיות תחזוקה עם הגברת השימוש ברכיב.	- תלות באמינות הרשת. - דרישה מתמדת לניטור מדדי QoS.



9. תיאור החלופה הנבחרת

הפתרון שנבחר היא מערכת VTM - פיתוח מערכת דיגיטלית לניהול תעבורת שמע בזמן אמת ברשתות IP, תוך תמיכה בפרוטוקולי זמן אמת ועיבוד מתקדם של נתוני שמע באמצעות מנגנוני transcoding, PLC, adaptive jitter buffer.

נימוקים עבור החלופה הנבחרת:

- ✓ **התאמה לטכנולוגיות תקשורת מודרניות:** בניגוד לרכיב האנלוגי הקיים, מערכת VTM פועלת ישירות מול פרוטוקולי זמן אמת ומייטרת את הצורך בהמרות כפולות.
 - ✓ **שיפור מהותי באיכות השמע:** שילוב מנגנוני PLC, adaptive jitter buffer מאפשר שמירה על רציפות שמע גבוהה גם בתנאי רשת לא יציבים, תוך הפחתת עיכובים ואובדן מידע.
 - ✓ **יכולת גידול והתרחבות:** המערכת בנויה באופן מודולרי ואסינכרוני, מה שמאפשר בקלות תמיכה בעשרות/מאות ערוצי שמע בו-זמנית ללא צורך בהוספת חומרה נוספת.
 - ✓ **התמודדות עם מצבי קצה:** ניתוח ובקרת QoS בזמן אמת מאפשרים למערכת להגיב לשינויים דינמיים ביציבות הרשת ולשפר את חוויית השמע עבור המשתמש.
 - ✓ **חיסכון בעלויות תחזוקה ושדרוג:** ביצוע תהליכי הסינון והעיבוד לתוכנה מצמצמת את הצורך בתחזוקת חומרה נוספת, ומאפשרת עדכונים והרחבות באמצעות שדרוגי תוכנה בלבד.
 - ✓ **מוכנות לשילוב בפרויקטים עתידיים:** תכנון המערכת מתחשב באינטגרציה עתידית עם מערכות נוספות, באמצעות הרחבת פרוטוקולים, קודקים ותצוגות משתמש.
- החלופה הנבחרת מציעה פתרון טכנולוגי מתקדם, יעיל וגמיש הרבה יותר מן הפתרון הקיים, ומותאמת לצורכי השטח והמערכת הצבאית בעידן הדיגיטלי.



10. אפיון המערכת שהוגדרה / מוצעת

10.1 ניתוח דרישות המערכת

דרישות פונקציונליות – דרישות המגדירות מה המערכת צריכה לעשות בפועל, כלומר הפעולות והשירותים שהיא מספקת למשתמש או לרכיבים חיצוניים:

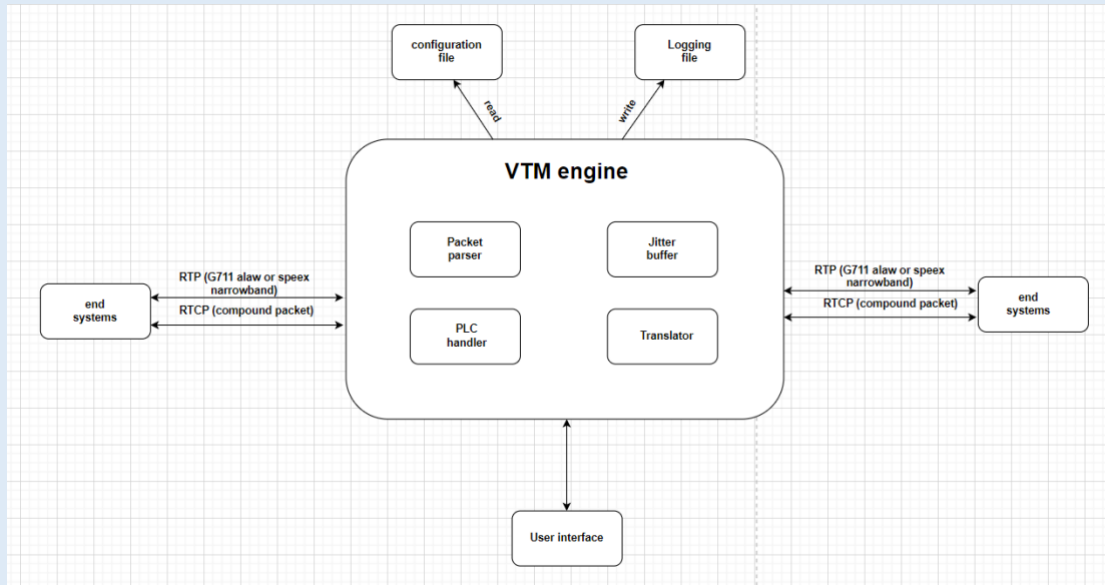
- המערכת תקבל פקטות שמע בפרוטוקול RTP, תבצע עיבוד בזמן אמת ותעביר את הפקטות לנקודת היעד, תוך שמירה על רציפות השמע ואיכות הנתונים.
- המערכת תאפשר ניהול וריבוי שיחות ומשתתפים בו-זמנית, ללא חסימות או פגיעה באיכות.
- לצורך התמודדות עם שונות בזמני הגעת פקטות (Jitter) המערכת תממש מנגנון באפר דינאמי (Adaptive jitter buffer) אשר יתאים את עצמו בזמן אמת לפי תנאי הרשת, על מנת להבטיח סנכרון תקין של השמע המתקבל. בנוסף, המערכת תמדוד ותנתח את ביצועי השמע באמצעות פרוטוקול RTCP כולל מטריקות כגון jitter, packet loss, latency ובהתאם לנתונים אלו, יתבצעו התאמות בזמן אמת.
- המערכת תבצע המרה בין קודקי שמע שונים (Transcoding), כגון G711 ו-Speex על מנת לאפשר תאימות בין מכשירים המשתמשים בקודקים שונים. ההמרה תתבצע בזמן אמת ותשמור על איכות שמע אופטימלית.
- המערכת תתעד את הפעולות הקריטיות, חריגות, תקלות ומדדים סטטיסטיים בקובץ ייעודי, לצורך ניתוח תקלות, איתור בעיות ואופטימיזציה עתידית.

דרישות לא-פונקציונליות – מתייחסות לאופן שבו המערכת צריכה לבצע את פעולותיה: באיזו רמת ביצועים, יציבות, אבטחה ותחזוקה.

- זמן ההשהיה הכולל (End-to-End Latency) בשיחה חייב לעמוד בתקנים המקובלים ב-VoIP בתנאים תקינים
- המערכת תזהה אובדן פקטות ברשת ותנקוט פעולות למזעור ההשפעה על איכות השמע באמצעות מנגנון של PLC (packet loss concealment)
- תכנון המערכת יתבצע כך שניתן יהיה להרחיב אותה בקלות – גם מבחינת מספר שיחות בו-זמנית וגם מבחינת רכיבים פונקציונליים – ללא צורך בשינוי משמעותי בקוד הקיים.
- הקוד ייכתב לפי עקרונות תכנות מונחה עצמים (OOP) תוך הפרדה ברורה בין מודולים, מה שיאפשר הוספת פיצ'רים, תיקון באגים או שדרוגים מבלי להשפיע על כלל המערכת.
- המערכת תנצל את משאבי החומרה בצורה אופטימלית והיא תעבור בדיקות עומסים כדי לוודא עמידות בתרחישים של ריבוי חיבורים במקביל, תנאי רשת בעייתיים או שידורים ממספר מקורות בו זמנית.



10.2 מודול המערכת



Packet parser: מודול המבצע ניתוח של הפקטות המתקבלות הן מפרוטוקול RTP והן מפרוטוקול RTCP לפי התקנים המוגדרים ב-RFC3550. עבור פקטת RTP, הוא מחלץ את שדות ה-header ומפריד את ה-payload תוך בדיקה של תקינות הפקטה. עבור פקטת RTCP, הוא מזהה את סוג הפקטה ומחלץ ממנה נתונים סטטיסטיים ונתוני חיווי השיחה. הפלט מן המודול הוא אובייקט פקטה המשמש כממשק נוח לקבלת ערכי השדות בפקטה.

Jitter buffer: באפר הנועד להתמודד עם תנודות רשת ולקבל החלטות בזמן אמת מתי לשחרר פקטת שמע להמשך העיבוד. הוא עושה זאת על ידי אחסון הפקטות בבאפר דינאמי תוך שקלול זמני ההגעה, חותמת זמן ומידע מפרוטוקול RTCP.

במהלך ריצתו הוא מחשב באופן מחזורי את זמן ההשהיה האופטימלי באמצעות זמני ההגעה של הפקטות ואם יש אובדן או שאין בזמן הנוכחי פקטה מתאימה להוצאה, יופעל מנגנון PLC.

PLC handler: מודול המטפל באובדן פקטות ברשת בזמן אמת על ידי יצירת פקטות שקט סינתטיות במקום הפקטות החסרות על ידי קבלת מידע על ערכי ה-header שבפקטה בנוסף לכמות השמע הרצוי בפקטה.

Translator: מודול האחראי להמרת הקודקים בהתאם לקודק שבו השתמש המקור לעומת ליעד כפי שמוגדר בקובץ קונפיגורציה. הוא משתמש באלגוריתמי הקידוד והפענוח של הקודקים לביצוע פענוח ולאחר מכן קידוד של ה-payload. במקביל, הוא מעדכן גם ה-header של ה-RTCP בהתאם לצורך. המודול מהווה את גשר הקידוד בין שני צדדים שונים של שיחה.

End systems: אלו הם התקנים חיצוניים (למשל מערכות טלפוניה או מכשירי VoIP) אשר שולחים ומקבלים פקטות RTP ו-RTCP דרך רשתות IP. המערכת מאזינה לפקטות המגיעות מן ה-end systems ומשדרת ליעדן את זרמי השמע בהתאם לקונפיגורציה. חשוב לציין, כי למערכת אין שליטה על אותן המקורות, אלא היא רק מגשרת ומעבדת את הפקטות.



קובץ קונפיגורציה: קובץ חיצוני המכיל את כל פרטי התצורה של המערכת. מכילה בתוכה מידע עבור תצורת השיחות, הגדרות הלוגים, והתקשורת עם הממשק משתמש. המערכת קוראת את הקובץ פעם אחת באתחולה באמצעות מבנה הנתונים property tree ונטענת ושולפת ממנו מידע בזמן ריצתה.

ממשק משתמש: ממשק ידידותי למשתמש המציג בזמן אמת את מצב השיחות, מידע על כל מקור ומדדי QoS. המנוע של המערכת שולח עדכונים בפורמט דרך WebSocket ל-UI.

קובץ לוגים: קובץ שמטרתו הוא לתעד את פעילויות המערכת על פי רמות סיווג שונות, לצורך ניתוח עתידי ותחקור תקלות. בזמן ריצה, כל אירוע משמעותי נכתב לקובץ לוג טקסטואלי באמצעות הספרייה boost log.

10.3 אפיון פונקציונלי

אחסון וניהול פקטות בתוך adaptive jitter buffer: המערכת מאחסנת את פקטות השמע בתוך באפר אדפטיבי שמטרתו היא להחליק תנודות בזמן הגעת הפקטות, לסדר מחדש את פקטות השמע במקרי קיצון ברשת ולהבטיח את רציפות שידור השמע גם במקרים בהם יש אובדן פקטות. המנגנון מתעדכן באופן מחזורי על ידי חישוב זמן ההשהיה של הפקטות באופן דינמי והיא מתבססת גם על פי הדיווחים מפרוטוקול RTCP.

קידוד מחדש בין קודקי שמע: לאחר הוצאת הפקטה מן ה-jitter buffer, המערכת מבצעת תהליך של transcoding הכולל פענוח של הפקטה לפורמט PCM ולאחר מכן קידוד מחדש אל הקודק של נקודת היעד. פעולה זו מאפשרת גישור בין מערכות קצה אשר משתמשות בקודקים שונים, ומאפשרת תקשורת חלקה ללא תלות בפורמט קלט.

ניהול שיחות שמע: עבור כל מקור ושיחת שמע המתרחשת במערכת, נשמר עבורם מידע בתוך המערכת מה שמאפשר ניהול נפרד שלהם. עבור כל מקור שמע המנסה להזרים מידע למערכת, יבדק אם הוא משתתף בשיחה פעילה ולאחר מכן יבדק אם הוא מופיע בקובץ קונפיגורציה במערכת. עבור כל שיחה המערכת אחראית על הקמתה עם השמת נקודות הקצה המתאימות, העברת הפקטות ליעדן בשיחה עם ביצוע עיבוד על הפקטות ועל סגירה ושחרור של השיחות מהמערכת בהתאם לנתונים המופיעים בפקטות RTCP.

יצירת פקטות שקט במקרה של אובדן פקטות (PLC): כאשר ה-jitter buffer מזהה שלא ניתן להוציא פקטה מהבאפר עקב אובדן פקטות או אם יש פער בתזמון, מופעל מנגנון PLC אשר מייצר פקטת שמע סינתטית המכילה "שקט". פקטה זו מחקה את מבנה הפקטות הקיימות כדי לשמור על עקביות במידע הנשלח תוך שמירה על ערכי ה-header.

ניתוח פקטות RTP: בעת קבלת פקטות השמע מנקודות הקצה, המערכת מבצעת ניתוח מלא של הפקטות לפי תקן RFC3550, תוך שימוש בפענוח ישיר של מבנה ה-header. תהליך זה כולל אימות תקינות של המבנה וחילוץ של השדות מהפקטה תוך הפרדה בין ה-header לבין ה-payload כהכנה להמשך העיבוד של הפקטה.

ניתוח פקטות RTCP: לצד פקטות השמע, המערכת מנתחת פקטות של פרוטוקול הבקרה RTCP לפי התקן RFC3550. ניתוח פקטות אלו מספקות למערכת מידע חיוני על איכות השידור של המקורות, ונתינת חיווי עבור שיחות השמע.

שידור מידע לממשק משתמש: המערכת מייצרת פקטות בפורמט JSON אשר מועברות לרכיב הממשק משתמש אשר מציג באופן ויזואלי את המידע עבור המשתמש. המידע אשר יוצג יכלול את חיווי שיחות השמע, סטטוס מקורות השמע בשיחות, ונתוני QoS עבור כל



מקורות השמע. חשוב להדגיש כי ממשק המשתמש אינו משפיע על התנהגות המערכת אלא רק מציג מידע למעקב.

תיעוד פעילות למערכת לוגים: לצורך תחקור, ניטור ובקרה עתידיים, כל אירוע במערכת מתועד לקובץ לוג טקסטואלי.

ניהול קובץ קונפיגורציה: בשלב האתחול, המערכת טוענת את קובץ הקונפיגורציה לזיכרון, מבצעת validation לכל ערך ומיישמת את ההגדרות בהתאם. במהלך הריצה, הערכים נשמרים במבנה נתונים פנימי ונגישים לכל מודול הזקוק להם – ללא צורך בקריאות חוזרות לדיסק, כדי להבטיח מהירות עבודה מרבית.

10.4 ביצועים עיקריים

המערכת VTM מסוגלת לקלוט ולנתב זרמי שמע בו-זמנית ממספר שיחות וממספר נקודות קצה שונות, תוך עיבוד שוטף של פקטות השמע והתאמה לתנאי רשת משתנים.

המערכת מבצעת קידוד, פענוח, ניתוח QoS והמרת קודקים ללא השפעה על זרימת השמע עבור השיחות האחרות במערכת.

עדכון פרטי השיחה, מדדי איכות וכתיבה ללוגים מתבצעים תוך שמירה על זמני תגובה ללא עיכובים מיותרים ונמוכים ככל שניתן ובנוסף ללא חסימת התקשורת אצל שאר השיחות הפעילות.

10.5 אילוצים

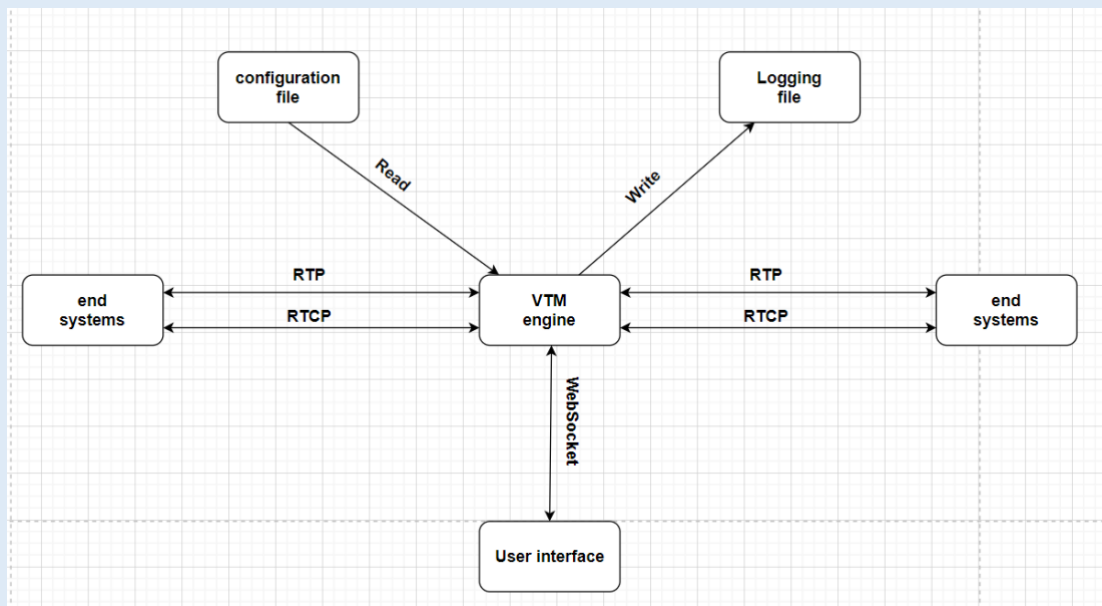
- ☒ המערכת פותחה על מערכת ההפעלה Windows בלבד, ולא נוסתה על מערכות הפעלה אחרות כגון Linux.
- ☒ המערכת מוגבלת לעבודה עם קודקים שתומכים בקצב דגימה של 8kHz. קצב זה מתאים לסטנדרט המקובל ב-VoIP, כאשר המערכת מתוכננת לעבוד בסביבות VoIP אך עלול להוות מגבלה בסביבות הדורשות איכות שמע גבוהה יותר.
- ☒ המערכת מתוכננת לעבד פקטות שמע המכילות 160 דגימות PCM – המהווה 20 מילי-שניות של שמע. גודל זה מהווה frame יחיד בקודקים נפוצים ב-VoIP. כלומר, המערכת מתוכננת לעבד פקטות המכילות frame יחיד.
- ☒ ממשק המשתמש של המערכת מיועד לתצוגה בלבד ואינו מאפשר שינוי בהגדרות המערכת או השפעה על פעילות המערכת במהלך ריצתה.
- ☒ המערכת משתמשת בקובץ קונפיגורציה סטטי וכל שינוי בו מחייב אתחול מחדש של המערכת.
- ☒ המערכת תומכת בפרוטוקולי זמן אמת על פי התקן RFC3550.



11. תיאור הארכיטקטורה

11.1 הארכיטקטורה של הפתרון המוצע בפורמט של Top-Down level design

תרשים DFD-0:



בתחילת ריצתה של המערכת, המערכת טוענת מידע מקובץ קונפיגורציה חיצוני המכיל את אופן תצורת המערכת – כגון תצורת השיחות, שליחת הלוגים ושליחת העדכונים לממשק משתמש. המערכת מוודאת את תקינות הקובץ ותוך כדי ריצה מבצעת קריאה ושליפה של מידע ממנו בעת הצורך.

נקודות הקצה ברשת, משדרות פקטות שמע בפרוטוקול RTP ודוחות ניטור ובקרה בפרוטוקול RTCP ברשת אל מערכת VTM.

VTM, המשמש כשרת Relay עצמאי, קולט את הפקטות הנכנסות, מנתח את המידע, ומבצע עליו עיבוד הכולל בדיקת תקינות, buffering והתאמות הנדרשות לקידוד השמע בהתאם למצב הרשת ולדרישות נקודות הקצה.

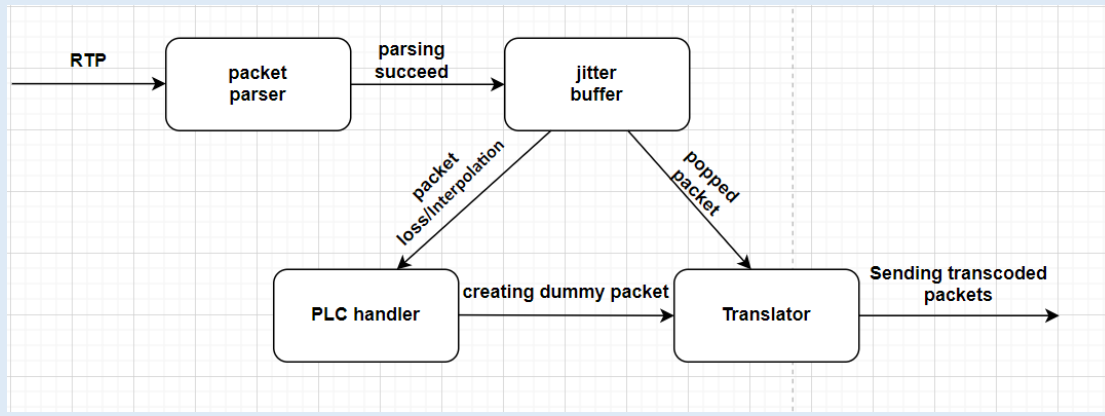
תוך כדי הפעולה, כל אירוע חשוב שמתבצע על גבי המערכת לגבי סטטוס השיחות, המקורות ועל תהליך עיבוד התקשורת – מתועד ונרשם אל תוך קובץ לוג ייעודי המיועד לבקרה, ניתוח ותחזוקה עתידית.

על מנת לאפשר למשתמשים לראות בצורה ויזואלית את פעילות המערכת, מתבצעת תקשורת WebSocket בין רכיב ה-VTM לבין ממשק המשתמש המשדר את המידע המתקבל ממנו עבור סטטוס השיחות, מקורות השמע ונתוני QoS לבקרה ויזואלית על מצב החיבור שלהם. רכיב זה מאפשר למשתמש לצפות בנתוני השיחות והמשתמשים בצורה נגישה וברורה, מבלי להשפיע בפועל על פעולת המערכת.

לאחר עיבוד הפקטות, המידע מנותב אל עבר נקודת היעד בהתאם למוגדר בקובץ קונפיגורציה.



תרשים 1-DFD:



כאשר המערכת מקבלת פקטת RTP מהרשת, היא מנותבת למודול packet parser שמנתח את מבנה הפקטה לפי RFC3550, בודק את תקינות הפקטה – ואם היא תקינה מקבלים אובייקט שמאפשר שימוש נוח בשדות השונים ביתר חלקי המערכת, אחרת זורקים את הפקטה וחוסמים אותה.

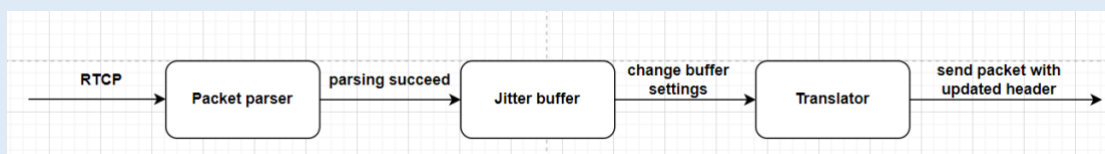
לאחר ביצוע ה-validation, הפקטה נשמרת בתוך ה-jitter buffer כל עוד היא לא מאוחרת מידי או שהיא פקטה שכבר התקבלה (משוכפלת).

עם הקריאה להוצאת פקטת RTP, אם התגלה מצב של packet loss או Interpolation מייצרים פקטה סינתטית של שקט דרך מודול ה-PLC. אחרת מוציאים פקטה תקינה מתוך הבאפר.

הפקטה המוצאת/הסינתטית מועברת למודול ה-Translator אשר ממיר את קודק השמע של הפקטה על פי הקונפיגורציה של השיחה.

לבסוף הפקטה נשלחת ליעדה על פי הנקבע בקונפיגורציית השיחה.

עבור כל אירוע המתרחש עם פקטת ה-RTP נשלח לקונסול ולקובץ ייעודי לוג מתאים על פי רמתו, ובנוסף גם ל-frontend לצורך חיווי המקורות והשיחות.



כאשר המערכת מקבלת פקטת RTCP מהרשת, היא מנותבת גם כן למודול ה-packet parser שמנתח את מבנה הפקטה לפי RFC3550, בודק את תקינות הפקטה – ואם היא תקינה מקבלים אובייקט שמאפשר שימוש נוח בשדות השונים ביתר חלקי המערכת, אחרת זורקים את הפקטה וחוסמים אותה.

לאחר מכן, הפקטה מועברת למודול ה-monitor תוך שימוש בפקטות מסוג SR/RR לקבלת נתוני QoS ממערכות הקצה לצורך שינויים בהגדרות של ה-jitter buffer. שינויים אלו חשובים למודול מפני שהוא פועל לא רק על סמך מה שהוא מקבל אלא גם על סמך ה-feedback של נקודות הקצה.

בנוסף, עבור סוג הפקטה SDES מושם שמות CNAME עבור מקורות השמע לצורך זיהוי ייחודי של המקורות, ועבור סוג הפקטה BYE מבצע ניתוק ושחרור של מקורות ושיחות שמע



מהמערכות על סמך המזהים הייחודיים (SSRC) המופיעים בפקטה.
עם ביצוע ההתאמות על סמך סוג הפקטה, פקטת ה-RTCP מועברת למודול ה-Translator

אשר מעדכן את השדה של `octets_sent` בתוך פקטת SR לביצוע התאמה עם תהליך ההמרה של ה-RTP.

לבסוף הפקטה נשלחת ליעדה על פי הנקבע בקונפיגורציית השיחה.
כל אירוע המתרחש עם פקטת ה-RTCP בתהליך ונתוני QoS נשלחות לקונסול ולקובץ לוג ייעודי מתאים על פי רמתו. בנוסף עבור פקטות RTCP נשלח לממשק משתמש נתוני QoS על מקורות השיחה ועל מצבם – אם הם עדיין בשיחה או התנתקו.

11.2 תיאור הרכיבים בפתרון

בפתרון המוצע, נכללים מספר רכיבים מרכזיים כאשר כל אחד מהם ממלא תפקיד מוגדר בתהליך גישור, עיבוד והצגת תקשורת שמע בזמן אמת.

הרכיב המרכזי בפתרון הוא ה-backend שהינו מנוע עיבוד תקשורת והוא מהווה את ליבת המערכת. תחום האחריות שלו בא לידי ביטוי בקבלת פקטות שמע בפרוטוקול RTP, ניתוח איכות התקשורת באמצעות פרוטוקול RTCP, ביצוע עיבוד של הפקטות המתקבלות על ידי ניהול `adaptive jitter buffer` וביצוע המרה בין קודקי שמע שונים ושידור הפלט ליעד. בנוסף, רכיב זה גם מתעד את פעילותו באמצעות לוגים, ושולח מידע עדכני על פעילותו לממשק הגרפי.

בנוסף, המערכת מכילה גם את רכיב ה-frontend, המהווה ממשק גרפי ייעודי שמטרתו לספק תצוגה בזמן אמת של פעילות רכיב ה-Backend לצורך ניטור ובקרה ויזואלית על תהליכי המערכת. הממשק מעוצב למשתמש את המידע ללא אפשרות להשפיע דרכו על תהליכי ה-backend או לשנות את קובץ הקונפיגורציה בזמן ריצה. כמו כן רכיב זה יציג למשתמש את פרטי השיחות הפעילות, לוגים בזמן אמת של אירועי עיבוד השמע במערכת והצגה חיה של פרמטרי איכות התקשורת (`packet loss, jitter, latency`).

המערכת עובדת באופן רציף מול רכיב של הקובץ קונפיגורציה. קובץ הקונפיגורציה הוא קובץ סטטי בפורמט ini אשר נטען עם אתחול המערכת. קובץ זה מגדיר את תצורת הלוגים, תצורת השיחות ואת פרטי המשתתפים. השימוש בקובץ חיצוני מאפשר גמישות בהגדרת המערכת מבלי צורך לבצע שינויים בקוד המקור, ומקל על התאמה למצבים שונים או הרחבות עתידיות.

11.3 תיאור תהליכים של מערכת ההפעלה שמתבצעים בפרויקט

המערכת VTM פועלת כתהליך עצמאי על גבי מערכת ההפעלה, תוך שימוש במגוון תשתיות פנימיות לניהול תקשורת, זיכרון וקבצים – באופן שמאפשר פעילות יעילה, מהירה ואסינכרונית.

ניהול תקשורת רשת: בפרויקט VTM, ניהול התקשורת מתבצע בגישת Asynchronous I/O. זוהי פרדיגמת תכנות אשר מאפשרת למערכת לטפל באירועים בצורה יעילה מבלי לחסום את זרימת התוכנית. לצורך כך נעשה שימוש בספריית `boost.asio` ובאובייקט אותו היא מספקת `io_context`.

העיקרון המרכזי בגישת התכנות Asynchronous I/O הוא שכל פעולות קלט/פלט, למשל קבלה/שליחה של פקטות ברשת, אינה מתבצעת בצורה שחוסמת את ריצת התוכנית. במקום



זאת, מבוצעת קריאה לפונקציה אסינכרונית, לדוגמה `async_send`, `async_receive` שבה נרשם `callback` - פונקציית שמתוזמנת לפעול ברגע שהמידע מגיע. תהליך זה מאפשר למערכת להמשיך לפעול באופן רגיל, מבלי להמתין לסיום הפעולה.

`io_context` הוא אובייקט הפועל כמרכז בקרה אסינכרוני ומטרתו היא לנהל את לולאת האירועים של המערכת (`event loop`) - מנגנון תכנותי שבו במקום שהמערכת תפעל בצורה לינארית על פי סדר קריאת הפעולות, היא מחכה לאירועים של קלט/פלט ומפעילה רק את הפונקציות הרלוונטיות שמטפלות בהן - `callbacks`.

ניהול לולאת האירועים מתבצעת על ידי קריאה לפונקציה `run` שנשארת פועלת כל עוד יש פעולות מתוזמנות או ממתונות במערכת. בכל פעם שאירוע כזה מתרחש, האובייקט מפעיל את ה-`callback` הייעודי לו. את האובייקט ניתן להפעיל בתהליכון אחד או במספר תהליכונים, אך בזכות המודל האסינכרוני, גם תהליכון יחיד מסוגל לטפל בו זמנית במספר פעולות רשת, טיימרים ואירועים פנימיים, כיוון שהשליטה מועברת ישירות ל-`callback` הייעודי לו רק כאשר האירוע מתרחש.

גישת תכנות זו מפחיתה את השימוש בתהליכונים, מונעת יצירת עומסים מיותרים על המעבד, ומאפשרת למערכת לפעול בצורה חלקה ויעילה, גם תחת עומס או עיכובים ברשת.

ניהול טיימרים אסינכרוניים: בנוסף לתקשורת עצמה, המערכת עושה שימוש במנגנון הטיימרים האסינכרוניים של הספרייה `boost.asio`, לצורך תזמון מדויק של פעולות פנימיות תוך כדי ריצת התוכנית. אחת הפעולות הקריטיות במערכת היא שליפה מחזורית של פקטות שמע מוכנות מה-`jitter buffer` בתזמונים זהים לצורך המשך עיבודם.

עבור כל מקור שמוגדר לו `jitter buffer`, מוגדר בנוסף טיימר אסינכרוני אשר מוגדר לו כמה זמן לחכות. עם פקיעת הטיימר, מופעלת פונקציה `callback` שמנסה לשלוף פקטה מתוך הבאפר.

בסיום פעולת השליפה, הטיימר מאותחל מחדש ומתחיל מחזור נוסף. מחזוריות זו מתבצעת באמצעות `async_wait` כך שלולאת האירועים פועלת בצורה לא חוסמת. מנגנון זה משתלב באותו אובייקט של `io_context` שמנהל את שאר המשימות, ומצבע תיאום מושלם עם שאר המרכיבים במערכת.

גישה זו מחליפה את השימוש בריבוי תהליכונים ומונעת צורך בפעולות `sleep` על התהליכון או שימוש בטיימר רגיל עליו אשר יחסום את ריצתו. כתוצאה מכך, השימוש במנגנון תורם לביצועים גבוהים ולצריכת מעבד נמוכה יותר. המנגנון מבטיח שהמערכת תוכל לספק קצב פלט קבוע ומדויק — דבר קריטי במיוחד בעיבוד שמע בזמן אמת, שבו כל סטייה בזמן ההשמעה עלולה לגרום לעיוותים או להשהיות בשיחה.

עבודה עם קבצים: המערכת כותבת את פעילותה לקובץ לוגים ייעודי בזמן אמת לצורכי ניטור, איתור תקלות וביצוע תחקור עתידי.

בנוסף בעת אתחול וריצת התוכנית, המערכת טוענת קובץ קונפיגורציה סטטי מסוג `ini` אשר מכיל את הגדרות המערכת ותצורת השיחות.

ניהול זיכרון דינמי: המערכת עושה שימוש נרחב בהקצאות זיכרון דינאמיות על זיכרון ה-`heap` לצורך:



1. אחסון זמני של פקטות שמע נכנסות
2. ניהול adaptive jitter buffer
3. הקצאת אובייקטים עבור שיחות ומקורות פעילים

4. הקצאת האובייקטים האחראים על המרת הקודק במנגנון ה-Translator.

תכנון המערכת שם דגש על מניעת דליפות זיכרון, הימנעות מ-stack overflow והפחתת עומס על המעבד, והוא מבוסס על עקרונות תכנות ++C מודרני תוך שימוש ב-move semantics, הפחתת פעולות העתקה מיותרות והימנעות מהקצאות מיותרות של זיכרון.

11.5 תיאור פרוטוקולי תקשורת

פרוטוקול UDP: פרוטוקול תקשורת לא מבוסס חיבור המשמש ברשתות להעברת נתונים בצורה מהירה ויעילה. הוא פועל מבלי ליצור חיבור או להבטיח מסירה אל נקודת היעד, מה שהופך אותו למתאים ליישומים בזמן אמת שבהם ניתנת עדיפות על מהירות שליחת הנתונים על פני האמינות של הנתונים שנשלחו. UDP תומך גם בשידור ובריבוי שידורים, המאפשרים שליחת נתונים למספר נמענים בו זמנית.

פרוטוקול RTP: פרוטוקול שנועד לטפל בתעבורה בזמן אמת כמו שמע ווידאו של האינטרנט. הפרוטוקול מספק אמצעים כדי לפצות על ריצוד וגילוי של פקטות מידע אבודות ולסדר פקטות מידע שהגיעו בסדר שונה בשליחתן, ומאפשר העברת מידע למספר יעדים שונים דרך שידורי IP. משתמשים בפרוטוקול זה עם שימוש בפרוטוקול UDP.

פרוטוקול RTCP: פרוטוקול שנועד לספק מידע ופיקוח על איכות השידור בזמן אמת של שמע ווידאו. פרוטוקול זה מספק משוב על איכות הפצת הנתונים במקרים בהם יש עיכובים או אובדן פקטות, מסייע בסינכרון מספר זרמי מדיה באותו הרשת, ובנוסף מנהל בקרה על העברת הנתונים כמו זיהוי המשתתפים וניהול סיום העברת הנתונים.

פרוטוקול TCP: פרוטוקול תקשורת אמין ומבוסס חיבור המשמש להעברת מידע בצורה רציפה ומסודרת בין שני צדדים, תוך הבטחת הגעה תקינה של כל פקטה, ניהול סדר ההעברה, זיהוי שגיאות.

פרוטוקול WebSocket: פרוטוקול תקשורת דו-כיווני, מהיר וקל משקל, הפועל מעל פרוטוקול TCP ומאפשר קישור רציף בין שרת ללקוח. פרוטוקול זה מאפשר שליחה וקבלה של נתונים באופן אסינכרוני ללא צורך בפתיחה חוזרת של חיבורים, מה שהופך אותו למתאים למערכות הדורשות עדכונים בזמן אמת.

11.6 שרת-לקוח

המערכת אינה מבוססת על מודל שרת-לקוח בצורתו הקלאסית.

מערכת VTM, פועלת בתור Relay server עצמאי – כלומר, היא מאזינה לזרמי שמע משיחות וממקורות שונים, מבצעת עיבוד וניתוב של השמע ושולחת לנקודת היעד – ללא צורך ברכיב נפרד של שרת אפליקטיבי או ממשק לקוח.



14. ניתוח ותרשים UML/Use cases של המערכת המוצעת

14.1 תיאור ה-UC העיקריים של המערכת

UC-0: המרת קודק שמע (Transcoding) בזמן אמת:

תיאור האלגוריתם: האלגוריתם מבצע המרה בין קידודי שמע שונים (כגון G711 Alaw ו-Speex) בהתאם להגדרות בקובץ קונפיגורציה. התהליך כולל פענוח של פקטת שמע RTP לפורמט PCM לא דחוס, ולאחר מכן קידוד מחדש לקודק הדרוש לנקודת היעד.

מטרת האלגוריתם: לשמש כמנגנון גישור בין מערכות ונקודות קצה שונות אשר משתמשות בקודקים לא זהים, ולאפשר תקשורת תקינה ביניהם. האלגוריתם תומך ומגשר בין המימושים של פונקציות הקידוד והפענוח של הקודקים ומספק אחידות באיכות השמע תוך עמידה בדרישות זמן אמת.

זרימת הפעולה (main flow):

1. ראשית, ה-Translator מזהה מן הקובץ קונפיגורציה אילו קודקים כל מקור משתמש לצורך התקשורת לצורך אתחול המודול.
2. פקטת שמע מועברת למנגנון לאחר שהוצאה מן ה-jitter buffer.
3. מתבצעת בדיקת קודק מקור וקודק יעד.
4. ההמרה מתבצעת בשלבים שלהלן:
 - a. פענוח: קוראים לפונקציית הפענוח של קודק המקור לקבלת פורמט לא דחוס PCM.
 - b. קידוד: מקודדים את פורמט ה-PCM שקיבלנו אל הקודק של נקודת היעד באמצעות פונקציית הקידוד המתאימה.
5. בונים פקטת RTP חדשה עם header מעודכן.
6. שולחים את הפקטה החדשה לשידור לנקודת היעד.

UC-1: ניהול אדפטיבי של ההשהיה והוצאה של פקטת שמע מה-jitter buffer:

תיאור האלגוריתם: אלגוריתם זה אחראי לחשב את זמן ההשהיה (delay) האופטימלי במערכות VoIP לצורך תזמון מדויק של פקטות שמע בתנאי רשת משתנים. מטרתו היא להחליט מתי להוציא פקטה לעיבוד/לשידור מתוך הבאפר ומתי לדחות או להשלים פקטות חסרות. החישוב מתבסס על ידי ניתוח ממושך של היסטוריית התזמונים של הפקטות שנכנסו לבאפר.

מטרת האלגוריתם:

1. לאזן בין זמן תגובה (latency) לבין אמינות/רציפות השמע
2. להפחית רעשים וקפיצות לא רצויות בשמע הנגרמות משיבושים בתזמון הפקטות
3. לספק "חלון דינאמי" של ההשהיה שמתאים את עצמו למצב הרשת בפועל

זרימת הפעולה (main flow):



1. הזנת נתוני התזמון: עבור כל פקטה שמאחרת, היא נרשמת בתוך טבלת תזמון פנימית, ועבור כל פקטה שלא מאחרת נרשם מתי היא הוכנסה לבאפר ביחס לציר הזמן הנוכחי.

2. חישוב השהיה אופטימלי: באופן מחזורי, המודול מחשב את ההשהיה האופטימלי ביחידות RTP אשר מבצעת ניתוח סטטיסטי של התזמונים של הפקטות, מחשבת פונקציית עלות משוקללת של השהיה + אחוז האיחורים, והיא בוחרת את ההשהיה היביא את האיזון הכי טוב בין קיצור ההשהיה למינימום פקטות מאוחרות.
3. עדכון השעון הפנימי של הבאפר: הבאפר מוציא פקטות על פי משתנה מטיפוס uint32_t המהווה את השעון הפנימי של הבאפר – איזה timestamp על הבאפר להוציא הוצאה הבאה. מתי שמחשבים את זמן ההשהיה האופטימלי מעדכנים את השעון הפנימי של המערכת:
 - a. אם האלגוריתם מציע לקצר את ההשהיה -> השעון הפנימי מוזז קדימה והמערכת תידרש לבצע interpolation - תהליך שבו המנגנון יוצר באופן סינתי פקטת שמע חדשה כאשר חסרה פקטה בזמן הרצוי בשביל לשמור על רציפות השמע.
 - b. אם האלגוריתם מציע להאריך את ההשהיה -> הפלט יתעכב אבל יפחית את אובדן הפקטות.
4. הוצאת פקטה מהבאפר: נבדוק אם קיימת פקטה מתאימה לערך של השעון הפנימי ועל פי מנגנוני falloff במקרי קצה – כאשר אין בבאפר פקטה שעונה בדיוק לדרישות בזמן הנתון, המנגנון מנסה לבדוק אפשרויות אלטרנטיביות להוצאה.
 - a. אם כן, הפקטה נשלפת ונשלחת להמשך תהליך העיבוד – transcoding
 - b. אחרת, מוחזרת בקשה ל-PLC ליצירת פקטת שקט חלופית.

14.2 הצגת מקרה (use case) עבור כל הפונקציות העיקריות בפרויקט

הפונקציות העיקריות עבור UC-0:

לצורך יישום מלא של מנגנון ה-Transcoding במערכת, נדרשת היכרות עם עקרונות הפעולה ומבני הנתונים של libspeex, ובנוסף היכרות עם המבנה הבינארי של ערכי G711 A-law והקבועים הנלווים הדרושים למימוש המרה.

מבוא עבור קידוד ופענוח Speex:

בכדי ליישם באופן מלא את תהליך המרת קודקי השמע במערכת, נדרש להכיר הן את מבני הנתונים המרכזיים המנוהלים על ידי ספריית libspeex, והן את אופן פעולתה הפנימי במצב narrowband. רכיבים אלו מהווים בסיס חיוני לפעולות הקידוד והפענוח של זרמי שמע בזמן אמת, ומאפשרים ניהול יעיל ומדויק של זרמי מידע דחוס.

Speex narrowband mode: בפרויקט זה נעשה שימוש בקודק speex במצב narrowband הפועל בקצב דגימה של 8kHz. במצב זה, כל frame מייצג מקטע של 20 מילי-שניות של שמע, המכיל 160 דגימות PCM. כל frame בקידוד מחולק ל-4 תתי frame בני 40 דגימות כל אחד.

תהליך הקידוד במצב זה מתבצע ב-2 שלבים:



1. Whole-frame analysis: בשלב זה מתבצע LPC על דגימות השמע, אשר מומר לערכים בשם LSP – המייצגים את מאפייני התדר של הקול בצורה יציבה. כמו כן, מחושב כללי gain שמיצג את עוצמת הקול ב-frame כולו.
2. Sub-frame Abs: בכל תת frame מתבצע חיפוש אחר ערכי pitch (מחזוריות קולית) ואחר innovation (אות חדשנות) שמיצגים את פרטי הצליל. Speex בנוסף

משתמש בשיטה שקוראים לה 3-tap predictor - שלושת הערכים שמיצגים את מחזוריות הקול בדיוק גבוה יותר.

אחד מן היתרונות ב-speex הוא שהוא לא יוצר תלות בין frames עוקבים, כך שאם יש אובדן של פקטה אין לכך השפעה על המשך השיחה. זה מושג על ידי כך שהמידע החשוב (כמו ה-gain של ה-codebook) מחושב עבור כל frame באופן עצמאי, ללא תלות ב-frame הקודם.

```
SpeexEncode m_encodeState;  
SpeexDecode m_decodeState;  
SpeexBits m_encodeBits;  
SpeexBits m_decodeBits;
```

SpeexBits: הוא מבנה האחראי על ניהול זרם הביטים במהלך תהליך הקידוד או הפענוח. בניגוד לפורמטים פשוטים שבהם יש גודל ביטים קבוע, Speex מקודד מידע בכמות משתנה של ביטים עבור פרמטרים שונים. לכן נדרש מנגנון גמיש ויעיל שיכול לקרוא ולכתוב ביטים בגמישות. מבנה זה הוא בעצם מהווה "צינור הקידוד/הפענוח" – כל התהליך עובר דרכו:

- במהלך הקידוד, SpeexBits משמש לאחסון זמני של המידע הדחוס שנוצר על ידי הקודק.
- במהלך הפענוח, SpeexBits נטען מהנתונים הדחוסים המגיעים מהרשת ומאפשר ל-decoder לפענח אותם בצורה תקינה.

SpeexEncoderState: הוא מבנה אשר מייצג מופע פעיל של תהליך קידוד. הוא שומר בתוכו את כל המידע הדרוש לקידוד אודיו – את כל הפרמטרים הדרושים לאלגוריתם CELP. כאשר המבנה מקבל אליו בלוק המייצג 20 מילי-שניות של שמע בפורמט PCM הוא מנתח את הנתונים הקוליים מבלוק השמע, מקודד אותו על פי אלגוריתם CELP, ולבסוף דוחס את המידע אל תוך זרם הביטים דרך מבנה SpeexBits.

SpeexDecoderState: הוא מבנה אשר מייצג מופע פעיל של פענוח המידע לפורמט PCM. מבנה זה מקבל את זרם הביטים מהמבנה SpeexBits, מבצע ניתוח הפוך לתהליך הקידוד ומשחזר את בלוק ה-PCM המקורי.



מבוא עבור קידוד ופענוח G711 A-law:

לאחר שהוצגו עקרונות פעולתו של מבני ספריית libspeex, נבחן כעת את המבנה הבינארי של ערך G711 A-law ואת הקבועים בהם נעשה שימוש בתהליך יצירת טבלאות LUT.

```
static constexpr int16_t SEG_SHIFT = 4;  
static constexpr int16_t QUANT_MASK = 0x0F;  
static constexpr int16_t SEG_MASK = 0x70;  
static constexpr int16_t SIGN_BIT = 0x80;  
static constexpr std::array<int16_t, 8> SEGMENTS_UPPER_BOUNDS =  
{ 0x1F, 0x3F, 0x7F, 0xFF, 0x1FF, 0x3FF, 0x7FF, 0xFFF };
```

קידוד של דגימת PCM של 16 ביטים לקודק G711 A-law מפיקה ערך בגודל של 8 ביטים, אשר מורכבים מ-3 חלקים עיקריים:

1. Sign bit: הביט השביעית בערך (MSB), מציינת אם הערך הוא חיובי (1) או שלילי (0). בשביל לבדד את ערכו, ניתן להשתמש במסכה SIGN_BIT.
2. Segment: שלושת הביטים הממוקמות בביטים 4-6 מייצגות את סגמנט העוצמה של דגימת הקול – התחום הלוגריתמי שבו נמצא ערך ה-PCM בתוך מערך הסגמנטים. ככל שאות השמע חזק יותר כך הוא נמצא בסגמנט גבוה יותר. משתמשים במסכה SEG_MASK ובהיסט SEG_SHIFT בכדי לבדד ולחלץ את ערך הסגמנט מן הערך המקודד.
3. Quantization: ארבעת הביטים הכי פחות משמעותיים (ביטים 0-3) מייצגות את כמות הדגימה בתוך הסגמנט – פרטים נוספים שעוזרים לשמר את מבנה הדגימה המקורית בהתאם לרזולוציה הנתונה. משתמשים במסכה QUANT_MASK כדי לבדד ערך זה.

בכדי לדעת באיזה סגמנט נמצאת דגימת PCM, נעשה שימוש במערך קבוע וידוע בשם SEGMENTS_UPPER_BOUNDS. מערך זה מגדיר את הגבולות העליונים של כל אחד משמונת הסגמנטים האפשריים בקידוד A-law. באופן זה ניתן לקבוע את הסגמנט אליו נופלת דגימת PCM באמצעות חיפוש בתוך טבלה ידועה מראש.



אתחול המבנים של הקידוד והפענוח של Speex:

```
void SpeexTranscoder::initialize()
{
    m_encodeState = speex_encoder_init(&speex_nb_mode);
    if (!m_encodeState)
    {
        throw std::runtime_error("Failed to initialize Speex encoder");
    }

    m_decodeState = speex_decoder_init(&speex_nb_mode);
    if (!m_decodeState)
    {
        speex_encoder_destroy(m_encodeState);
        throw std::runtime_error("Failed to initialize Speex decoder");
    }

    speex_bits_init(&m_encodeBits);
    speex_bits_init(&m_decodeBits);
}
```

קלט: אין, מתבססת על פרמטרים פנימיים וקבועים.

פלט: אין, תוצאת הפעולה היא אתחול תקין של תכונות המחלקה.

תיאור האלגוריתם: הפונקציה אחראית לאתחול את כל המשאבים הדרושים לצורך קידוד ופענוח זרמי שמע בפורמט narrowband mode עבור Speex.

זרימת הפעולה (main flow):

1. מאתחלים את מבנה הקידוד שיפעל ב-narrowband mode.
2. אם הקצאת הזיכרון נכשלה, זורקים חריגה למערכת.
3. מאתחלים את מבנה הפיענוח שיפעל ב-narrowband mode.
4. אם הקצאת הזיכרון נכשלה, זורקים חריגה למערכת.
5. מאתחלים את מבני SpeexBits - לניהול זרמי הביטים לקידוד ולפענוח.

הסבר על המשתנה speex_nb_mode: זהו משתנה מוגדר מראש בספריית libspeex שמכיל את כל ההגדרות הדרושות להפעלת הקודק במצב narrowband. במקום להגדיר ידנית פרמטרים כמו תדר דגימה או גודל של frame, מעבירים את המשתנה הזה לאתחול של ה-encoder או ה-decoder והוא מגדיר אוטומטית את ההתנהגות הרצויה לפי התקן של 8kHz.



אתחול טבלאות LUT עבור קידוד ופענוח G711:

```
uint8_t G711Transcoder::linear_to_alaw(int16_t pcm_value)
{
    uint8_t alaw_value;
    int16_t mask, segment_index;

    pcm_value >>= 3;

    if (pcm_value > 0)
        mask = 0xD5;
    else
    {
        mask = 0x55;
        pcm_value = -pcm_value - 1;
    }

    segment_index = std::lower_bound(SEGMENTS_UPPER_BOUNDS.begin(), SEGMENTS_UPPER_BOUNDS.end(), pcm_value) -
        SEGMENTS_UPPER_BOUNDS.begin();

    if (segment_index >= SEGMENTS_UPPER_BOUNDS.size())
        alaw_value = static_cast<uint8_t>(0x7F ^ mask);
    else
    {
        alaw_value = static_cast<uint8_t>(segment_index << SEG_SHIFT);
        if (segment_index < 2)
            alaw_value |= (pcm_value >> 1) & QUANT_MASK;
        else
            alaw_value |= (pcm_value >> segment_index) & QUANT_MASK;
        alaw_value ^= mask;
    }

    return alaw_value;
}
```

קלט: ערך דגימת שמע בפורמט PCM.

פלט: ערך מקודד בגודל 8 ביט בפורמט G711 A-law.

תיאור אלגוריתם: הפונקציה ממירה ערך PCM לא דחוס לפורמט A-law בהתאם לעיקרון Companding ולמבנה 8 ביט A-law.

זרימת הפעולה (main flow):

1. מורידים את הרזולוציה של דגימת ה-PCM בשביל להתאים את הערך לאלגוריתם הקידוד.
2. קובעים את ערך המסכה, כאשר עבור ערך שלילי ממירים אותו לערך חיובי על פי שיטת המשלים ל-2. ערך המסכה הוא בהתאמה לסטנדרט של קידוד G711 לשמירת הסימן באמצעות XOR.
3. מוצאים את הסגמנט אליו הדגימה נופלת.
4. אם הערך חורג מהטווח של G711 - מחזירים את הערך המקסימלי של 8 ביט.
5. מבצעים השמה של רכיב הסגמנט בתוך ה-8 ביט, כאשר את רכיב ה- משתנה בהתאם לגובה הסגמנט Quantization.
6. מבוצע XOR עם ערך המסכה כדי להשלים את הקידוד של G711.



```
int16_t G711Transcoder::alaw_to_linear(uint8_t alaw_value)
{
    int16_t pcm_value, segment_index;

    alaw_value ^= 0x55;

    pcm_value = (alaw_value & QUANT_MASK) << 4;
    segment_index = (alaw_value & SEG_MASK) >> SEG_SHIFT;

    switch (segment_index)
    {
        case 0: pcm_value += 8; break;
        case 1: pcm_value += 0x108; break;
        default: pcm_value += 0x108; pcm_value <= segment_index - 1; break;
    }

    pcm_value = ((alaw_value & SIGN_BIT) ? pcm_value : -pcm_value);

    return pcm_value;
}
```

קלט: ערך שמע מקודד בגודל 8 ביט בפורמט A-law.

פלט: ערך דגימת שמע בפורמט PCM אשר שוחזר מערך המקודד בפורמט A-law.

תיאור האלגוריתם: הפונקציה מבצעת פענוח של ערך G711 A-law בחזרה לפורמט PCM.

זרימת הפעולה (main flow):

1. מסירים את ה-XOR שחותם את קידוד Alaw. אם מבצעים על ערך XOR עם אותה מסכה פעמיים מקבלים בחזרה את הערך ההתחלתי.
2. מחלצים את שדה ה-Quantization ומשחזרים את קנה המידה גולמי.
3. מחלצים את שדה הסגמנט מתוך הערך המקודד.
4. משחזרים את ערך ה-PCM על פי הסגמנט אליו הוא נופל.
5. אם ערך ביט הסימן דולקת, הערך חיובי. אחרת הערך הוא שלילי.

```
void G711Transcoder::generate_alaw_lookup_table()
{
    for (size_t i = 0; i < s_alawLookupTable.size(); ++i)
        s_alawLookupTable[i] = linear_to_alaw(static_cast<int16_t>(i));
}
```

קלט: אין.



תיאור האלגוריתם: הפונקציה מבצעת מעבר על כל ערך אפשרי בטווח של 16 ביטים בעלי סימן. כל ערך כזה מייצג דגימת PCM כאשר לאחר ההמרה הוא יוכל לשמש כאינדקס בתוך טבלת LUT. בפונקציה זו מתבצעת המרה לפורמט A-law והתוצאה מאוחסנת בטבלה לצורך שימוש עתידי מהיר.

```
void G711Transcoder::generate_pcm_lookup_table()
{
    for (size_t i = 0; i < s_pcmLookupTable.size(); ++i)
        s_pcmLookupTable[i] = static_cast<uint16_t>(alaw_to_linear(static_cast<uint8_t>(i)));
}
```

קלט: אין.

פלט: אין.

תיאור האלגוריתם: הפונקציה ממלאת את טבלת ה-LUT של ערכי PCM כאשר כל אינדקס מייצג ערך A-law והערך השמור בטבלה הוא הפענוח המתאים לו בפורמט PCM. עבור כל ערך אפשרי של 8 ביט A-law מבצעים פענוח ל-PCM והתוצאה נשמרת בתוך הטבלה.

הערה – הפלט נשמר בתוך הטבלה בתור `uint16_t` במקום בטיפוס המתקבל שהוא `int16_t` לצורך אחסון טכני יעיל בתוך הטבלה, ללא קונפליקטים בין ערכים signed לבין unsigned.

פונקציות הקידוד והפענוח של Speex:

```
void SpeexTranscoder::compress(std::vector<uint8_t>& encoded_audio, const std::vector<int16_t>& linear_pcm_audio)
{
    encoded_audio.clear();
    std::array<int16_t, FRAME_SIZE> frame_data;
    size_t total_samples = linear_pcm_audio.size();

    for (size_t offset = 0; offset < total_samples; offset += FRAME_SIZE)
    {
        std::fill(frame_data.begin(), frame_data.end(), 0);
        size_t samples_to_copy = std::min(static_cast<size_t>(FRAME_SIZE), total_samples - offset);
        std::copy_n(linear_pcm_audio.begin() + offset, samples_to_copy, frame_data.begin());

        speex_bits_reset(&m_encodeBits);
        speex_encode_int(m_encodeState, frame_data.data(), &m_encodeBits);

        size_t encoded_bytes = speex_bits_nbytes(&m_encodeBits);
        size_t old_size = encoded_audio.size();

        encoded_audio.resize(old_size + encoded_bytes);

        int written = speex_bits_write(
            &m_encodeBits,
            reinterpret_cast<char*>(encoded_audio.data() + old_size),
            encoded_bytes
        );

        encoded_audio.resize(old_size + written);
    }
}
```

קלט: מערך שיחזיק בתוכו את המידע המקודד.

פלט: מערך המחזיק בתוכו את המידע מקודד בקודק speex. הפלט נכתב ישירות נכתב ישירות לפרמטר המועבר לפונקציה `by reference`.



תיאור האלגוריתם: הפונקציה ממירה נתוני שמע לא דחוסים PCM לפורמט דחוס של הקודק `speex`. תהליך זה חיוני כחלק ממנגנון גישור הקודקים של המערכת כאשר נדרש לשלוח את השמע אל נקודות קצה המשתמשות ב-`speex`.

זרימת הפעולה (main flow):

1. כל עוד יש דגימות לעיבוד בצע –

- העתקה/השלמה – כל `frame` מועתק לתוך באפר זמני, כאשר אם כמות הדגימות שנשארו לקידוד קטן מגודל `frame` מוסיפים לו `padding` בשביל להשלים לגודל `frame`.
- מבצעים `reset` עבור המבנה `SpeexBits` בשביל להימנע מדליפות מידע מ-`frames` קודמים.
- מפעילים את אלגוריתם הקידוד שמעבד את המידע ומעדכן את המבנה `SpeexBits` עם הפלט.
- על פי כמות הבתים שקודדו ועל פי כמות הבתים שנכתבו בפועל לתוך הפקטה המקודדת מכניסים את ה-`frame` המקודד לתוך ה-`payload` המקודד מחדש.

```
void SpeexTranscoder::decompress(const std::vector<uint8_t>& encoded_audio, std::vector<int16_t>& linear_pcm_audio)
{
    speex_bits_read_from(&m_decodeBits, reinterpret_cast<const char*>(encoded_audio.data()), encoded_audio.size());
    std::array<int16_t, FRAME_SIZE> temp_frame;
    bool is_processing = true;

    while (is_processing)
    {
        std::fill(temp_frame.begin(), temp_frame.end(), 0);
        int ret = speex_decode_int(m_decodeState, &m_decodeBits, temp_frame.data());
        switch (ret)
        {
            case 0:
                linear_pcm_audio.insert(linear_pcm_audio.end(), temp_frame.begin(), temp_frame.end());
                break;
            case -1:
                is_processing = false;
                break;
            case -2:
                is_processing = false;
                std::fill(temp_frame.begin(), temp_frame.end(), 0);
                linear_pcm_audio.insert(linear_pcm_audio.end(), temp_frame.begin(), temp_frame.end());
                break;
        }
    }
}
```

קלט: מערך שמחזיק בתוכו את המידע המקודד.

פלט: מערך המחזיק בתוכו את המידע הלא דחוס בפורמט PCM. הפלט נכתב ישירות נכתב ישירות לפרמטר המועבר לפונקציה `by reference`.

תיאור האלגוריתם: הפונקציה אחראית לפענוח פקטות שמע בקודק `speex` ולהמרתם לפורמט PCM הנדרש לצורך המשך העיבוד וביצוע קידוד מחדש של הפקטה.

זרימת הפעולה (main flow):

- אתחול הביט-באפר – הנתונים המקודדים מוכנסים לתוך המבנה `SpeexBits` מה שמאפשר לספרייה לבצע את אלגוריתמי הפענוח.



2. לולאת הפענוח - כל עוד לא מתקבל סטטוס שגיאה/סיום הפענוח של הפקטה משתמשים במשתנה אשר משמש את המידע של ה-frame הנוכחי לשמירה של המידע הלא דחוס. עבור כל frame חדש מאפסים את תוכן הבאפר בכדי למנוע דליפות של תוכן השמע בין ה-frames.
3. בדיקת סטטוס החזרה -
- a. במקרה שמתקבל $0 < -$ הפענוח הצליח ומוסיפים את המידע החדש לתוך מבנה ה-PCM.
- b. במקרה שמתקבל $(-1) < -$ אין יותר frames לפענח וכל הנתונים פוענחו.
- c. במקרה שמתקבל $(-2) < -$ הפקטה שהוכנסה היא פגומה, ומכניסים לתוך מבנה ה-PCM שקט סינתטי. באמצעות תנאי זה, המנגנון יודע להתמודד עם מצבים בהם הפקטה פגומה.
4. איסוף הפלט – כל הנתונים הלא דחוסים מצטברים לתוך מערך המייצג את מבנה ה-PCM שימש לקידוד מחדש של הפקטה.

פונקציות הקידוד והפענוח של G711:

```
void G711Transcoder::compress(std::vector<uint8_t>& encoded_audio, const std::vector<int16_t>& linear_pcm_audio)
{
    encoded_audio.resize(linear_pcm_audio.size());
    std::transform
    (
        linear_pcm_audio.begin(), linear_pcm_audio.end(), encoded_audio.begin(),
        [](int16_t pcm_value) -> uint8_t
        {
            uint16_t unsigned_pcm = static_cast<uint16_t>(pcm_value);
            return s_alawLookupTable[unsigned_pcm];
        }
    );
}
```

קלט: מערך של ערכי אודיו לא דחוסים בפורמט PCM.

פלט: מערך של ערכי אודיו מקודדים בקודק G711 A-law. הפלט נכתב ישירות נכתב ישירות לפרמטר המועבר לפונקציה by reference.

תיאור האלגוריתם: הפונקציה מבצעת דחיסת שמע לאותות קול בפורמט PCM לקודק A-law על ידי שימוש בטבלת LUT מוכנה מראש. במקום לחשב את הקידוד מחדש בכל פעם, מתבצע חיפוש מהיר במערך LUT באמצעות ערך הקלט.

```
void G711Transcoder::decompress(const std::vector<uint8_t>& encoded_audio, std::vector<int16_t>& linear_pcm_audio)
{
    linear_pcm_audio.resize(encoded_audio.size());
    std::transform
    (
        encoded_audio.begin(), encoded_audio.end(), linear_pcm_audio.begin(),
        [](uint8_t alaw_value) -> int16_t
        {
            return static_cast<int16_t>(s_pcmLookupTable[alaw_value]);
        }
    );
}
```

קלט: מערך של ערכי שמע מקודדים בקודק G711 A-law.

פלט: מערך פלט של ערכי PCM לצורך קידוד מחדש בתהליך. הפלט נכתב ישירות לפרמטר המועבר לפונקציה by reference.



תיאור האלגוריתם: הפונקציה מבצעת את פעולת הפענוח - היא לוקחת כל ערך מקודד ב-Alaw, מחפשת את הערך המקורי שלו בטבלת הפענוח ומשחזרת את דגימת ה-PCM המתאימה. כמו בפונקציית הקידוד, גם כאן נעשה שימוש בגישה מבוססת LUT כדי להבטיח ביצועים מהירים ביותר בזמן ריצה.

הערה בקשר לקידוד והפענוח של G711: בגלל שנתון כי אלגוריתם הקידוד דוחס כל ערך 16 ביט PCM ל-8 ביט ערך Alaw ועבור אלגוריתם הפענוח מפענחים כל ערך 8 ביט Alaw בחזרה לערך 16 ביט PCM, אז ניתן לשנות בפונקציות הקידוד והפענוח שגודל המערך המשמש לפלט יהיה זהה בגודלו לערך של מערך הקלט.

פונקציה האחראית על תהליך ה-Transcoding:

```
void Translator::transcode_packet(std::vector<uint8_t>& packet, const std::shared_ptr<RtpPacket>& rtp,
    CodecVariant& from_codec, CodecVariant& to_codec)
{
    std::vector<int16_t>& active_pcm_buffer =
        (&from_codec == &m_codecA) ? m_pcmBufferAToB : m_pcmBufferBToA;

    active_pcm_buffer.clear();

    auto start = packet.begin() + rtp->payload_offset();
    auto end = packet.end() - rtp->number_of_padding_bytes();
    std::vector<uint8_t> raw_payload(start, end);

    std::visit([&](auto& codec) { codec.decompress(raw_payload, active_pcm_buffer); }, from_codec);
    std::visit([&](auto& codec) { codec.compress(raw_payload, active_pcm_buffer); }, to_codec);
    uint8_t new_payload_type = std::visit(
        [&](auto& codec)
        {
            return codec.get_payload_type();
        }, to_codec);

    packet.erase(start, end);
    packet.insert(start, raw_payload.begin(), raw_payload.end());

    update_payload_type(packet, new_payload_type, rtp->marker());
    LOG_DEBUG("Packet transcoded. New payload size: " + std::to_string(raw_payload.size()));
}
```

קלט: אובייקט עזר המייצג את פקטת ה-RTP, מזהה קודק המקור, ומזהה קודק היעד.

פלט: פקטת RTP שעברה תהליך transcoding על פי הקונפיגורציה. הפלט נכתב ישירות נכתב ישירות לפרמטר המועבר לפונקציה by reference.

מטרת האלגוריתם: הפונקציה אחראית על ביצוע תהליך ה-transcoding לפקטת RTP – המרת ה-payload של הפקטה מקודק המקור לקודק היעד. מדובר בפונקציית ליבה בפרויקט המאפשרת למערכת לשמש כמתווך בין נקודות קצה המשתמשות בקודקי שמע שונים.

זרימת הפעולה (main flow):

1. על פי קודק המקור, מזהים באיזה באפר PCM להשתמש בתהליך. באפר זה יהווה container עבור החזקה זמנית של מידע לא דחוס בפורמט PCM.
2. מאתחלים מערך דינמי שיכיל בתוכו רק את תוכן ה-payload עליו עושים את ה-transcoding.
3. על פי קודק המקור קוראים לפונקציית הפענוח.
4. על פי קודק היעד קוראים לפונקציית הקידוד.



5. מסירים מהפקטה לשליחה את ה-payload שמקודד בקודק המקור ושמים את ה-payload המקודד בקודק היעד.
6. מעדכנים את שדה ה-payload type שבתוך ה-RTP header של הפקטה.

הפונקציות העיקריות עבור UC-1:

עדכון דינאמי של ההשהיה ב-jitter buffer:



```
int16_t JitterBuffer::compute_optimal_delay()
{
    int16_t optimal_delay = 0;
    int minimum_cost = INT_MAX;
    int late_packet_count = 0;
    std::array<int, TIMING_HISTORY_COUNT> history_positions = { 0 };
    float late_packet_penalty_factor;
    bool penalty_applied = false;
    int latest_timestamp = 0;
    int earliest_timestamp = 0;
    int timestamp_difference;
    int total_count = 0;

    for (const TimingHistory& history : m_internalTimingHistories)
        total_count += history.m_totalTimingCount;

    if (total_count == 0)
        return 0;

    late_packet_penalty_factor = static_cast<float>(m_latencyTradeoff * m_latePacketWindowDuration) / total_count;

    for (size_t i = 0; i < TOP_DELAY; i++)
    {
        int selected_history_index = -1;
        int smallest_timing_value = INT16_MAX;
        for (size_t j = 0; j < m_internalTimingHistories.size(); j++)
        {
            if (history_positions[j] < m_internalTimingHistories[j].m_timings.size() &&
                m_internalTimingHistories[j].m_timings[history_positions[j]] < smallest_timing_value)
            {
                selected_history_index = j;
                smallest_timing_value = m_internalTimingHistories[j].m_timings[history_positions[j]];
            }
        }

        if (selected_history_index != -1)
        {
            int cost;
            if (i == 0)
            {
                earliest_timestamp = smallest_timing_value;
                latest_timestamp = smallest_timing_value;

                smallest_timing_value = round_down(smallest_timing_value, m_delayAdjustmentStep);
                history_positions[selected_history_index]++;

                cost = -smallest_timing_value + late_packet_penalty_factor * late_packet_count;
                if (cost < minimum_cost)
                {
                    minimum_cost = cost;
                    optimal_delay = smallest_timing_value;
                }
            }
            else
                break;

            late_packet_count++;
            if (smallest_timing_value >= 0 && !penalty_applied)
            {
                penalty_applied = true;
                late_packet_count += 4;
            }
        }

        timestamp_difference = latest_timestamp - earliest_timestamp;
        m_latencyTradeoff = 1 + timestamp_difference / TOP_DELAY;

        if (total_count < TOP_DELAY && optimal_delay > 0)
            return 0;

        return optimal_delay;
    }
}
```

קלט: אין.



פלט: ערך מספרי המייצג את זמן ההשהיה האופטימלי ביחידות RTP timestamp שבו השעון הפנימי של המערכת צריך להתעדכן

מטרת האלגוריתם: להעריך את זמן ההשהיה האופטימלי של פקטות השמע המגיעות למודול ה-jitter buffer, כך שהמערכת תוכל להוציא אותן בקצב סדיר ובמינימום עיוותים. האלגוריתם משקלל נתוני היסטוריה של זמני הגעה של הפקטות תוך נתינת עונש לפקטות שאיחרו ומציאת איזון בין הקטנת ההשהיה של הבאפר ובין הקטנה של איבוד פקטות מאוחרות.

זרימת פעולה (main flow):

1. חישוב של כמות הפקטות שעברו בתוך המערך של התזמונים (כולל כאלה שלא נכנסו).
2. חישוב מקדם העונש עבור פקטות מאוחרות. ערך זה מייצג את המשקל היחסי של כל פקטה שתחשב כמאחרת והוא העונש היחסי שמתווסף לכל תרחיש השהיה. משתנה זה מבטא עד כמה המערכת סובלת אובדן לעומת השהיה.
3. הלולאה העיקרית מבצעת בחינה של עד TOP_DELAY פקטות בעלות ערכי תזמון "מאוחרים" ביותר, במטרה להעריך לכל אחת את עלות השימוש בהשהיה הקשור אליה. בכל איטרציה:
 - a. מאותרת הפקטה הבאה בעלת ערך הזמן המינימלי (כלומר המאוחרת ביותר בפועל).
 - b. ערך התזמון מעוגל כלפי מטה לפי קפיצת השהיה על פיו הבאפר פועל.
 - c. מחושבת פונקציית עלות (cost function) על פי הנוסחה:

$$\text{cost} = -\text{latest} + \text{late_factor} * \text{late};$$

latest הוא זמן ההגעה של הפקטה, late הוא מספר הפקטות שנחשבות כמאוחרות בתרחיש זה, cost משקלל את האיזון בין ההשהיה לבין השפעת הפקטות המאוחרות. המטרה היא לחפש את המחיר הכי נמוך. בנוסחה, הפרמטר (-latest) מייצג את מחיר השהיה – ככל שההשהיה גבוהה יותר כך נרצה להקטין את ההשהיה כדי לשפר את התגובתיות, ולכן מפחיתים אותה מהעלות. בנוסחה, החישוב $\text{late_factor} * \text{late}$ מייצג את מחיר איבוד הפקטות – ככל שנקצר את ההשהיה, יותר פקטות ייחשבו כמאוחרות וייאבדו. עלות זו חיובית מפני שהיא פוגעת באיכות השמע.
- d. בכדי למנוע קפיצות חדות בזמן ההשהיה, לפונקציה יש מנגנון Hysteresis – ברגע שהפונקציה מזהה שזמן ההשהיה המוצע חיובי (כלומר צריך להגדיל את ההשהיה) ועדיין לא נלקח עונש על ההשהיה, מוסיפים באופן מלאכותי 4 פקטות מאוחרות לחישוב. עושים זאת כדי להגדיל את עלות האפשרות של הגדלת ההשהיה ובכך למנוע מהמנגנון לקפוץ מידית להשהיה גבוהה מדי בגלל פקטה אחת חריגה.
4. מתבצע עדכון של התכונה m_latencyTradeoff על סמך טווח הפיזור של זמני ההגעה שנמדדו כך שבפעמים הבאות המערכת תתאים טוב יותר את הערכה על סמך שונות בתזמון.
5. מחזירים את הערך של ההשהיה האופטימלית ביחידות RTP timestamp בתנאי שגם היו מספיק פקטות לאסוף עליהם מידע ולהסיק מהם מידע מספק.



```
void JitterBuffer::update_delay()
{
    int16_t opt = compute_optimal_delay();

    if (opt < 0)
    {
        shift_timings(-opt);
        m_nextPlayoutTimestamp += opt;
        m_interpolationRequested += opt;

        LOG_INFO("JitterBuffer::update_delay(): advanced playout backwards (interpolation). "
            "opt=" + std::to_string(opt) +
            ", next_playout=" + std::to_string(m_nextPlayoutTimestamp) +
            ", interpolation_requested=" + std::to_string(m_interpolationRequested));
    }
    else if (opt > 0)
    {
        shift_timings(-opt);
        m_nextPlayoutTimestamp += opt;

        LOG_INFO("JitterBuffer::update_delay(): delayed playout forward. "
            "opt=" + std::to_string(opt) +
            ", next_playout=" + std::to_string(m_nextPlayoutTimestamp));
    }
    else
    {
        LOG_INFO("JitterBuffer::update_delay(): no delay adjustment needed. opt=0");
    }
}
```

קלט: אין.

פלט: אין.

מטרת האלגוריתם: לבצע התאמה דינמית לערך ההשהיה של ניגון פקטות השמע בהתאם לשינויים בזמן הגעת הפקטות. מטרת ההתאמה היא להבטיח רציפות שמע חלקה, גם בתנאים של תנודות ברשת תוך הימנעות מהגדלה מיותרת של ההשהיה או איבוד פקטות.

זרימת הפעולה (main flow):

1. ראשית קוראים לפונקציה `compute_optimal_delay` לקבלת ערך ההשהיה נדרש.
2. בדיקת הערך שהתקבל:
 - a. אם מוחזר ערך שלילי -> נדרש להפחית את ההשהיה. השעון הפנימי מוזז אחורה ונרשם גודל הפקטה שצריך לצורך interpolation.
 - b. אם מוחזר ערך חיובי -> זה אומר שהפקטות "מתקרבות מהר מידי" לכן מזיזים אחורה את התזמונים שנאגרו בהיסטוריה ודוחים את זמן ההשמעה. כלומר במצב כזה מגדילים את ההשהיה כדי למנוע הגדלת כמות איחורים בלתי צפויים.
 - c. אחרת המצב תקין ולא מבצעים שום שינוי.



```
void JitterBuffer::advance_time()
{
    LOG_DEBUG("JitterBuffer::advance_time(): advancing time...");

    update_delay();

    if (m_applicationBufferedTime >= 0)
    {
        m_nextRetrievalDeadline = m_nextPlayoutTimestamp - m_applicationBufferedTime;
        LOG_INFO("JitterBuffer::advance_time(): next_retrieval_deadline = next_playout_timestamp - application_buffered_time = " +
            std::to_string(m_nextPlayoutTimestamp) + " - " + std::to_string(m_applicationBufferedTime) +
            " = " + std::to_string(m_nextRetrievalDeadline));
    }
    else
    {
        m_nextRetrievalDeadline = m_nextPlayoutTimestamp;
        LOG_WARNING("JitterBuffer::advance_time(): application_buffered_time < 0. Using playout timestamp directly.");
    }

    m_applicationBufferedTime = 0;
}
```

קלט: אין.

פלט: אין.

תיאור האלגוריתם: לנהל את התקדמות הזמן הפנימית של מנגנון ה-jitter buffer בהתאם לתנאי הרשת ולחשב את זמן ה-deadline שבו פקטת שמע צריכה להישלף מהבאפר לצורך המשך העיבוד. פונקציה זו מסופקת לאפליקציה בתור API ציבורי והיא מופעלת באופן מחזורי על מנת לעדכן את מצבה הפנימי של המנגנון לקראת הוצאה של הפקטה הבאה.

זרימת הפעולה (main flow):

1. מתבצע עדכון דינמי של השעיית השמע בהתאם למצב הרשת על ידי הקריאה לפונקציה `update_delay`.
2. חישוב מועד ה-deadline הבא –
 - a. אם בהוצאה האחרונה מהבאפר הוצאו בדיוק/פחות כמות השמע שהאפליקציה ביקשה -> מתבצע סנכרון בין זמן ההוצאה עם זמן ההשמעה הרצוי תוך הבטחת השהיה מינימלית
 - b. במקרה בו הוצא מהבאפר יותר ממנה שהאפליקציה ביקשה -> המנגנון מתריע על תקלה אפשרית בלוג, ומשתמשת ישירות בשעון הפנימי כבסיס להשמעה.
3. מאפסים את כמות הזמן שנאגר לקראת המחזור הבא.



חיפוש אחר פקטה להוצאה מהבאפר:

```
JitterBuffer::StatusType JitterBuffer::pop(JitterBufferPacket& packet, uint32_t desired_duration)
{
    if (m_buffer.empty())
    {
        LOG_INFO("JitterBuffer::pop(): Buffer empty, returning null packet.");
        packet.timestamp = 0;
        packet.duration = 0;
        return JITTER_BUFFER_EMPTY;
    }

    if (m_isResetPending)
    {
        align_playout_timestamp();
        m_isResetPending = false;
    }

    m_lastReturnedTimestamp = m_nextPlayoutTimestamp;

    if (m_interpolationRequested != 0)
    {
        LOG_INFO("JitterBuffer::pop(): Interpolation triggered. duration=" + std::to_string(m_interpolationRequested));
        handle_interpolation_request(packet, desired_duration);
        return JITTER_BUFFER_INSERTION;
    }

    auto remove_packet = find_best_packet(desired_duration);

    if (remove_packet != m_buffer.end())
    {
        LOG_INFO("JitterBuffer::pop(): Selected packet ts=" + std::to_string(remove_packet->timestamp));
        process_selected_packet(packet, remove_packet, desired_duration);
        return JITTER_BUFFER_OK;
    }

    LOG_INFO("JitterBuffer::pop(): Packet loss, no suitable packet found.");
    return handle_packet_loss(packet, desired_duration);
}
```

קלט: אורך השמע ביחידות RTP timestamp שהאפליקציה רוצה לקבל.

פלט: סטטוס היציאה המתאים. הפלט נכתב ישירות לפרמטר המועבר לפונקציה by reference.

מטרת האלגוריתם: הפונקציה אחראית על הוצאת פקטת שמע להמשך עיבוד מתוך ה-jitter buffer. היא מנהלים מצבים שונים כגון אם יש פקטה להוצאה, אובדן פקטות וביצוע interpolation במידה ויש חוסר נתונים. מטרתה להבטיח זרימה חלקה של השמע לעבר נקודת היעד.

זרימת פעולה (main flow):

1. החזרת סטטוס מתאים עבור באפר ריק – אין פקטות להשמעה.
2. אם זו הקריאה הראשונה לשליפה אחרי האיפוס של הבאפר – מאתחלים את השעון הפנימי של הבאפר.
3. אם לאור הקריאה לפונקציה advance_time נוצרה בקשה ל-Interpolation יוצרים פקטת שקט סינתטית.
4. מחפשים את הפקטה הכי מתאימה להוצאה אשר מתאימה גם לשעון הפנימי וגם לכמות השמע המבוקשת
5. עבור התוצאה שקיבלנו:
 - a. אם מצאנו פקטה מתאימה -> מוציאים אותה מהבאפר ומעדכנים את תכונות המחלקה.
 - b. אם לא מצאנו פקטה מתאימה -> יוצרים פקטה סינתטית של שקט.



```
JitterBufferIterator JitterBuffer::find_best_packet(uint32_t desired_duration)
{
    if (auto it = find_exact_match(desired_duration); it != m_buffer.end()) return it;
    if (auto it = find_overlap_match(desired_duration); it != m_buffer.end()) return it;
    if (auto it = find_fuzzy_match(desired_duration); it != m_buffer.end()) return it;
    return find_best_partial_fit(desired_duration);
}
```

קלט: אורך השמע הרצוי לעיבוד

פלט: iterator לפקטה הכי מתאימה להוצאה מהבאפר

תיאור האלגוריתם: פונקציה זו אחראית לחפש את הפקטה הטובה ביותר להוצאה מתוך ה-jitter buffer בהתאם לזמן הרצוי לניגון. החיפוש מתבצע בצורה הדרגתית, תוך ניסיון לאתר התאמה אופטימלית, ואם אין התאמה מושלמת – לבחור את הפקטה הקרובה ביותר מבחינת התאמה. מטרת האלגוריתם הוא למצוא את הפקטה הכי מתאימה בזמן המבוקש, להבטיח קצב השמעה עקבי אף במצבים של חוסר עקביות בהגעת הפקטות ולאזן בין איכות השמע לבין רציפות השידור גם בתנאי רשת לא יציבים.

```
/* Search the buffer for a packet with the right timestamp and spanning the whole current chunk */
JitterBufferIterator JitterBuffer::find_exact_match(uint32_t desired_duration)
{
    return std::find_if(m_buffer.begin(), m_buffer.end(), [&](const JitterBufferPacket& item)
    {
        return item.timestamp == m_nextPlayoutTimestamp &&
            RtpUtilities::is_timestamp_newer_or_equal(item.timestamp + item.duration,
                m_nextPlayoutTimestamp + desired_duration);
    });
}
```

קלט: אורך השמע הרצוי לעיבוד.

פלט: iterator לפקטה הראשונה בבאפר שעונה על הדרישות.

מטרת האלגוריתם: לנסות למצוא התאמה מושלמת של פקטה שתכסה במדויק את אורך הזמן הנדרש לעיבוד, ללא צורך בהשלמות או תיקונים.

זרימת פעולה (main flow):

1. עבור כל פקטה בבאפר בצע:
 - a. אם שדה ה-timestamp של הפקטה תואם בדיוק את זמן הניגון הבא וגם אורכה זהה או גדול מאורך השמע המבוקש:
 - i. החזר iterator לפקטה זו.
 2. החזר iterator המייצג את סוף הבאפר אם לא נמצאה תוצאה.



```
/* If no match, try for an "older" packet that still spans (fully) the current chunk */
JitterBufferIterator JitterBuffer::find_overlap_match(uint32_t desired_duration)
{
    return std::find_if(m_buffer.begin(), m_buffer.end(), [&](const JitterBufferPacket& item)
    {
        return RtpUtilities::is_timestamp_older_or_equal(item.timestamp, m_nextPlayoutTimestamp) &&
            RtpUtilities::is_timestamp_newer_or_equal(item.timestamp + item.duration,
                m_nextPlayoutTimestamp + desired_duration);
    });
}
```

קלט: אורך השמע הרצוי לעיבוד.

פלט: iterator לפקטה הראשונה בבאפר שעונה על הדרישות.

מטרת האלגוריתם: אם לא נמצאה התאמה עבור התנאי הראשון, מחפשים פקטה ישנה יותר מהזמן שבשעון הפנימי, אך ארוכה מספיק כדי לכסות את הזמן המבוקש על ידי האפליקציה. המטרה היא לאפשר שימוש בפקטות שהגיעו קצת מוקדם יותר מהנדרש, כל עוד הן עדיין רלוונטיות ומספיקות לאורך הזמן הדרוש.

זרימת פעולה (main flow):

1. עבור כל פקטה בבאפר בצע:

a. אם ה-timestamp של הפקטה ישן או זהה מהזמן שבשעון הפנימי וגם הוא

מכסה את הזמן המבוקש על ידי האפליקציה:

i. החזר iterator לפקטה זו.

2. החזר iterator המייצג את סוף הבאפר אם לא נמצאה תוצאה.

```
/* If still no match, try for an "older" packet that spans part of the current chunk */
JitterBufferIterator JitterBuffer::find_fuzzy_match(uint32_t desired_duration)
{
    return std::find_if(m_buffer.begin(), m_buffer.end(), [&](const JitterBufferPacket& item)
    {
        return RtpUtilities::is_timestamp_older_or_equal(item.timestamp, m_nextPlayoutTimestamp) &&
            RtpUtilities::is_timestamp_newer(item.timestamp + item.duration, m_nextPlayoutTimestamp);
    });
}
```

קלט: אורך השמע הרצוי לעיבוד.

פלט: iterator לפקטה הראשונה בבאפר שעונה על הדרישות.

תיאור האלגוריתם: אם עדיין לא נמצאה פקטה, מחפשים פקטה ישנה יותר שמתחילה לפני זמן הניגון אבל לא בהכרח מכסה את כל הטווח. המטרה היא לשמר את זרימת השמע גם אם יש אובדן חלקי, ולהשתמש בפקטות זמינות גם אם הן לא מושלמות.

זרימת פעולה (main flow):

1. עבור כל פקטה בבאפר בצע:

a. אם ה-timestamp של הפקטה ישן או זהה מהזמן שבשעון הפנימי וגם אורכו

עובר את הזמן של השעון הפנימי (וידוא שהפקטה חוצה לפחות את זמן

הניגון ולא נגמרת לפניו)

i. החזר iterator לפקטה זו.

2. החזר iterator המייצג את סוף הבאפר אם לא נמצאה תוצאה.



```
/* If still no match, try for earliest packet possible */
JitterBufferIterator JitterBuffer::find_best_partial_fit(uint32_t desired_duration)
{
    bool found = false;
    uint32_t best_time = 0;
    int best_duration = 0;
    auto besti = m_buffer.begin();

    for (auto item = m_buffer.begin(); item != m_buffer.end(); ++item)
    {
        /* check if packet starts within current chunk */
        if (RtpUtilities::is_timestamp_older(item->timestamp, m_nextPlayoutTimestamp + desired_duration) &&
            RtpUtilities::is_timestamp_newer_or_equal(item->timestamp, m_nextPlayoutTimestamp))
        {
            if (!found || RtpUtilities::is_timestamp_older(item->timestamp, best_time) ||
                (item->timestamp == best_time && RtpUtilities::is_timestamp_newer(item->duration, best_duration)))
            {
                best_time = item->timestamp;
                best_duration = item->duration;
                besti = item;
                found = true;
            }
        }
    }

    return found ? besti : m_buffer.end();
}
```

קלט: אורך השמע הרצוי לעיבוד.

פלט: iterator לפקטה הראשונה בבאפר שעונה על הדרישות.

תיאור האלגוריתם: בפונקציה זו מחפשים את הפקטה הראשונה האפשרית שמתחילה בתוך חלון הזמן המבוקש, גם אם היא חלקית ולא מושלמת. מטרתה היא להבטיח שתמיד תהיה פקטה לניגון גם אם היא לא לגמרי מתאימה. באלגוריתם ניתנת עדיפות לפקטות שהתחילו הכי קרוב לזמן הניגון.

זרימת פעולה (main flow):

1. עבור כל פקטה בבאפר בצע:
 - a. בחר את הפקטה שהכי מתאימה להתחיל לשדר ממנה.
 - b. אם יש כמה אפשרויות – בחר את הפקטה עם ה-timestamp הישן יותר או עם ה-duration הארוך יותר.
2. עבור סיום הסריקה:
 - a. אם נמצאה פקטה -> החזר iterator אליה בבאפר.
 - b. אחרת -> החזר iterator המייצג את סוף הבאפר.



```
JitterBuffer::StatusType JitterBuffer::handle_packet_loss(JitterBufferPacket& packet, uint32_t desired_duration)
{
    m_consecutiveLostPacketsCount++;
    int16_t opt = compute_optimal_delay();

    if (opt < 0)
    {
        shift_timings(-opt);
        packet.timestamp = m_nextPlayoutTimestamp;
        packet.duration = -opt;
        packet.sequence_number = m_lastReturnedSequenceNumber++;
        packet.data = m_concealer->get_packet(packet.sequence_number, packet.timestamp, packet.duration);
        packet.rtp_obj = std::make_shared<RtpPacket>(packet.data);
        m_applicationBufferedTime = packet.duration - desired_duration;
        return JITTER_BUFFER_INSERTION;
    }
    else
    {
        packet.timestamp = m_nextPlayoutTimestamp;
        desired_duration = round_down(desired_duration, m_concealmentUnitSize);
        packet.duration = desired_duration;
        packet.sequence_number = m_lastReturnedSequenceNumber++;
        packet.data = m_concealer->get_packet(packet.sequence_number, packet.timestamp, packet.duration);
        packet.rtp_obj = std::make_shared<RtpPacket>(packet.data);
        m_nextPlayoutTimestamp += desired_duration;
        m_applicationBufferedTime = packet.duration - desired_duration;
        return JITTER_BUFFER_MISSING;
    }
}
```

קלט: אורך השמע המבוקש.

פלט: המבנה בו יאוחסן הפקטה (הנתונים לביצוע PLC). הפלט נכתב ישירות נכתב ישירות לפרמטר המועבר לפונקציה by reference.

תיאור האלגוריתם: הפונקציה אחראית להתמודדות עם מצבי קצה בהם אין פקטה זמינה בבאפר להשמעה בנקודת זמן נדרשת. היא מבצעת פעולה אוטומטית ליצירת פקטה סינתטית תוך ניסיון למזער את השפעת האובדן על רציפות השמע. מטרת האלגוריתם הוא לשמר את רצף הזרימה של השמע גם כאשר יש אובדן פקטות, להקטין את תחושת השיבוש בשיחה על ידי יצירת פקטות שקט סינתטיות ולבצע תיקון של השעון הפנימי כדי לשמור על סנכרון עם הזרמת השמע.

זרימת פעולה (main flow):

1. מגדילים את הרצף של אובדן הפקטות, בכדי לעקוב אחר הרצף ובכדי לקבל אינדיקציה עתידית אם הבאפר אינו נמצא עוד בסנכרון עם הזרמת התקשורת.
2. מתאימים מחדש את ההשהיה כדי לבדוק אם יש צורך להקטין את ההשהיה של הבאפר.
3. עבור הערך שקיבלנו:
 - a. אם קיבלנו ערך שלילי -> המשמעות היא שהבאפר רוצה להקדים את זמן ההשמעה. לכן, מתאימים את התזמונים בהתאם ומכינים פקטה לצורך interpolation. בכוונה לא מעדכנים את השעון הפנימי כדי לשמור על סנכרון עם ניגון הפקטות. אי עדכון של השעון הפנימי חיוני בכדי להימנע בצבירה של טעויות סנכרון לאורך זמן.
 - b. אחרת -> מדובר באובדן פקטות רגיל. מייצרים פקטת PLC של שקט כאשר אורך הפקטה מעוגל כלפי מטה ביחידות של ה-concealment.



14.3 מבני נתונים בשימוש

1. **vector**: מבנה נתונים לינארי בגודל משתנה, המשמש לאחסון רציף של פריטים בזיכרון. מאפשר גישה אקראית מהירה ותומך בהוספה והסרה דינמית. בפרויקט נעשה בו שימוש ב-2 אופנים:
 - a. אחסון פקטות מהרשת – כאשר מתקבלות פקטות מהרשת, הן נפרסות לזיכרון באמצעות הקצאת vector אשר מאפשר עבודה נוחה עם המידע.
 - b. אחסון מידע PCM – מהווה buffer זמני עבור המידע הלא מקודד של הפקטה, משמש בתהליכי הפענוח והקידוד.
2. **array**: מערך סטטי בגודל קבוע הידוע בזמן קומפילציה. בפרויקט נעשה בו שימוש ב-3 אופנים:
 - a. אחסון frames בגודל קבוע – תהליכי הקידוד והפענוח של speex נעשים עבור כל frame כאשר כל frame באורך קבוע של 160 דגימות
 - b. **Lookup tables (LUT)** - מבנה נתונים המאחסן מראש תוצאות של חישובים כך שבמקום לחשב ערכים בזמן ריצה – המערכת פשוט מבצעת גישה ישירה לטבלה מוכנה. בפרויקט נעשה בו שימוש עבור ההמרה בין PCM ל-Alaw ולהיפך מה שמאפשר קידוד ופענוח מהירים יותר.
 - c. **Circular buffer** - מבנה נתונים לינארי עם התחלה וסוף "מחוברים", כך שברגע שמגיעים לסוף המערך – חוזרים להתחלה. משמש לאחסון זמני של תזמוני הפקטות בתוך ה-jitter buffer תוך מעקב מתמיד על הפקטות הכי עדכניות ומניעת הקצאות זיכרון חוזרות.
3. **deque**: מבנה המייצג Double Ended Queue המאפשר הוספה והסרה של פריטים הן מהקצה הקדמי והן מהאחורי. בפרויקט נעשה בו שימוש ב-2 אופנים:
 - a. מימוש ה-jitter buffer – השימוש המרכזי במבנה זה היה לצורך מימוש adaptive jitter buffer האחראי על אחסון ותזמון של פקטות לפי סדר הגעתן וזמן השמעתן. מכיוון שפקטות שמע מגיעות ברשת בקצב משתנה ולעיתים גם לא בסדר המקורי, מבנה זה מאפשר לנהל את הפקטות בצורה גמישה:
 - i. הוספת פקטות חדשות לקצה האחורי.
 - ii. שלילת פקטות מוכנות להשמעה מהקצה הקדמי.
 - iii. טיפול בפקטות שהגיעו שלא לפי הסדר, מבלי להזיז את שאר המבנה.
 - iv. ניהול תקין של השדה timestamp בעל תכונת wrap around - בזכות מבנה זה ניתן להבחין בין פקטות לפני ואחרי האיפוס, ולשמר את הסדר הלוגי הנכון שלהן גם כאשר הערכים מתאפסים.
 - b. ניהול תת חלונות תזמון – באמצעות deque, נעשה ניהול היסטוריית התזמונים של הפקטות המאחרות בכל אחד מן תתי החלונות במנגנון הניתוח הסטטיסטי של הרשת. כל תת חלון שומר את התזמון של הפקטות שהוא קלט וכאשר מגיע פקטה out-of-order אין צורך בהזזה יותר מידי של תזמונים בתוך המבנה.
 - c. ניהול שליחת הפקטות אל ה-frontend – המערכת משתמשת ב-deque כתור של הודעות שצריך לשלוח ל-frontend. כאשר מתקבלת בקשה לשליחת הודעה, היא מוכנסת לסוף המבנה, וברגע שהמבנה אינו עסוק



בכתיבה, מופעל תהליך שליחה אסינכרוני שבו מוסרת ההודעה הראשונה מן המבנה. באופן זה, ניתן לקבל בקשות לשליחה מכל מקום במערכת.

4. **unordered_map**: טבלה מבוססת hashing המאפשרת גישה מהירה לערכים לפי מפתח. בפרויקט נעשה בו שימוש ב-2 אופנים:

- a. מיפוי כתובות IP של מקורות שמע אל אובייקט השיחה המתאים. המפתח הוא כתובת ה-IP והפורט של מקור השמע והערך הוא מצביע לאובייקט RTP session המייצג את השיחה הרלוונטית.
- b. מיפוי המזהים הייחודיים של מקורות השמע (SSRC) אל אובייקט מקור השמע המתאים. המפתח הוא ערך SSRC והערך הוא מצביע לאובייקט RTP source המייצג את המקור הרלוונטי. כך בתוך השיחות, המיפוי בין המקורות מתבצע ברמת ה-RTP על ידי מיפוי לפי המזהים הייחודיים אשר הוקצו להם על ידי הפרוטוקול.

5. **variant**: מבנה נתונים המסוגל להכיל טיפוס אחד מתוך קבוצת טיפוסים נתמכים, בדומה למבנה union עם הוספה של type safety מה שהופך אותו ליותר בטוח לשימוש.

בפרויקט נעשה בו שימוש ב-2 אופנים:

- a. מודול ה-Translator – מאפשר לאחסן במשתנה יחיד את כל סוגי הקודקים בהם המערכת תומכת ולבחור את הקודק המתאים לפי קובץ קונפיגורציה.
- b. ניתוח פקטות RTCP – לצורך תמיכה במספר סוגי פקטות RTCP נעשה שימוש במבנה זה שמכיל את כל סוגי הפקטות. כך, ניתן לגשת אל האובייקטים המתאימים של כל סוג מבלי צורך בפולימורפיזם.

6. **property_tree**: עץ היררכי המייצג מבנה של קובץ קונפיגורציה. מתאים לקריאת קבצי קונפיגורציה כגון JSON/XML/INI תוך שמירה על גישה נוחה למפתחות וערכים. בפרויקט, מבנה זה שימש לטעינת קובץ הקונפיגורציה ולקבלת המידע בצורה היררכית. מבנה זה מאפשר שליפה נוחה של ערכים מהקובץ ודרכו המערכת מתנהלת תצורת הפרויקט כאשר בתחילת הריצה מבנה זה נוצר, ובזמן ריצה ניתן לגשת אל ערכיו בצורה נוחה ופשוטה. מבנה נתונים זה מסופק על ידי הספרייה boost::property_tree.

7. **unordered_set**: מבנה נתונים המייצג אוסף של ערכים ייחודיים, המאוחסנים בצורה שאינה תלויה בסדר ההכנסה. מבוסס על hash table ומספק זמן ריצה קבוע $O(1)$ עבור פעולות חיפוש, הוספה ומחיקה.

נעשה בו שימוש לצורך שליפה וניהול של רשימת הפורטים עליהם המערכת צריכה להאזין. המערכת במידת הצורך, מאזינה למספר פורטים עבור פקטות זמן אמת ועבור פקטות ניטור בזמן אמת על פי הקונפיגורציה, כך ששליפת הפורטים להאזנה בתוך מבנה זה מבטיח כי כל פורט יופיע פעם אחת בלבד מה שמאפשר יצירה מהירה של סוקט לכל זרם תקשורת נדרש.



14.4 חישוב יעילות האלגוריתם

חישוב יעילות האלגוריתם G711 A-law:

- **שלב יצירת טבלאות LUT – טבלאות ה-LUT של קידוד ופענוח G711 הן מערכים סטטיים בגודל קבוע וידוע מראש הנוצרים באתחול המערכת.** כיוון שהחישובים המבוצעים עבור כל איבר בטבלאות הם קבועים וגודל המערך ידוע מראש, זמן הריצה הוא $O(1)$.
- **תהליכי הקידוד והפענוח – עבור כל ערך של A-law או PCM מתבצע חיפוש שלו בתוך טבלת LUT על פי אינדקס – הזמן ריצה של המרה של ערך בודד הוא $O(1)$.** נסמן את גודל המערך של הערכים N . כאשר מבצעים קידוד/פענוח של מערך שלם המייצג את ה-payload זמן הריצה הכולל הוא $O(N)$.

קידוד ופענוח של Speex narrowband:

- **שלב יצירת מבני הנתונים – כל קריאה לאתחול מבני הנתונים של libspeex מבצעת הקצאת זיכרון קבועה, עם ערכים המוגדרים מראש על ידי מצב narrowband וערכי ברירת המחדל של המבנה SpeexBits.** לכן זמן הריצה הוא $O(1)$.
- **תהליך הקידוד – נגדיר N מספר הבתים שיש ב-payload ונגדיר F גודל frame אחד.** הלולאה בפונקציה רצה N/F פעמים כאשר ביצוע פונקציית הקידוד `speex_encode_int` היא בזמן ריצה $O(F)$. לכן, זמן הריצה הכולל של הפונקציה הוא $O(N/F) * O(F) = O(N)$.
- **תהליך הפענוח – נגדיר N מספר הבתים שיש ב-payload ונגדיר F גודל frame אחד ונגדיר B מספר ה-frames שמקודדים בפקטה.** טוענים את הפקטה לתוך `SpeexBits` שזהו $O(N)$, ומבצעים את פונקציית הפענוח `speex_decode_int` N/B פעמים שזמן ריצתה הוא $O(F)$. לכן, זמן הריצה הכולל של הפונקציה הוא $O(N) + O(F * N/B) = O(N)$.

חישוב זמן ההשהיה:

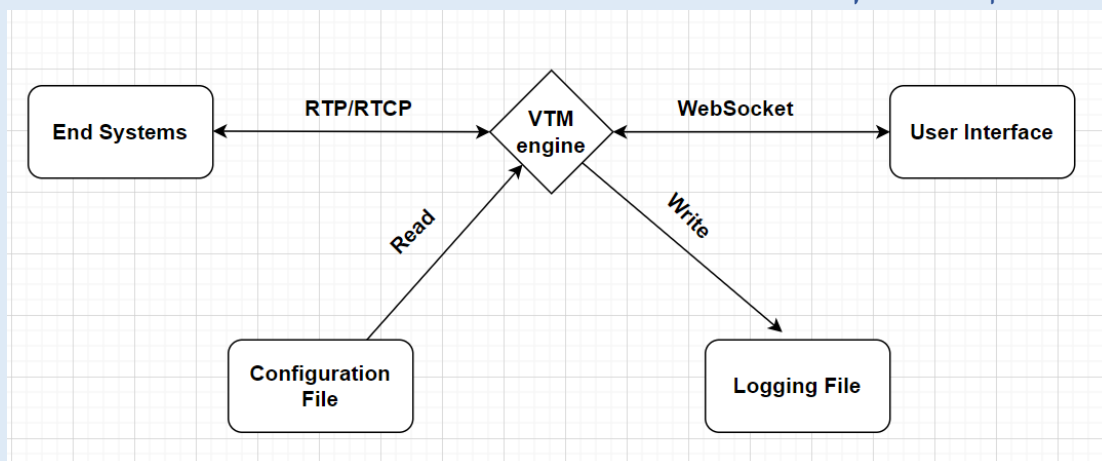
בחישוב זמן ההשהיה, נעשה שימוש ב-3 תת חלונות זמן, שכל אחת מייצגת רשימה של תזמונים של הפקטות המאוחרות. עבור כל תת חלון זמן, עוברים על `TOP_DELAY` פקטות "שמועמדות" לשמש לערך ההשהיה העתידי של הבאפר. מכיוון ש-`TOP_DELAY` הוא משתנה שהוא קבוע, זמן הריצה של האלגוריתם הוא $O(1) = O(3 * TOP_DELAY)$.

הוצאת פקטה מהבאפר:

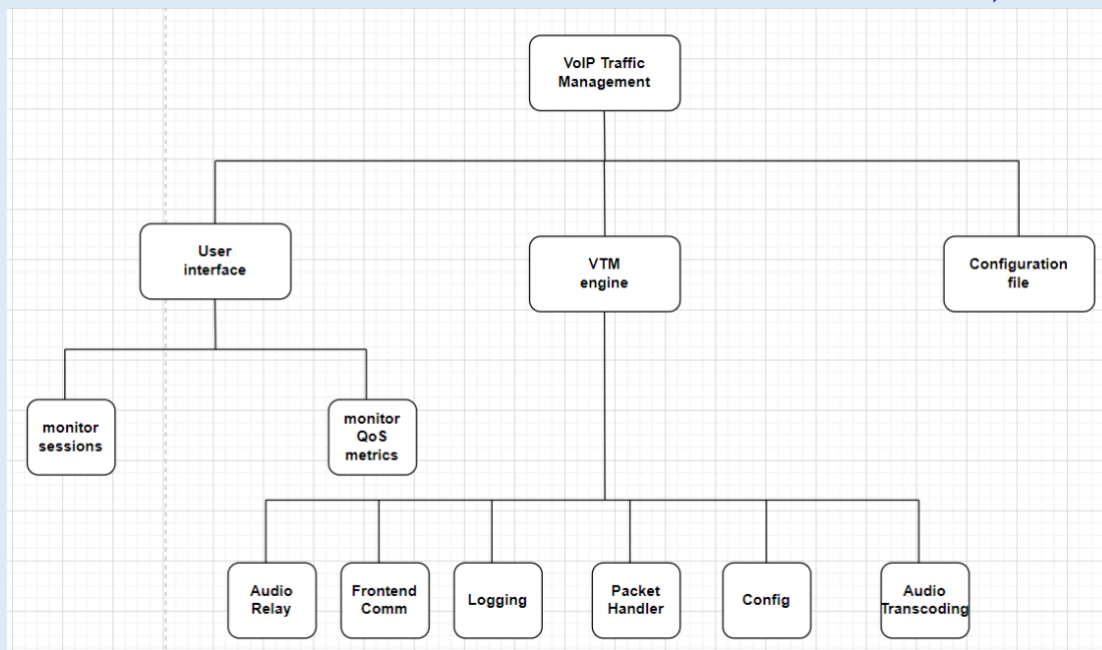
נסמן את גודל הבאפר בתור N . בתהליך הוצאת הפקטה, עושים בסך הכל 4 חיפוסים לינאריים על הבאפר, כאשר אם אין פקטה להוציא מתוך הבאפר עושים חישוב של זמן ההשהיה של הבאפר. לכן זמן הריצה של האלגוריתם הוא $O(4N + TOP_DELAY) = O(N)$.



14.5 הקשרים בין היחידות השונות

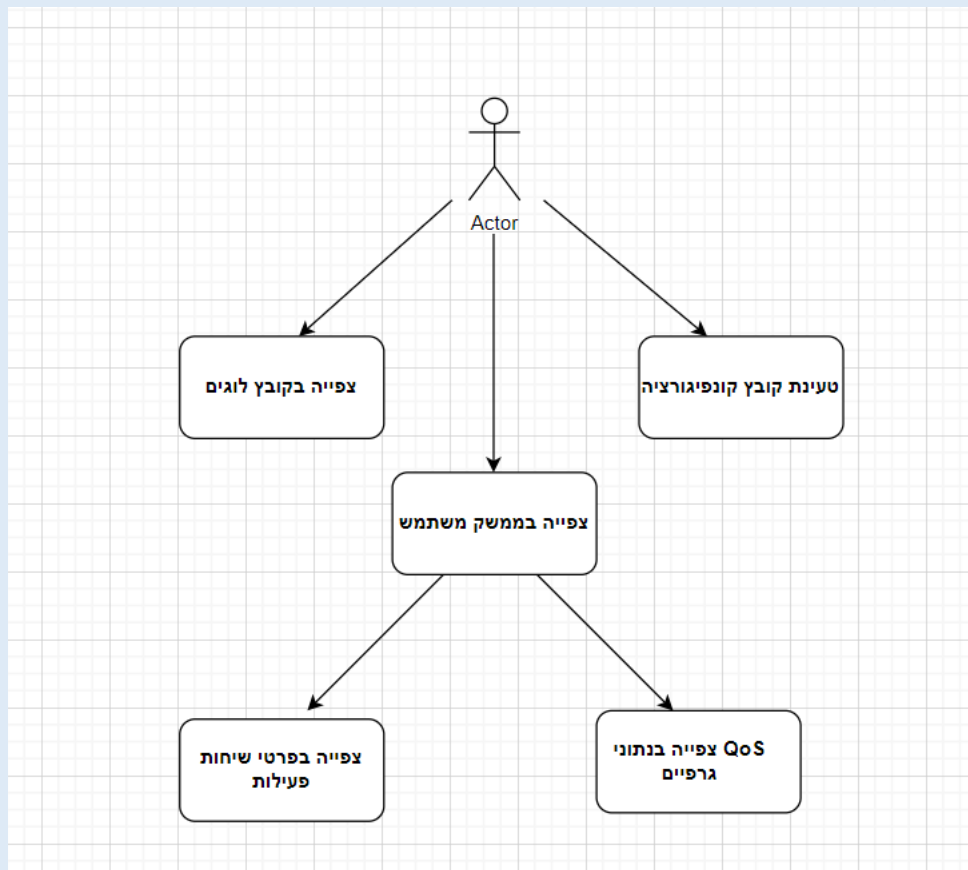


14.6 עץ מודולים



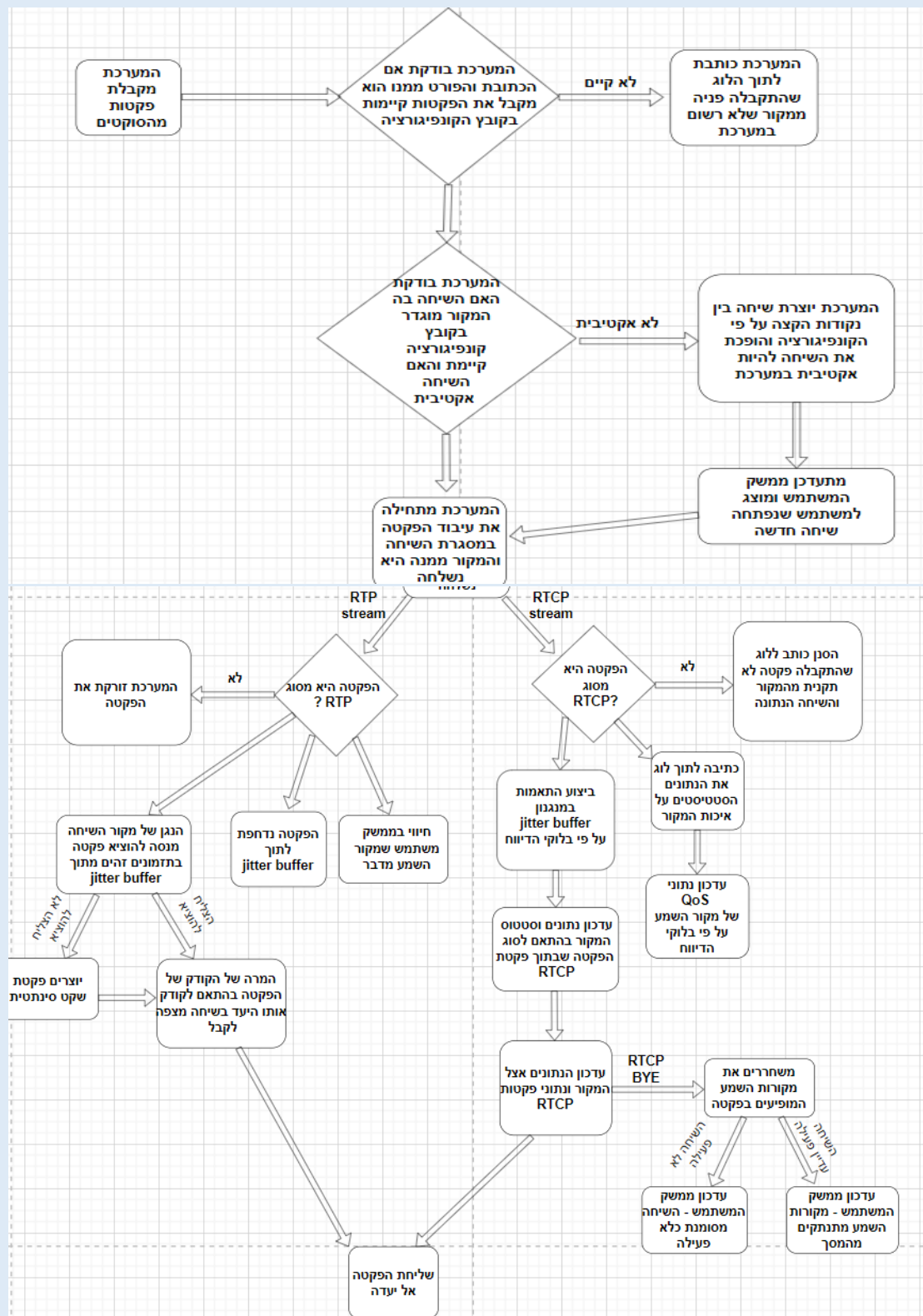


Use case Diagram 14.7



Use cases רשימת 14.8

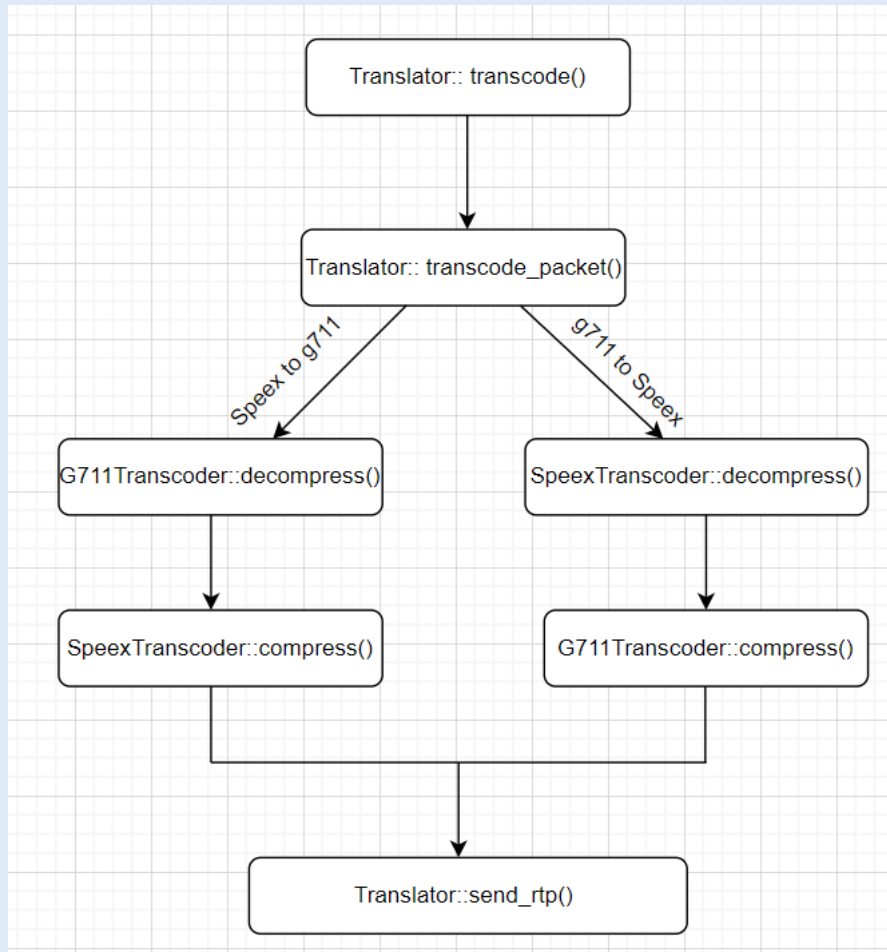
- **צפייה בשיחות פעילות:** במסך הראשי של הממשק משתמש, ניתן לצפות ברשימת שיחות פעילות ומתעדכן בסטטוס שלהן. ניתן לרענן את הרשימה או לבחור שיחה לצפייה.
- **ניתוח גרפי QoS:** עבור כל מקור שמע המופיע בממשק משתמש, ניתן ללחוץ על ה-icon שלו לקבלת גרפי QoS המציגים את מדדי הרשת שלו לאורך זמן לצורך ניטור.
- **העלאת קובץ הקונפיגורציה:** לפני הרצת המערכת, מנהל הרשת יכול לערוך את תצורת המערכת.
- **צפייה ובדיקת לוגים:** כל אירוע, שגיאה או תהליך קריטי נרשמים בזמן אמת לקובץ לוג ייעודי. המשתמש יכול לעיין בלוג כדי להבין פעילות פנימית של המערכת.

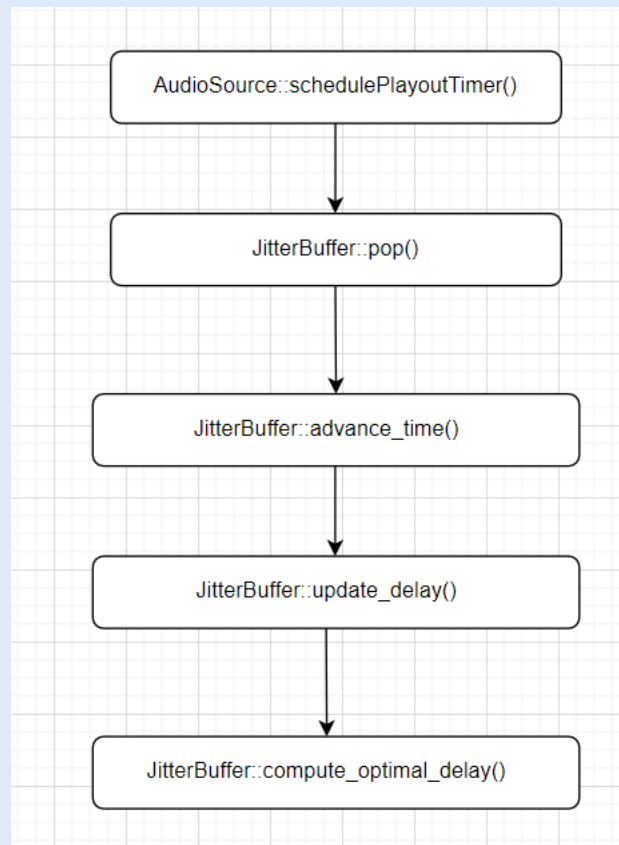




Sequence Diagrams 14.9.1

תרשים עבור UC-0:

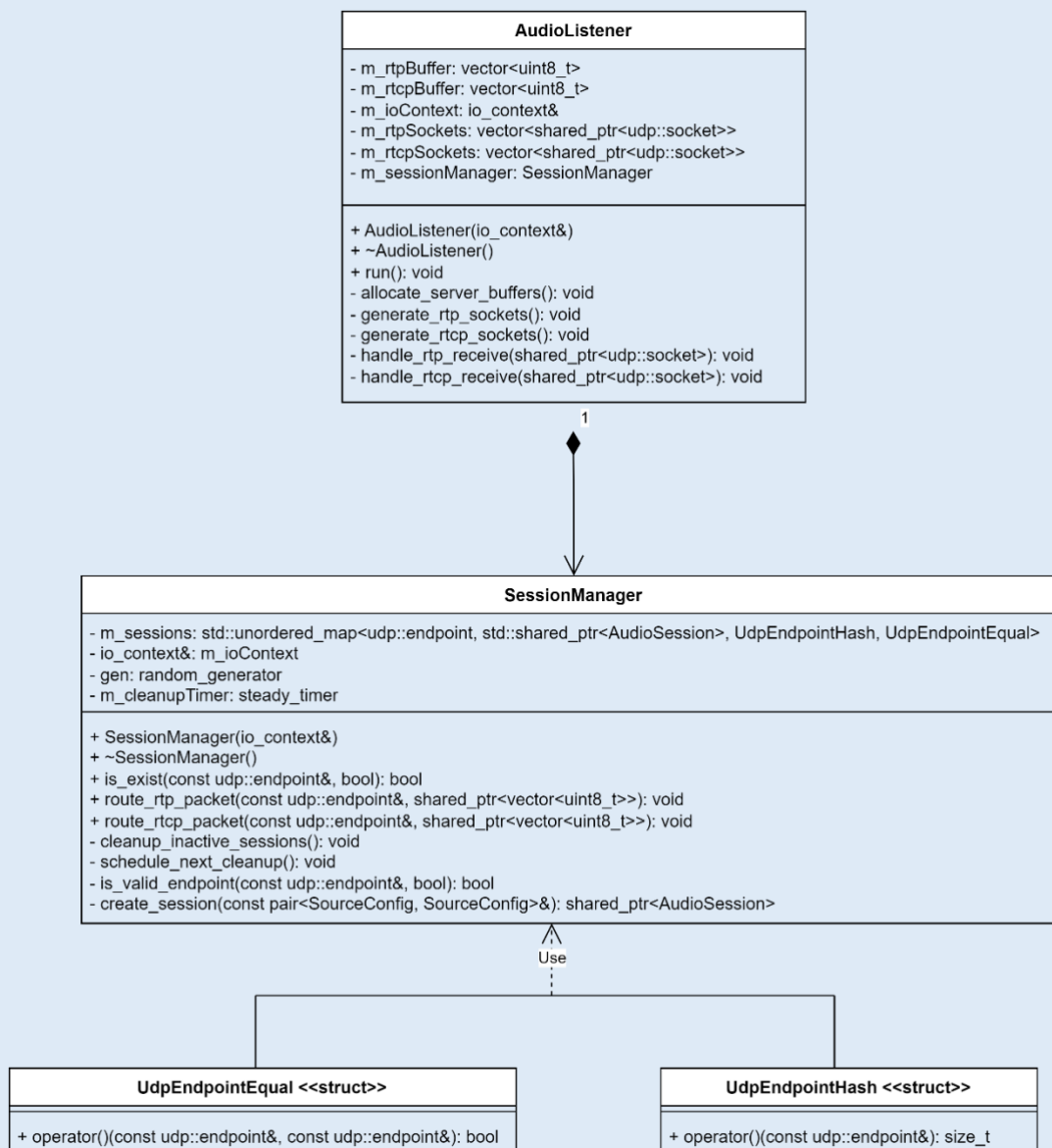


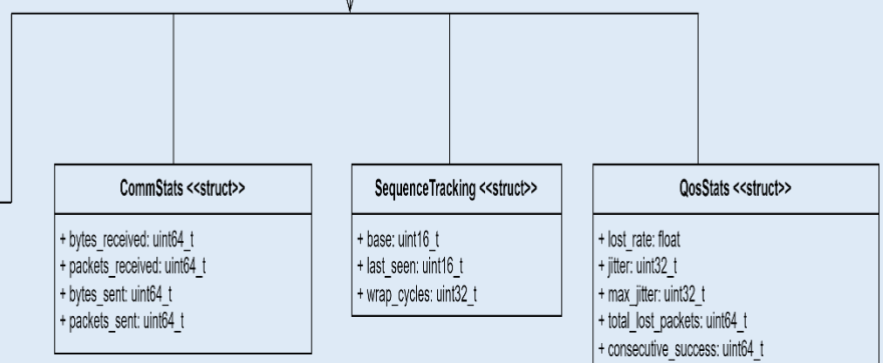
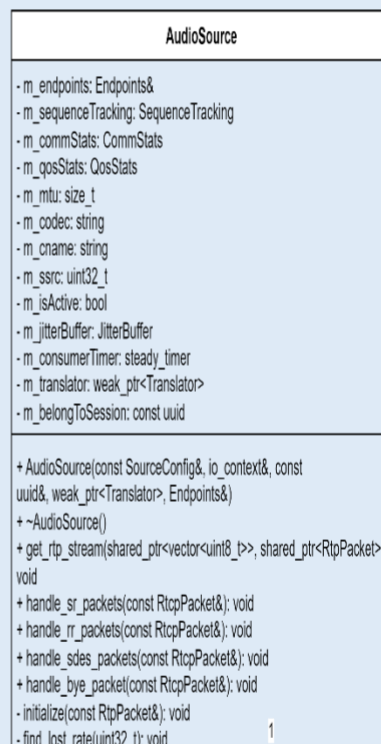
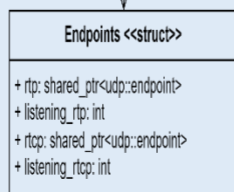
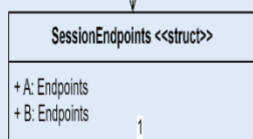
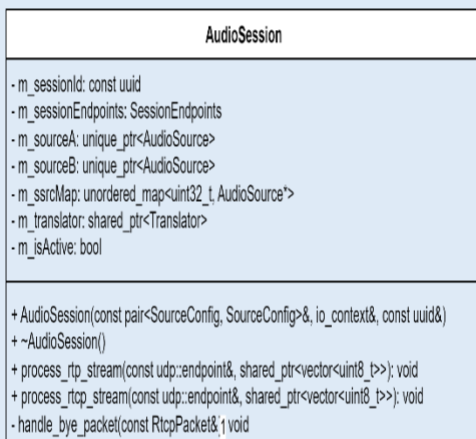
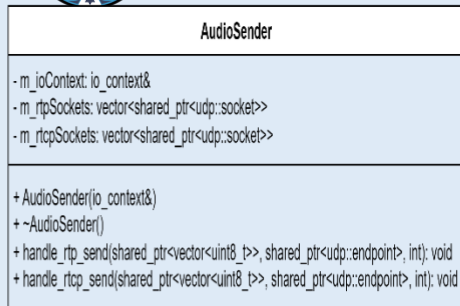




Design class Diagram 14.10

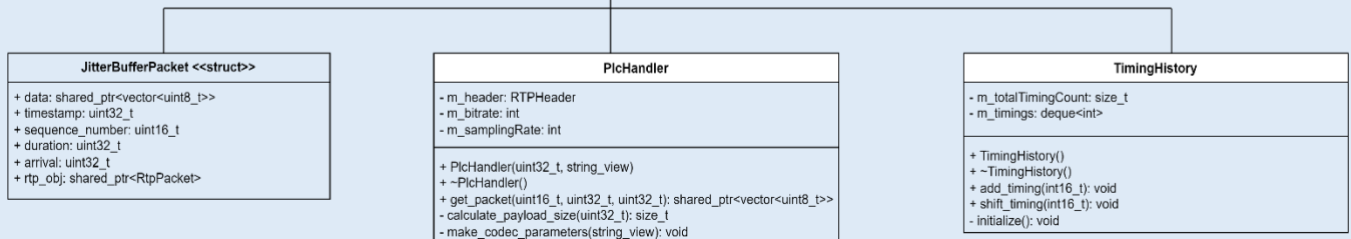
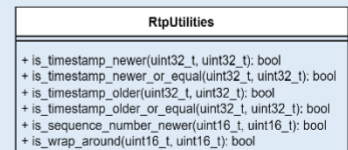
:Namespace AudioRelay





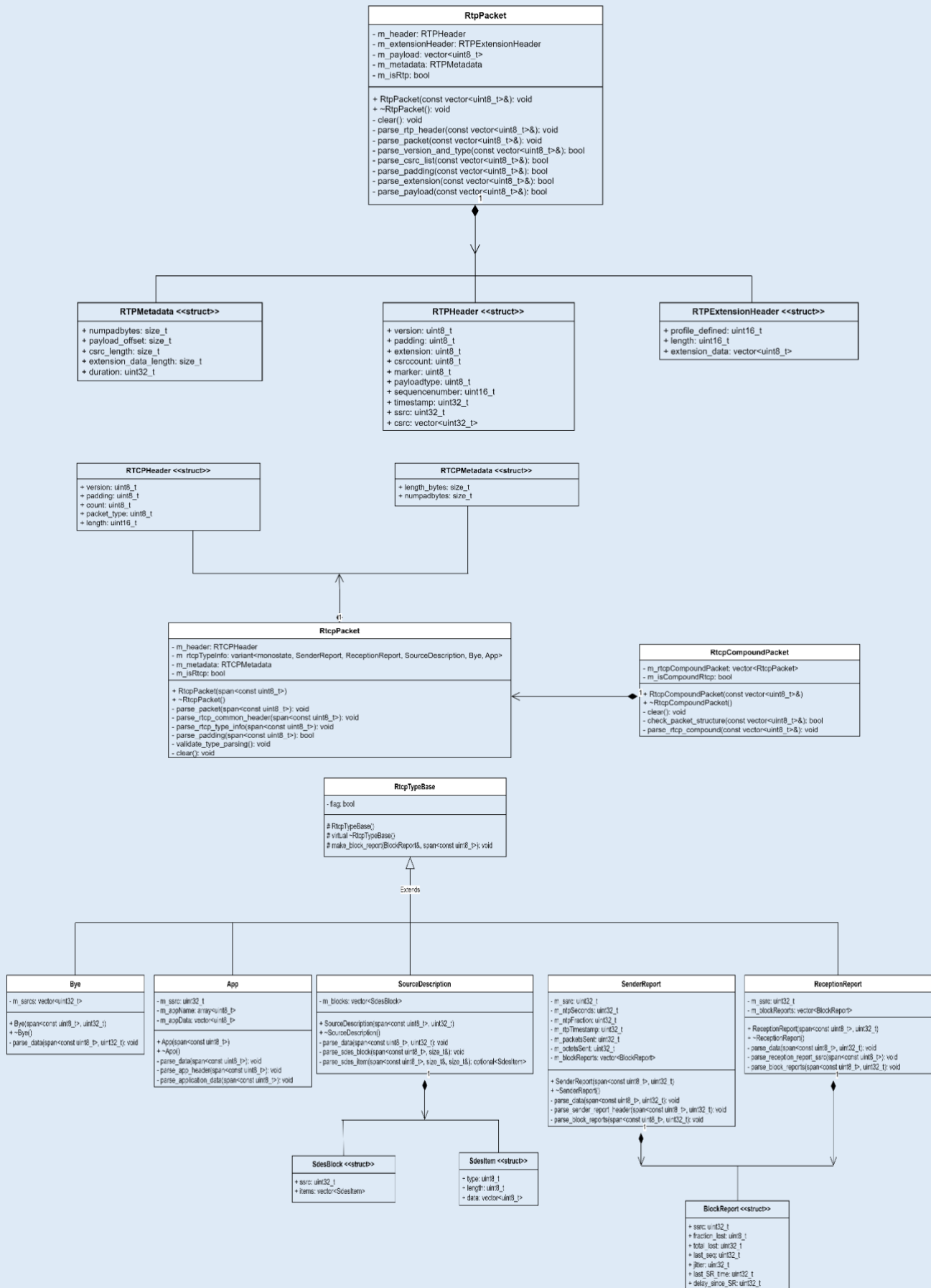


:Namespace PacketHandler





:Namespace PacketParser



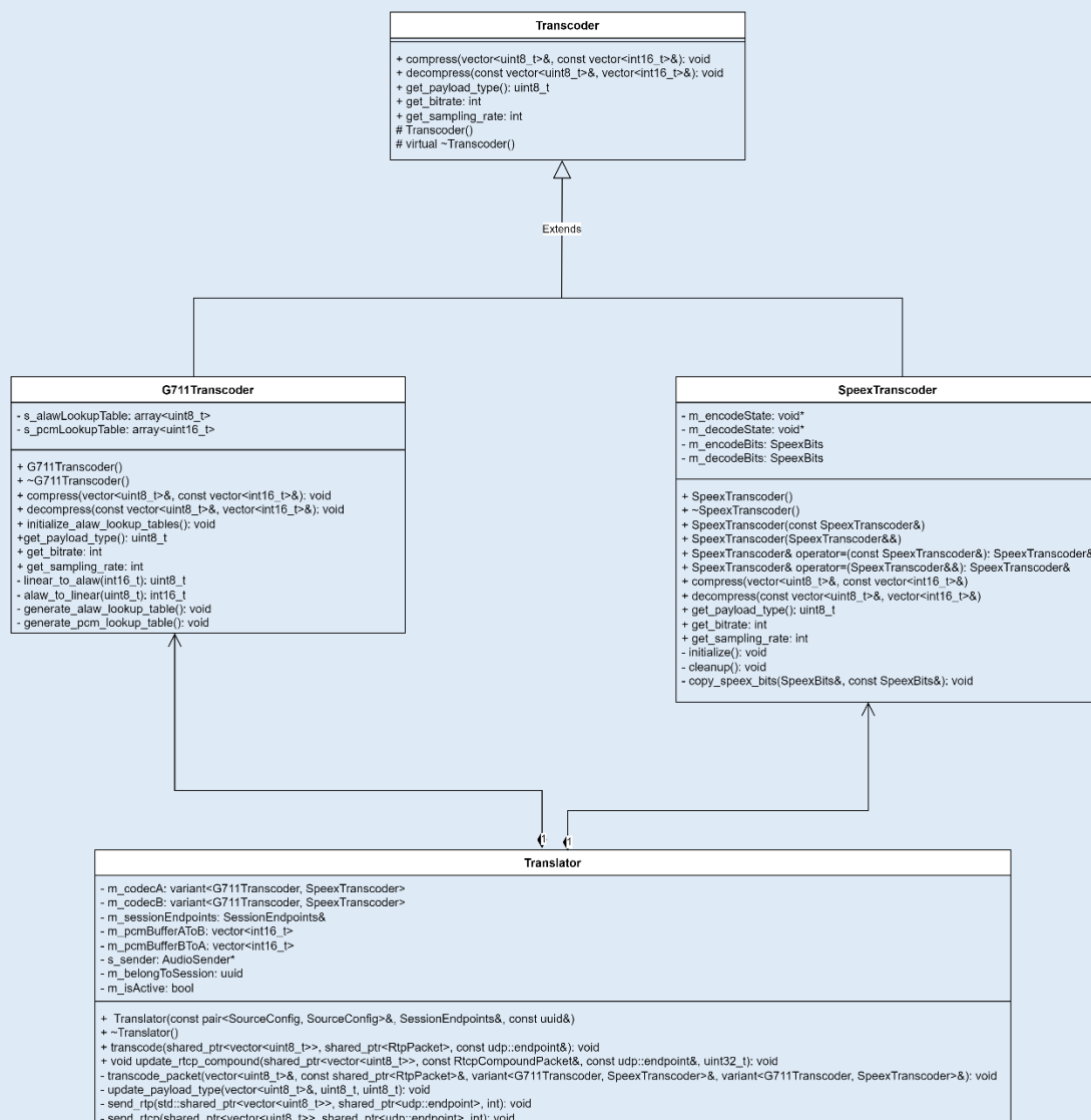


:Namespace FrontendComm

FrontendBridge
- m_ioContext: io_context - m_acceptor: tcp::acceptor - m_ws: shared_ptr<websocket::stream<tcp_stream>> - m_writeQueue: deque<string> - m_writing: bool - m_ioContextThread: thread - m_readBuffer: flat_buffer
- FrontendBridge() + ~FrontendBridge() + get_instance(): FrontendBridge& + send(const json::object&): void - FrontendBridge() - ~FrontendBridge() - do_accept(): void - on_accept(tcp::socket socket): void - do_write(): void - do_read(): void

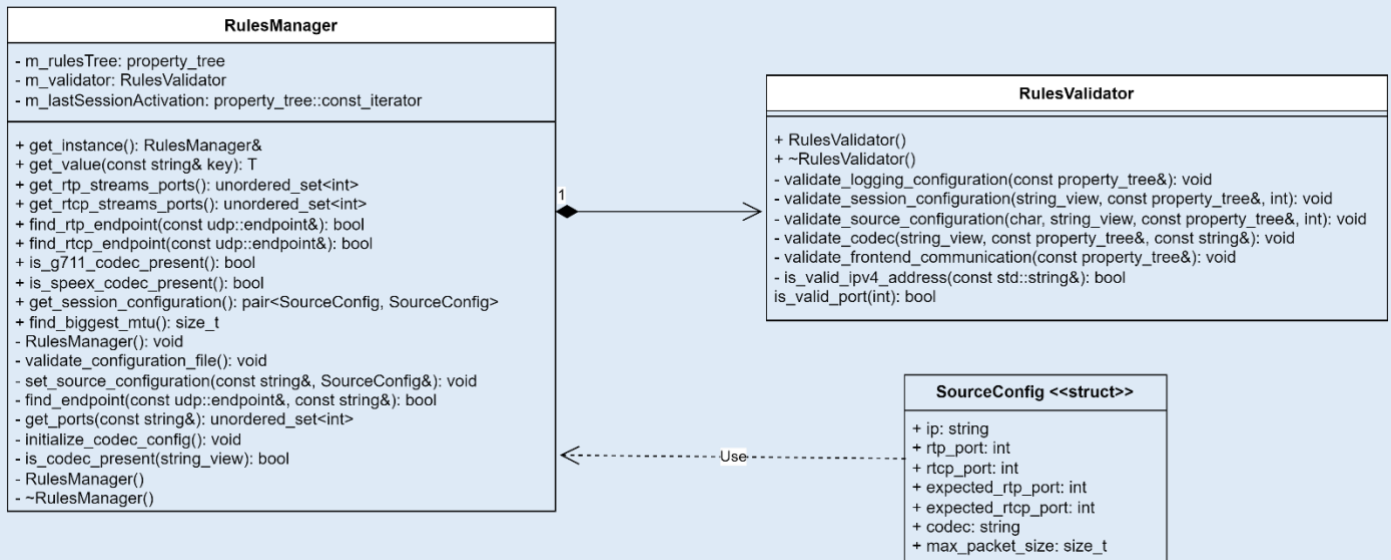
JsonMessageBuilder
+ session_init(const pair<SourceConfig, SourceConfig>&, const uuids::uuid&): json::object + session_end(const uuids::uuid&): json::object + talking(const udp::endpoint&, const uuids::uuid&): json::object + disconnected(const udp::endpoint&, const uuids::uuid&): json::object + qos_update(const udp::endpoint&, const uuids::uuid&, float, float): json::object

:Namespace AudioTranscoding

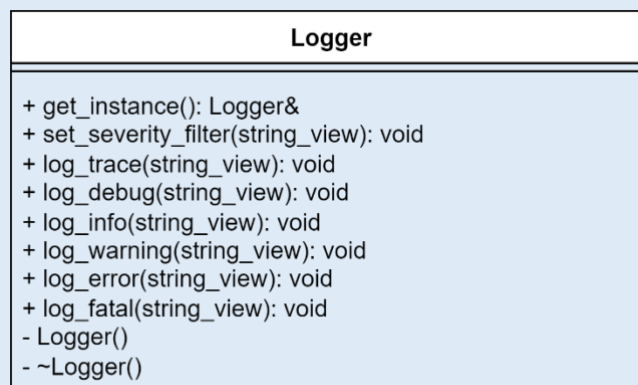




:Namespace Config



:Namespace Logging





14.11 תרשים מחלקות

ראו ערך בפרק 14 סעיף 10 (14.10).

14.12 תיאור המחלקות המוצעות

Namespace AudioRelay

מחלקת AudioListener

תפקיד המחלקה: המחלקה אחראית על הקמת סוקטים להאזנה לתעבורה ברשת, אחסונם בתוך באפר זמני והעברתם לתחילת העיבוד. המחלקה מהווה את שכבת הקלט של המערכת.

קלט המחלקה: אובייקט מסוג io_context לביצוע פעולות קלט/פלט אסינכרוניות.

פלט המחלקה: פקטות UDP המתקבלות מהסוקטים.

מחלקת AudioSender

תפקיד המחלקה: המחלקה אחראית על שליחה אסינכרונית ליעד ברשת של פקטות RTP/RTCP דרך הסוקטים. המחלקה מהווה את שכבת הפלט של המערכת.

קלט המחלקה: פקטות RTP/RTCP אותה רוצים להעביר ברשת, עם הכתובת המתאימה.

פלט המחלקה: אין.

מחלקת SessionManager

תפקיד המחלקה: המחלקה אחראית על ניהול של כל השיחות הפעילות במערכת. מבין תפקידיה המחלקה מבצעת את הפעולות הבאות:

- למפות נקודות קצה אל השיחות המתאימות.
- ליצור שיחות חדשות בהתאם לקונפיגורציה.
- לנתב פקטות RTP/RTCP לשיחות המתאימות במערכת.
- לשחרר מן המערכת שיחות לא פעילות.

קלט המחלקה: אובייקט המייצג כתובת רשת.

פלט המחלקה: אין.

מחלקת AudioSession

תפקיד המחלקה: מחלקה האחראית על ניהול שיחה אחת בין נקודות קצה. מחלקה זו מייצגת מופע של שיחת שמע.

קלט המחלקה: הקונפיגורציה של מקורות השמע, מזהה הייחודי של השיחה (מוקצה על ידי SessionManager) ואובייקט io_context לניהול אסינכרוני.

פלט המחלקה: אין.

מחלקת AudioSource

תפקיד המחלקה: מחלקה האחראית על עיבוד תקשורת שמע וניטור בזמן אמת המתקבל ממקור יחיד. תפקידיה של המחלקה כוללים:



- ניהול ותזמון של ה-jitter buffer.
- אגירת נתונים על תקשורת המקור לאורך זמן.

- שליחת הפקטה להמשך עיבוד ב-Translator.

קלט המחלקה: פקטות מסוג RTP/RTCP.

פלט המחלקה: פקטות מסוג RTP/RTCP להמשך עיבוד במערכת.

Namespace PacketHandler

מחלקת TimingHistory:

תפקיד המחלקה: מחלקה זו מהווה את רכיב ה-delay histogram של מנגנון ה-jitter buffer. מחלקה זו אחראית על שמירת ההיסטוריה של זמני ההגעה של הפקטות, בצורה שמאפשרת לבצע ניתוח סטטיסטי של עיכובים ולהסיק מתוכם את ההשקיה האופטימלית עבור עיבוד השמע.

קלט המחלקה: זמן ההגעה של הפקטה.

פלט המחלקה: אין.

מחלקת JitterBuffer:

תפקיד המחלקה: המחלקה אחראית על תיאום זמני של פקטות שמע נכנסות בתנאי רשת לא יציבים, תוך איזון בין עיכוב השליחה בין איכות השמע.

קלט המחלקה: פקטות שהתקבלו מהרשת, עם האובייקט המייצג אותה.

פלט המחלקה: מחזירה פקטת שמע מוכנה לעיבוד או יצירת פקטת שקט סינתטית במקרה של אובדן.

מחלקת PlcHandler:

תפקיד המחלקה: המחלקה אחראית על יצירת פקטות שקט סינתטיות לצורך הסתרת אובדן פקטות שמע.

קלט המחלקה: ערכי שדות ה-timestamp, sequence number של הפקטה, עם כמות הדגימות הרצויה.

פלט המחלקה: פקטת שקט סינתטית.

מחלקת RtpUtilities:

תפקיד המחלקה: המחלקה מהווה כמחלקת עזר לניתוח הלוגיקה של פקטות RTP על פי שדות ה-timestamp וה-sequence_number.

קלט המחלקה: אין.

פלט המחלקה: אין.



:Namespace PacketParser

:RtpPacket

תפקיד המחלקה: מחלקה זו אחראית על פענוח וניתוח של פקטות שמע מסוג RTP המתקבלות מהרשת. אובייקט זה נוצר עבור כל פקטה שמתקבלת בסוקט RTP over UDP ולה שני תפקידים עיקריים:

1. לוודא שמבנה הפקטה עומד בתאימות לתקן RFC3550.
2. לנתח את הפקטה על פי שדות ה-header.

קלט המחלקה: פקטת UDP המתקבלת מהרשת בצורה של מערך של בתיים.

פלט המחלקה: אובייקט אשר מהווה ממשק לשימוש עבור שאר המודולים במערכת לחילוץ מהיר של שדות הפקטה. אם הפקטה לא עמדה בתנאי התקן אז יוחזר אובייקט ריק.

:RtcpCompoundPacket

תפקיד המחלקה: המחלקה מנתחת פקטות מסוג Compound RTCP ומוודאת את תקינותם בהתאם ל-RFC3550. בהתאם לניתוח, היא מפרקת את הפקטה לקבוצה של פקטות כאשר כל אחת מייצגת סוג אחד של פקטת RTCP.

קלט המחלקה: פקטה שהתקבלה מהרשת בצורה של מערך בתיים.

פלט המחלקה: רשימה של אובייקטים של פקטות RTCP. במקרה שהניתוח נכשל תוחזר רשימה ריקה.

:RtcpPacket

תפקיד המחלקה: מחלקה האחראית על ניתוח של פקטת RTCP מסוג יחיד לפי תקן RFC3550.

פלט המחלקה: פקטה שהתקבלה מהרשת בצורה של מערך בתיים.

פלט המחלקה: אובייקט המייצג פקטת RTCP על פי ערכי ה-header.

:RtcpTypeBase

תפקיד המחלקה: מחלקה שנועדה לשמש כבסיס עבור מחלקות המייצגות סוגים שונים של פקטות RTCP.

קלט המחלקה: אין.

פלט המחלקה: אין.

:SenderReport

תפקיד המחלקה: המחלקה לנתח מידע מסוג SenderReport מתוך פקטות RTCP בסוג המתאים.

קלט המחלקה: מערך בתיים המייצג את המידע של סוג הפקטה.



פלט המחלקה: אובייקט המשמש לגישה ישירה לשדות המידע.

מחלקת SourceDescription:

תפקיד המחלקה: המחלקה לנתח מידע מסוג SourceDescription מתוך פקטות RTCP בסוג המתאים.

קלט המחלקה: מערך בתים המייצג את המידע של סוג הפקטה.

פלט המחלקה: אובייקט המשמש לגישה ישירה לשדות המידע.

מחלקת ReceptionReport:

תפקיד המחלקה: המחלקה לנתח מידע מסוג ReceptionReport מתוך פקטות RTCP בסוג המתאים.

קלט המחלקה: מערך בתים המייצג את המידע של סוג הפקטה.

פלט המחלקה: אובייקט המשמש לגישה ישירה לשדות המידע.

מחלקת Bye:

תפקיד המחלקה: המחלקה לנתח מידע מסוג Bye מתוך פקטות RTCP בסוג המתאים.

קלט המחלקה: מערך בתים המייצג את המידע של סוג הפקטה.

פלט המחלקה: אובייקט המשמש לגישה ישירה לשדות המידע.

מחלקת App:

תפקיד המחלקה: המחלקה לנתח מידע מסוג App מתוך פקטות RTCP בסוג המתאים.

קלט המחלקה: מערך בתים המייצג את המידע של סוג הפקטה.

פלט המחלקה: אובייקט המשמש לגישה ישירה לשדות המידע.

Namespace FrontendComm

מחלקת JsonMessageBuilder:

תפקיד המחלקה: מחלקה שתפקידה ליצור הודעות JSON לביצוע תקשורת עם ממשק המשתמש.

קלט המחלקה: אין.

פלט המחלקה: אובייקטים מסוג JSON המייצגים הודעות לממשק משתמש.

מחלקת FrontendBridge:

תפקיד המחלקה: ניהול תעבורת WebSocket אל מול ממשק המשתמש. המחלקה בין היתר מבצעת את תחומי האחריות הבאים:

- הקמת החיבור עם הממשק משתמש.



- קריאת הודעות סטטוס מהממשק משתמש.
- שליחת נתונים אל הממשק משתמש.

קלט המחלקה: אין.

פלט המחלקה: פקטה בפורמט JSON המכילה עדכון אל הממשק משתמש.

Namespace AudioTranscoding

מחלקת Transcoder

תפקיד המחלקה: המחלקה מהווה template שמטרתה להגדיר ממשק אחיד לקידוד ופענוח שמע עבור קודקים שונים. זהו מחלקת template ממנה עושים ירושה באמצעות ה- design pattern CRTP (Curiously recurring template pattern).

קלט המחלקה: אין.

הפלט של המחלקה: אין.

מחלקת G711Transcoder

תפקיד המחלקה: מחלקה זו אחראית על מימוש תהליכי הקידוד והפענוח של השמע על פי הקודק G711-Alaw. אובייקט זה נוצר עבור כל מקור שבקונפיגורציה מוגדר לו הקודק G711-Alaw

קלט המחלקה: כאשר המחלקה מבצעת קידוד היא מקבלת כקלט זרם שמע בפורמט PCM, וכאשר המחלקה מבצעת פענוח היא מקבלת כקלט זרם שמע מקודד בקודק G711-Alaw.

פלט המחלקה: הפלט בסיום תהליך הקידוד הוא זרם מידע מקודד בקודק G711-Alaw, והפלט בסיום תהליך הפענוח הוא זרם מידע בפורמט PCM.

מחלקת SpeexTranscoder

תפקיד המחלקה: מחלקה זו אחראית על מימוש תהליכי הקידוד והפענוח של השמע על פי הקודק Speex. אובייקט זה נוצר עבור כל מקור שבקונפיגורציה מוגדר לו הקודק Speex.

קלט המחלקה: כאשר המחלקה מבצעת קידוד היא מקבלת כקלט זרם שמע בפורמט PCM, וכאשר המחלקה מבצעת פענוח היא מקבלת כקלט זרם שמע מקודד בקודק Speex.

פלט המחלקה: הפלט בסיום תהליך הקידוד הוא זרם מידע מקודד בקודק Speex, והפלט בסיום תהליך הפענוח הוא זרם מידע בפורמט PCM.

מחלקת Translator

תפקיד המחלקה: המחלקה אחראית על ביצוע המרה בזמן אמת בין שני קודקי שמע שונים בשיחה אחת. בנוסף היא אחראית על עדכון ה- header של פקטות RTCP לצורך סנכרון עם פקטות RTP.

קלט המחלקה: פקטה שמקודדת בקודק המקור.

פלט המחלקה: פקטה שקודדה מחדש בקודק היעד.



:Namespace Config

:RulesManager מחלקת

תפקיד המחלקה: המחלקה אחראית על ניהול קובץ הקונפיגורציה של המערכת ועל חילוף המידע ממנו עבור כלל המודולים במערכת.

קלט המחלקה: קובץ סטטי מסוג ini המייצג את תצורת המערכת.

פלט המחלקה: מבנה נתונים מסוג property_tree המכיל את נתוני תצורת המערכת בצורה של עץ.

:RulesValidator מחלקת

תפקיד המחלקה: המחלקה אחראית על בדיקת תקינותו של קובץ הקונפיגורציה לפני הפעלתה. המחלקה משמשת כשכבת הגנה למערכת מפני נתונים שגויים העלול לגרום להתנהגות לא צפויה או קריסות מצד המערכת.

קלט המחלקה: צמתים מתוך מבנה הנתונים property_tree.

פלט המחלקה: אין.

:Namespace Logging

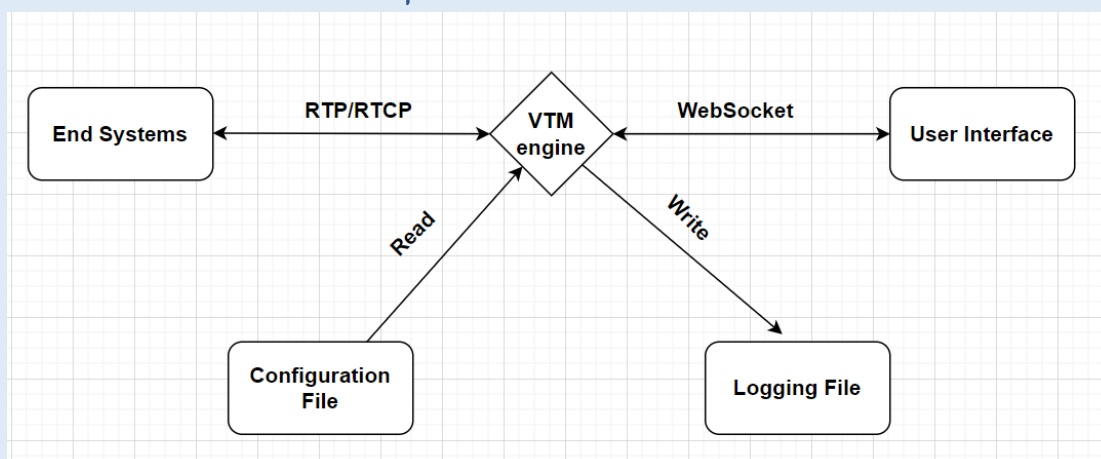
תפקיד המחלקה: המחלקה אחראית על רישום הדעות הלוגים מכלל המערכת על פי רמות ההודעה.

קלט המחלקה: מחרוזת המייצגת הודעת לוג.

פלט המחלקה: רישום של ההודעה אל קובץ לוגים ייעודי והדפסה אל ה-console.



15. רכיבי ממשק



End systems: אלו הם התקנים חיצוניים (למשל מערכות טלפוניה או מכשירי VoIP) אשר שולחים ומקבלים פקטות RTP ו-RTCP דרך רשתות IP. המערכת מאזינה לפקטות המגיעות מן ה-end systems ומשדרת ליעדן את זרמי השמע בהתאם לקונפיגורציה. חשוב לציין, כי למערכת אין שליטה ישירה על אותן המקורות, אלא היא רק מגשרת ומעבדת את הפקטות.

קובץ קונפיגורציה: קובץ חיצוני המכיל את כל פרטי התצורה של המערכת. מכילה בתוכה מידע עבור תצורת השיחות, הגדרות הלוגים, וההגדרת התקשורת עם הממשק משתמש. המערכת קוראת את הקובץ פעם אחת באתחולה באמצעות מבנה הנתונים property tree ונטענת ושולפת ממנו מידע בזמן ריצתה.

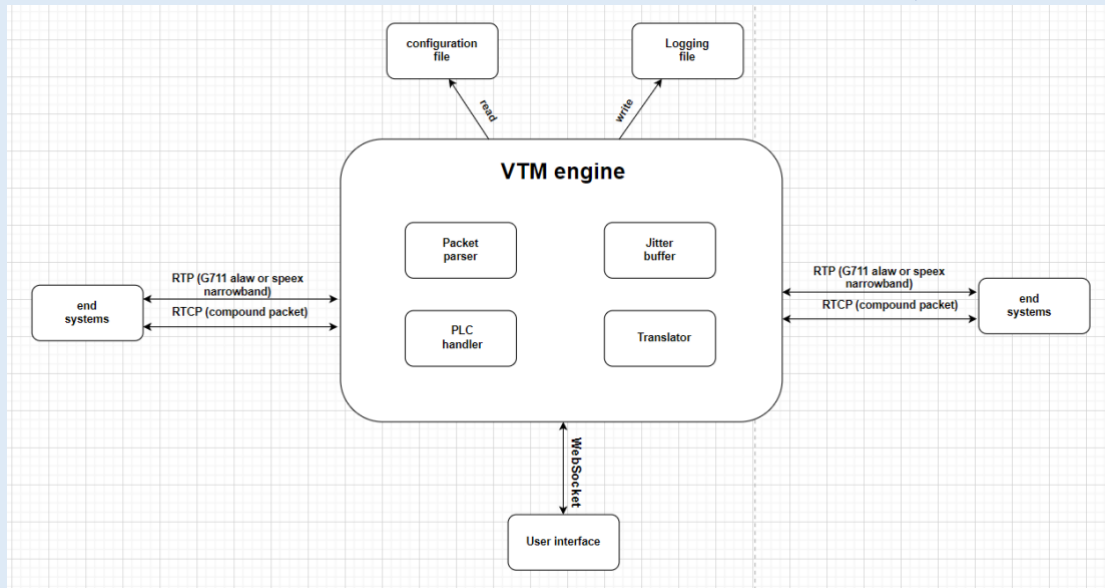
ממשק משתמש: ממשק ידידותי וויזואלי שמציג בזמן אמת את מצב השיחות, מידע על כל מקור ומדדי QoS. המערכת שולחת עדכונים בפורמט JSON דרך WebSocket ל-UI. ה-backend מעדכן את ה-frontend בזמן שה-frontend שולח אל ה-backend סטטוס קבלה.

קובץ לוגים: קובץ שמטרתו הוא לתעד את פעילויות המערכת על פי רמות סיווג שונות, לצורך ניתוח עתידי ותחקור תקלות. בזמן ריצה, כל אירוע משמעותי נכתב לקובץ לוג טקסטואלי באמצעות הספרייה boost log.



16. תיכון המערכת

16.1 ארכיטקטורת המערכת



16.2 תיכון מפורט

התיכון מערכת VTM מבוסס על עקרונות של ארכיטקטורה מודולרית וביצוע הפרדה ברורה בין שכבות התקשורת לבין עיבוד השמע.

בתיכון המערכת, שכבות הקלט והפלט של המערכת פועלות אל מול נקודות קצה חיצוניות ומבצעת קליטה ושליחה של פקטות באופן אסינכרוני של פקטות שמע על גבי UDP. רכיבי Packet Parser ו-Translator פועלים באופן סימטרי לשני כיווני השיחה כאשר בכל צד מתבצעת קליטה של פקטות, ניתוח, עיבוד, ולעיתים המרה של קידוד הפקטה בהתאם לקונפיגורציה לפני שליחתה ליעדה.

אחד המרכיבים הקריטיים הוא ה-jitter buffer אשר משמש כשכבת ביניים בין קליטת הפקטות לבין שליחתן לעיבודן. באמצעותו נעשה סנכרון של שליחת הנתונים גם תחת תנאים לא יציבים של הרשת. בתיאום, פועל לצידו רכיב ה-PLC Handler שמבצע גישור לאובדן פקטות במידת הצורך.

התיכון שם דגש על ניטור ובקרה לאורך זמן באמצעות קובץ לוגים. כל פעולה משמעותית, שגיאה או חריגה נרשמת לתוך logging file. זאת בנוסף לקובץ קונפיגורציה סטטי אשר נטען בתחילת הריצה ומשמש כתשתית להקמת רכיבי המערכת והתצורה שלה.

בנוסף, קיים ממשק משתמש הפועל כתהליך עצמאי ממנוע העיבוד, ומתקשר עם מנוע העיבוד באמצעות תקשורת WebSocket לצורך עדכון התצוגה למשתמש בזמן אמת.

עקרונות התיכון של המערכת כוללים הקפדה על תהליכון ייעודי לתקשורת עם ממשק המשתמש, ותהליכון לניהול התקשורת עם נקודות הקצה, תוך שימוש בשני תהליכונים במנגנון Asynchronous I/O בכדי לאפשר עבודה לא חוסמת, וניהול שיחות וחיבורים רבים במקביל.



16.3 חלופות לתיכון המערכת

חלופה אפשרית אשר היה ניתן לממש במערכת היא שימוש בריבוי תהליכונים (multithreading) לניהול תקשורת הרשת. בגישה זו עבור כל שיחה פעילה על גבי המערכת, היה מוקצה לה תהליכון ייעודי אשר היה אחראי על עיבוד וניתוב זרמי המידע של השיחה.

יתרונות:

- ✓ כל שיחה נפרדת לחלוטין, לכן קל לבודד ולנתח אותה.
- ✓ תקלה או עומס המתרחשים בשיחה אחת אינו משפיע על השיחות האחרות.
- ✓ מאפשר ניצול מירבי של ליבות המעבד – ביצועים טובים תחת עומס נמוך.

חסרונות:

- ☒ תכנון שהוא לא scalable, עלול להאט את פעילות המערכת כשיש עשרות/מאות שיחות.
- ☒ הקצאת תהליכונים רבים צורכת הרבה זיכרון וניהול מעבד.
- ☒ דורש תכנון זהיר של המערכת – יצירת מנגנוני תיאום, ונעילה במיוחד עבור משאבים משותפים וקטעים קריטיים במערכת.



17. תיאור התוכנה

17.1 סביבת עבודה

Cmake – cmake הוא מחולל מערכת בנייה בקוד פתוח המפשט את תהליך ניהול תצורת הבנייה של פרויקטי תוכנה על פני פלטפורמות ומערכות שונים. Cmake תומך במבני פרויקטים מודולריים, פקודות בנייה מותאמות אישית ותאימות בין פלטפורמות, מה שהופך אותה לבחירה פופולרית עבור יישומי ++C קטנים וגדולים כאחד.

Microsoft Visual Studio: סביבת פיתוח מתקדמת לכתיבת קוד בשפת ++C ושפות נוספות, עם תמיכה מובנית בכלי Build כמו Cmake. בנוסף היא בעלת תמיכה ביכולות debug מתקדמות, וניהול פרויקטים מורכבים.

PyCharm: סביבת פיתוח לכתיבת קוד בשפת Python. כוללת Terminal פנימי, ניהול חבילות, כלי debug מתקדמים ועורך קוד חכם.

FFmpeg: חבילת תוכנה בקוד פתוח לעיבוד מדיה דיגיטלית. היא תומכת בקידוד, פענוח, המרה, streaming וניהול של קבצי וידאו ושמע בפורמטים רבים. מאפשרת עבודה גמישה עם פרוטוקולי מדיה מה שהופך אותו לכלי עזר חשוב בבדיקת שליחת וקבלת פקטות שמע בפרויקט.

GStreamer: פלטפורמה מודולרית, בקוד פתוח, לעיבוד מדיה בזמן אמת. המערכת בנויה על בסיס pipelines של רכיבי עיבוד הניתנים להרכבה. תומכת בפרוטוקולי מדיה בצורה מובנית ומאפשר בניית תוכנות מורכבות לשידור, קליטה, המרה וסנכרון של זרמי מדיה – מה שהופך אותה לבחירה אידיאלית עבור בדיקות של מערכות VoIP.

17.2 שפות תכנות

C++: שפת תכנות בסגנון high-level אשר מהווה superset לשפת C. השפה תומכת בתכנות מסוג מונחה עצמים וכמו כן גם בתכנות פרוצדורלי וגנרי. שפה זו היא שפה רבת עוצמה מאפשרת למפתחים לכתוב קוד עבור יישומים גדולים ופיתוח תוכנה בעלי ביצועים גבוהים.

יתרונות השימוש בשפת ++C להכנת הפרויקט:

1. למערכת זו, חשוב שיהיו ביצועים כמה שיותר מהירים ויעילים בשל עצם העובדה שבין דרישות המערכת היא חייבת לעבד שיחות בזמן אמת. שפת תכנות זו היא שפה מאוד חזקה הידועה במהירותה ויעילותה היחסית בביצועיה ובעבודתה הקרובה לחומרה, ולכן היא עונה על דרישה זו.
2. שפת תכנות זו תומכת בתכנות מסוג מונחה עצמים מה שמאפשר את חלוקת הפרויקט למחלקות. תכונה זו מאפשרת ביצוע של ניהול ותחזוקת הפרויקט בצורה טובה יותר.
3. לשפה זו יש מגוון רחב של ספריות וכלים זמינים.

Python: python היא שפת תכנות בסגנון high-level נפוצה ורב-תכליתית, המשמשת לפיתוח יישומים במגוון תחומים – כולל פיתוח אתרים, ניתוח נתונים, בינה מלאכותית, אוטומציה ומחשוב מדעי. השפה תומכת בפרדיגמות תכנות מגוונות, לרבות תכנות מונחה עצמים, תכנות פרוצדורלי ופונקציונלי, וכן כוללת מערכת טיפוס דינאמית וניהול זיכרון אוטומטי, מה שמפשט את תהליך הפיתוח ומפחית את הסיכון לשגיאות.

יתרונות השימוש בשפת python להכנת הפרויקט:



1. אחד המאפיינים הבולטים של השפה היא התחביר הפשוט והקריא שלה, מאפיין המקל על פיתוח מהיר, תיעוד והבנה של הקוד.
2. שפה זו המחזיקה בספריות ממשק משתמש אינטראקטיביות הכוללות תצוגות טבלאיות, גרפים ועדכונים בזמן אמת.
3. שפה זו מנהלת את הזיכרון באופן אוטומטי, מה שמפחית את הסיכון לדליפות זיכרון או קריסות – חשוב במיוחד בממשקים שמריצים תהליכים לאורך זמן.



18. תיאור מסכים

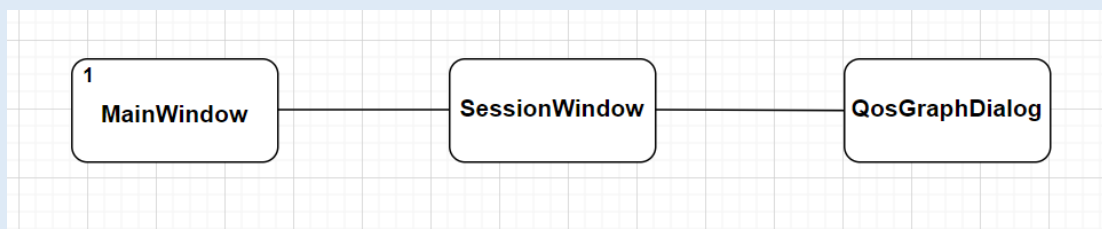
המסך הראשי (MainWindow): זהו המסך הראשון שנפתח עם הפעלת התוכנה, בו מוצג רשימת השיחות הפעילות/הלא פעילות. דרכו ניתן לעקוב אחרי סטטוס החיבור ל-backend, ולבחור שיחה אותה המשתמש רוצה לראות.

מסך השיחה (SessionWindow): מציג את פרטי השיחה שנבחרה, והסטטוס שלה (פעילה/לא פעילה). כולל מידע על מקורות השמע כגון חיווי עליהם (מדבר/לא מדבר), קודק שנומצא בשימוש ונתוני QoS (packet loss, jitter) עדכניים עבור כל מקור.

מסך גרפי QoS (QosGraphDialog): חלון נפרד המציג גרפים של נתוני QoS של מקור השמע לאורך זמן.



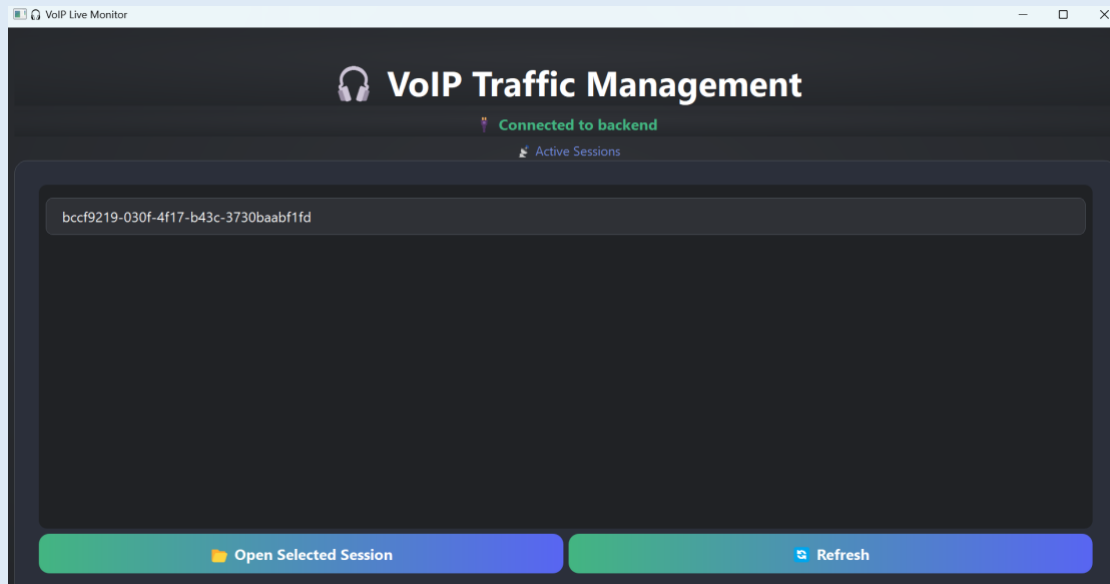
19. תרשים מסכים המתאר את היררכיית המסכים (Screen flow diagram) והמעברים ביניהם





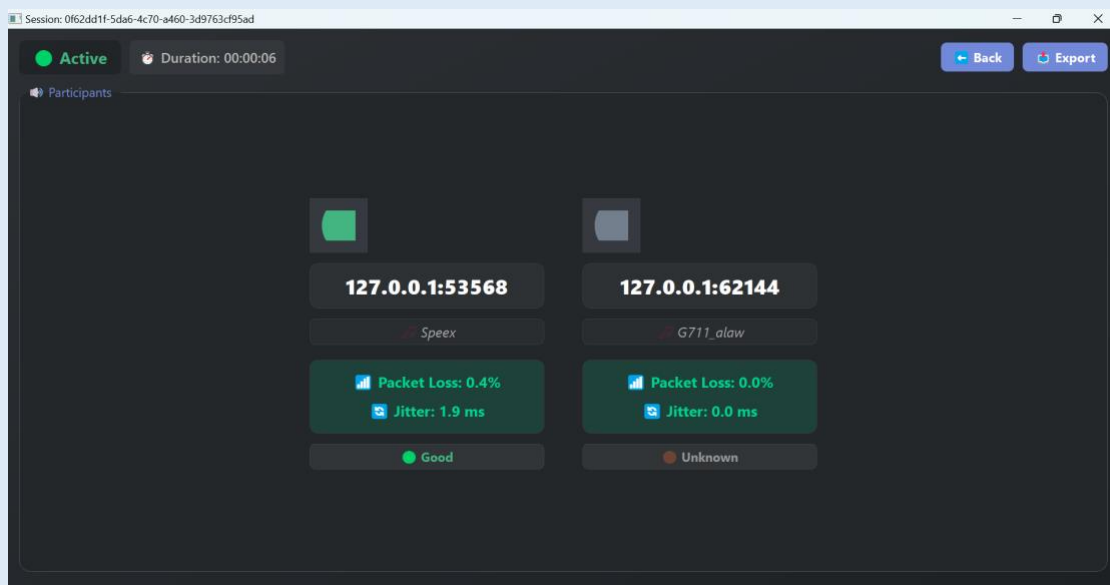
20. מה תפקידו של כל מסך / חלון עם צילום מסך של החלון הרלוונטי

המסך הראשי:



במסך זה מתבצעת הצפייה ברשימת השיחות הפעילות, סטטוס החיבור ל-backend, ובחירה של שיחה לצפייה בתוך הרשימה של השיחות. המשתמש יכול לבחור שיחה מהרשימה ולפתוח אותה בחלון חדש או לבצע ריענון ולמחוק שיחות לא פעילות מהמסך.

מסך השיחה:

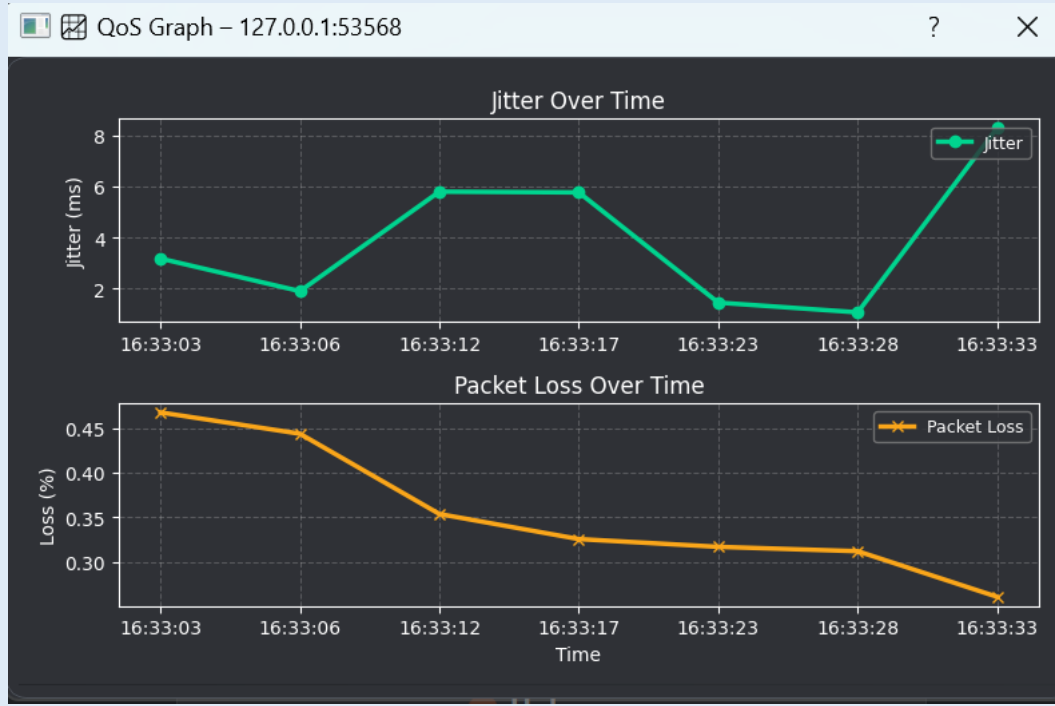


מטרתו של מסך זה הוא להציג בזמן אמת את פרטי השיחה שנבחרה, לרבות רשימת המקורות בה, הקודק של כל מקור שמע והמדדים של איכות החיבור. כל מקור מוצג בצורה של כרטיס/קלף הכולל נתונים עליו. בנוסף, מוצג חיווי ויזואלי עבור כל משתתף המעדכן את מצבו – אם הוא מדבר, לא מדבר או מתנתק מהשיחה. בחלק העליון של המסך מוצג



הסטטוס של השיחה (פעיל/לא פעיל), זמן הכולל של השיחה וכפתור ייבוא של תצוגת השיחה בפורמט JSON.

מסך גרפי QoS:



חלון גרפי ייעודי שנפתח מתי שלוחצים על ה-icon של מקור השמע בשיחה. מטרתו היא זה להציג באופן גרפי את השינויים בערכים של מדדי ה-QoS לאורך זמן. תצוגה זו מסייעת למשתמש להבין מגמות בתקלות רשת או ירידה באיכות השיחה.



21. תיאור מסך הפתיחה – מה הוא מכיל והאם משמש

נקודת ניווט

מסך הפתיחה הוא המסך הראשון שנפתח עם הפעלת התוכנה, והוא משמש כנקודת ניווט עבור המשתמש במהלך פעילות התוכנה.

בחלק העליון של המסך מופיע שם המערכת, וסטטוס חיבור ל-backend, חיווי זה מתעדכן בזמן אמת בהתאם למצב החיבור עם ה-backend.

תחת הכותרת ממוקמת רשימת השיחות במערכת, המוצג בפורמט של List widget ממנו ניתן לראות אילו שיחות מתקיימות במערכת.

המסך כולל 2 כפתורים עיקריים - כפתור פתיחת שיחה המאפשר למשתמש לצפות בפרטי שיחה מסוימת, וכפתור ריענון המעדכן את רשימת השיחות ומסיר שיחות שנסגרו. שיחות שהסתיימו מוצגות על המסך הראשי כלא פעילות כדי לספק חיווי מידי למשתמש.



22. כל מסכי האפליקציה / אתר / מערכת מנהלית, בליווי הסברים

המסך הראשי:

מסך הפתיחה של הממשק משתמש. המשתמש נחשף אליו עם פתיחת המערכת והוא משמש כנקודת הכניסה לממשק משתמש. הוא כולל רשימת שיחות פעילות, מידע על מצב החיבור ל-backend, וכפתורים לפתיחה וריענון של השיחות.

מסך השיחה:

מסך זה מציג את פרטי השיחה שנבחרה על ידי המשתמש מהמסך הראשי. במסך זה ניתן לצפות בכל מקורות השמע שמשתתפים בשיחה עם חיווי ויזואלי של המקור ונתונים חשובים על המקור. בנוסף, מוצג חיווי על משך השיחה, שמתעדכן בזמן אמת כל עוד השיחה פעילה.

כרטיס משתמש:

כרטיס זה מוצג בתוך מסך השיחה עבור כל מקור שמע בשיחה. כל כרטיס הוא תצוגה של נתוני המקור עם עיצוב המשלב icon של המקור, כתובת, קודק ומדדי QoS עדכניים עם חיווי כללי על מצב החיבור.

גרפי QoS:

זהו מסך נפרד שנפתח כאשר המשתמש לוחץ על icon של אחד מהמקורות. במסך זה מוצגים גרפים אינטראקטיביים של מדדי ה-QoS לאורך זמן במטרה לספק כלי ניתוח חזותי עבור איכות התקשורת.



23. עבור כל אלמנט תצוגה כדוגמת: כפתור, תיבת טקסט יש להסביר את תפקידם

המסך הראשי:

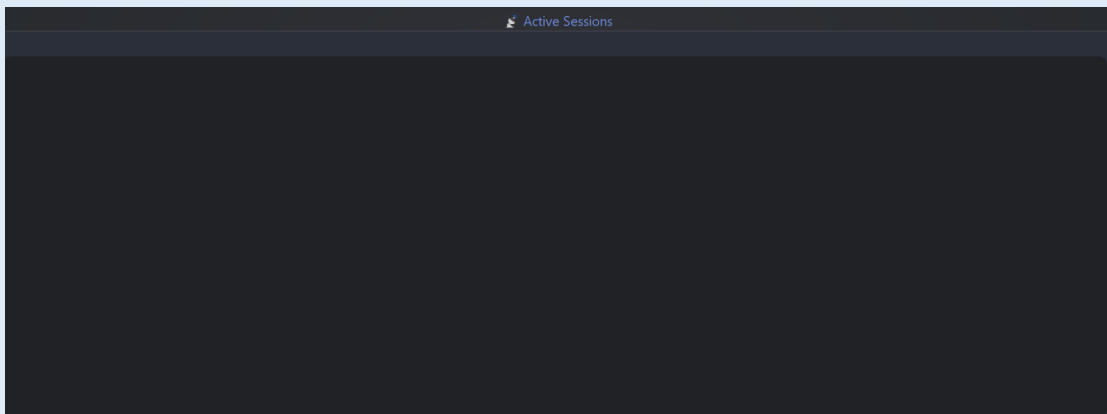
כפתור Open Selected Session - כפתור שלחיצה עליו פותח את חלון השיחה של השיחה שנבחרה מתוך הרשימה.

 Open Selected Session

כפתור Refresh – כפתור המאפשר לרענן את רשימת השיחות במסך הראשי, למשל בשביל להוריד מתצוגת המשתמש שיחות לא פעילות.

 Refresh

רשימת השיחות – אלמנט תצוגה אשר מציג את כל השיחות על גבי המערכת. כל שורה בה מייצגת שיחה ובה כתוב המזהה הייחודי שלה, ואם שיחה אינה פעילה היא תסומן עם תווית של inactive. לחיצה על שורה מתוך הרשימה פותחת את חלון השיחה שלה.



תווית סטטוס חיבור ל-backend - תווית זו מעדכנת את המשתמש בזמן אמת אם ה-frontend מחובר ל-backend.



VoIP Traffic Management



Connected to backend



חלון השיחה:

תווית סטטוס השיחה – תווית זו מעדכנת את המשתמש האם השיחה בה הוא צופה פעילה או לא.

 **Active**

תווית Duration – תווית המשמשת לחיווי זמן, כלומר כמות הזמן שעברה מתחילת השיחה.

 **Duration: 00:00:33**

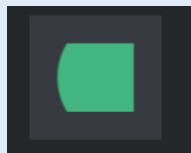
הכפתור back – כפתור שלחיצה עליו מחזירה את המשתמש ממסך השיחה, חזרה אל המסך הראשי.

 **Back**

הכפתור export – כפתור שלחיצה עליו מורידה למחשב של המשתמש קובץ JSON המכיל נתוני זמן אמת על מצב השיחה.

 **Export**

Icon – מראה את סטטוס החיווי של מקור השמע (מדבר/לא מדבר). לחיצה עליו פותח את גרפי ה-QoS של אותו מקור.



תווית IP:PORT – תווית בה רשום הכתובת של מקור השמע.

127.0.0.1:62144

תווית Codec – תווית בה רשום הקודק של מקור השמע.

 **G711_alaw**

QoS Box – תת אלמנט המציג את נתוני ה-QoS הנוכחיים.

 **Packet Loss: 0.3%**

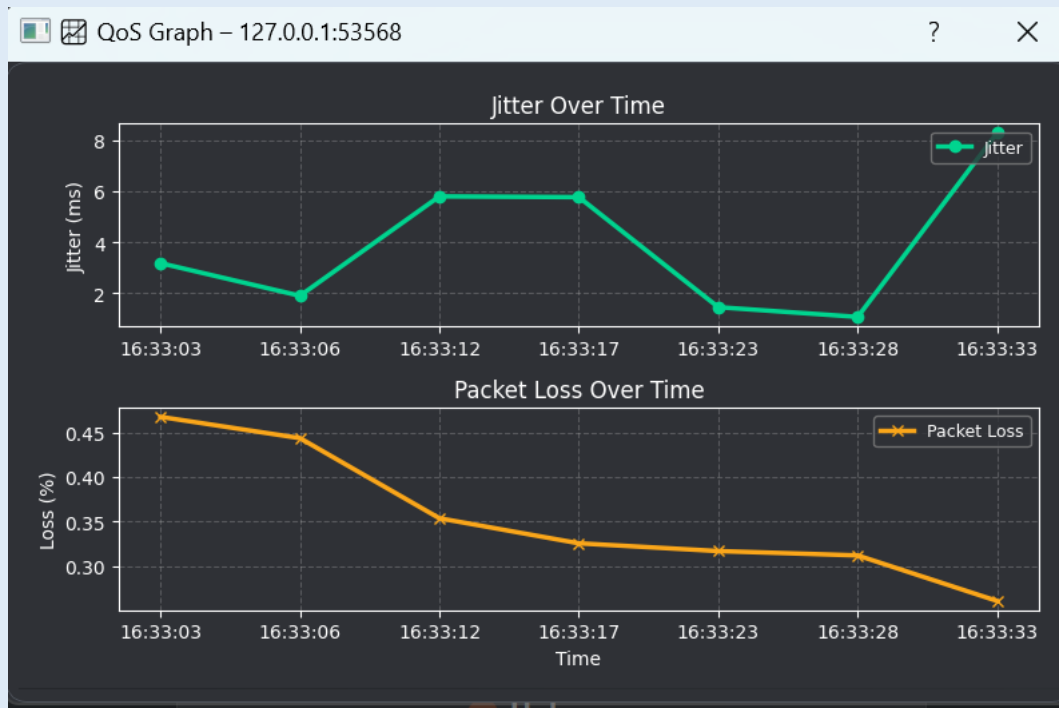
 **Jitter: 9.3 ms**

Quality Indicator – תווית צבעונית המתעדכנת לפי איכות מדדי ה-QoS של מקור השמע.

 **Good**



גרפי QoS:



אלמנט גרפי המציג בזמן אמת את היסטוריית איכות התקשורת של מקור השמע. אלמנט זה מורכב מתת האלמנטים הבאים:

- Axes - צירי זמן וערכים.
- Legend - סימני זיהוי של כל ערוץ מידע.
- Grid - עוזר בקריאת הנתונים.



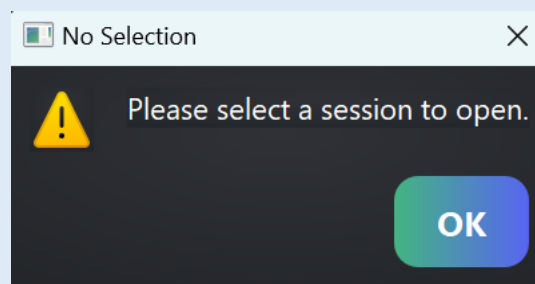
24. הודעות למשתמש (alert) למיניהם

שגיאה בהתחברות ל-backend:

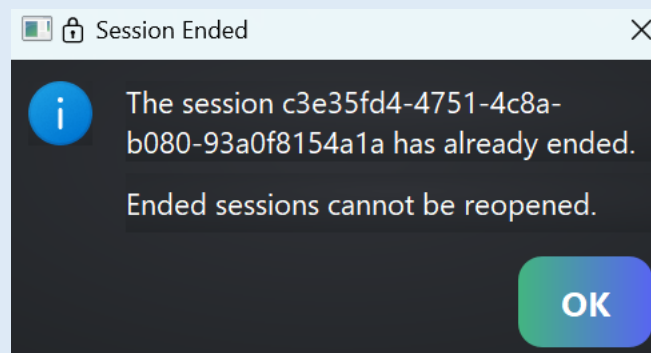


תווית סטטוס, שמטרתה להתריע בפני המשתמש כי ה-frontend לא מחובר ל-backend ולא מתבצעת תקשורת ביניהם.

שגיאה בפתיחת שיחה:



חלופית שגיאה המופיעה אם המשתמש ניסה ללחוץ על כפתור "open selected session" מבלי שלחץ על שיחה לצפייה לפני כן.



חלופית שגיאה המופיעה אם המשתמש ניסה להיכנס לשיחה שמסומנת כלא פעילה.



25. ממשק משתמש

ממשק המשתמש של המערכת VTM תוכנן במטרה לספק חווית שימוש ידידותית ואינטואיטיבית למשתמש לצרכי ניטור בזמן אמת של שיחות VoIP.

המסך הראשי של ממשק המשתמש מציג למשתמש את רשימת שיחות המתרחשות במערכת, שכל שיחה מופיעה עם המזהה הייחודי שלה. לצד כל שיחה מוצג הסטטוס שלה כאשר אם שיחה לא פעילה היא תסומן עבור המשתמש כ-inactive. רשימה זו מאפשרת למשתמש לבחור שיחה לצפייה ולניטור על ידי לחיצת כפתור.

בעת פתיחת השיחה, מוצג חלון ובו מופיעים כרטיסי מקורות השמע שכל אחד מייצג end system המשתתף בשיחה. כל כרטיס כולל מידע על מקור השמע כגון הכתובת שלו, הקודק שבשימוש, נתוני QoS עדכניים, חיווי חזותי על מקור השמע (מדבר/לא מדבר) וחיווי על מצב החיבור שלו (Good/Fair/Poor/Unknown).

לחיצה על ה-icon של מקור השמע פותח חלון נוסף המציג גרפי QoS של אותו מקור שמע – זהו כלי ויזואלי שנותן תמונת מצב על איכות הרשת של מקור השמע לאורך זמן.



26. קוד התוכנית

```
bool RtpPacket::parse_version_and_type(const std::vector<uint8_t>& packet)
{
    if (static_cast<int>(m_header.version) != RTP_VERSION)
    {
        LOG_ERROR("RTP version mismatch. Expected=" + std::to_string(RTP_VERSION) + " Got=" + std::to_string(m_header.version));
        return false;
    }

    if (static_cast<int>(packet[1]) == RtcpPacket::SR || static_cast<int>(packet[1]) == RtcpPacket::RR)
    {
        LOG_ERROR("RTCP packet detected instead of RTP. Type=" + std::to_string(packet[1]));
        return false;
    }

    m_metadata.payload_offset = RTP_BASIC_HEADER_SIZE;
    return true;
}
```

קלט: מערך המכיל את הפקטה כפי שהתקבלה.

פלט: אמת אם הפקטה עומדת בתנאים, אחרת שקר

תיאור האלגוריתם: פונקציה זו אחראית לאימות ראשוני של פקטת RTP. היא בודקת:

1. האם הפקטה משתמשת בגרסה התקנית של RTP (שהיא 2).
2. האם בטעות התקבלה פקטת RTCP על ידי בדיקת הבית הראשון בפקטה.

```
bool RtpPacket::parse_csrc_list(const std::vector<uint8_t>& packet)
{
    if (m_header.csrc_count == 0)
        return true;

    m_metadata.csrc_length = static_cast<size_t>(m_header.csrc_count * sizeof(uint32_t));
    if (packet.size() < RTP_BASIC_HEADER_SIZE + m_metadata.csrc_length)
    {
        LOG_ERROR("Packet too short for CSRC list.");
        return false;
    }

    m_header.csrc.assign(packet.begin() + RTP_BASIC_HEADER_SIZE,
        packet.begin() + RTP_BASIC_HEADER_SIZE + m_metadata.csrc_length);
    m_metadata.payload_offset += m_metadata.csrc_length;

    LOG_TRACE("Parsed CSRC list. Count=" + std::to_string(m_header.csrc_count));
    return true;
}
```

קלט: מערך המכיל את הפקטה כפי שהתקבלה.

פלט: אמת אם אין CSRC list או שהפקטה עומדת בתנאים והצלחנו לחלץ את השדה CSRC list, אחרת שקר.

תיאור האלגוריתם: פונקציה זו מבצעת ניתוח של רשימת מזהי מקורות תורמים (CSRC) בפקטת RTP. שדה זה מאפשר לציין מקורות שמע נוספים שהשתתפו ביצירת התוכן של הפקטה. מטרת הפונקציה היא לחלץ את רשימת CSRC אם קיימת ולוודא שהפקטה כוללת את כל הנתונים הדרושים לכך.



```
bool RtpPacket::parse_padding(const std::vector<uint8_t>& packet)
{
    if (!m_header.padding)
        return true;

    m_metadata.numpadbytes = packet.back();

    if (m_metadata.numpadbytes == 0 || m_metadata.numpadbytes > packet.size() - m_metadata.payload_offset)
    {
        LOG_ERROR("Invalid RTP padding length: " + std::to_string(m_metadata.numpadbytes));
        return false;
    }

    LOG_TRACE("Padding bytes: " + std::to_string(m_metadata.numpadbytes));
    return true;
}
```

קלט: מערך המכיל את הפקטה כפי שהתקבלה.

פלט: אמת אם אין padding או שהוא זוהה וחולץ בהצלחה, אחרת שקר.

תיאור האלגוריתם: פונקציה זו מנתחת את שדה ה-padding בפקטת RTP. ב-RTP ניתן להוסיף בתים נוספים לסוף הפקטה לצרכים שונים כאשר הבית האחרון ב-padding bytes מסמל כמה בתים נוספו לפקטה. מטרת הפונקציה היא לבדוק האם הפקטה מכילה padding ואם כן לחלץ את אורכו ולבדוק את תקינותו.

```
bool RtpPacket::parse_extension(const std::vector<uint8_t>& packet)
{
    if (!m_header.extension)
        return true;

    if (packet.size() < m_metadata.payload_offset + RTP_BASIC_EXTENSION_HEADER_SIZE)
    {
        LOG_ERROR("Insufficient packet size for RTP extension header.");
        return false;
    }

    m_extensionHeader.profile_defined = (packet[m_metadata.payload_offset] << 8) | packet[m_metadata.payload_offset + 1];
    m_extensionHeader.length = (packet[m_metadata.payload_offset + 2] << 8) | packet[m_metadata.payload_offset + 3];
    m_metadata.payload_offset += RTP_BASIC_EXTENSION_HEADER_SIZE;

    m_metadata.extension_data_length = m_extensionHeader.length * sizeof(uint32_t);
    if (packet.size() < m_metadata.payload_offset + m_metadata.extension_data_length)
    {
        LOG_ERROR("Insufficient packet size for RTP extension data.");
        return false;
    }

    m_extensionHeader.extension_data.assign(packet.begin() + m_metadata.payload_offset,
        packet.begin() + m_metadata.payload_offset + m_metadata.extension_data_length);
    m_metadata.payload_offset += m_metadata.extension_data_length;

    LOG_TRACE("Parsed RTP extension. Profile=" + std::to_string(m_extensionHeader.profile_defined) +
        " Length=" + std::to_string(m_extensionHeader.length));
    return true;
}
```

קלט: מערך המכיל את הפקטה כפי שהתקבלה.

פלט: אם לא נמצא בפקטה extension header או שנמצא וחולץ בהצלחה, אחרת שקר.

תיאור האלגוריתם: פונקציה זו מטפלת בשדה ה-extension בתוך פרוטוקול RTP. שדה זה מאפשר למערכות להוסיף מידע נוסף לפקטה מבלי לשבור תאימות לפרוטוקול. מטרת הפונקציה היא לזהות על פי ה-header הבסיסי אם קיים extension header ואם יש לנתח אותו על פי השדות הסטנדרטים בו.



```
bool RtpPacket::parse_payload(const std::vector<uint8_t>& packet)
{
    LOG_TRACE("Packet size: " + std::to_string(packet.size()) +
              ", Payload offset: " + std::to_string(m_metadata.payload_offset) +
              ", Padding: " + std::to_string(m_metadata.numpadbytes));

    if (packet.size() < m_metadata.payload_offset + m_metadata.numpadbytes)
    {
        LOG_ERROR("Invalid RTP payload bounds.");
        return false;
    }

    m_payload.assign(packet.begin() + m_metadata.payload_offset, packet.end() - m_metadata.numpadbytes);
    LOG_TRACE("Payload parsed. Size: " + std::to_string(m_payload.size()) + " bytes.");
    return true;
}
```

קלט: המערך המכיל את הפקטה כפי שהתקבלה.

פלט: אמת אם ה-payload חולץ בהצלחה, אחרת שקר.

תיאור האלגוריתם: פונקציה זו היא שלב הסיום של ניתוח פקטת RTP – חילוץ תוכן השמע עצמו ה-payload. עושים את חילוץ ה-payload ע פי המרחק מתחילת הפקטה שממנו ה-payload מתחיל עד לבתים בפקטה שמייצגים את ה-padding bytes.



```
bool Rtcp::check_packet_structure(const std::vector<uint8_t>& packet)
{
    if (packet.size() < RtcpPacket::RTCP_COMMON_HEADER_SIZE)
    {
        return false;
    }

    uint8_t first_packet_padding = (packet[0] >> 5) & 0x01;
    uint8_t first_packet_type = packet[1];

    if (first_packet_padding || (first_packet_type != RtcpPacket::SR && first_packet_type != RtcpPacket::RR))
    {
        return false;
    }

    size_t offset = 0;
    int packet_count = 0;

    do
    {
        if (packet.size() - offset < RtcpPacket::RTCP_COMMON_HEADER_SIZE)
        {
            return false;
        }

        uint8_t packet_version = (packet[offset] >> 6) & 0x03;
        uint8_t packet_type = packet[offset + 1];
        uint16_t length_field = (packet[offset + 2] << 8) | packet[offset + 3];
        size_t packet_length = (static_cast<size_t>(length_field + 1)) * sizeof(uint32_t);

        if (packet_version != RtcpPacket::RTP_VERSION ||
            (packet.size() - offset < packet_length) ||
            (packet_type < RtcpPacket::SR || packet_type > RtcpPacket::APP))
        {
            return false;
        }

        offset += packet_length;
        packet_count++;
    } while (offset < packet.size());

    if (offset != packet.size())
    {
        return false;
    }

    m_rtcpCompoundPacket.reserve(packet_count);
    return true;
}
```

קלט: מערך המכיל בתוכו פקטה גולמית של RTCP היכולה להכיל בתוכה מספר פקטות RTCP (Compound RTCP Packet).

פלט: אמת אם מבנה הפקטה תקין ועומד בדרישות פרוטוקול RTCP, אחרת שקר.

תיאור האלגוריתם: פונקציה זו מבצעת בדיקת תקינות לפקטת RTCP. מדובר בבדיקת תקינות חיונית לפני ניסיון לנתח, לפענח או להגיב לפקטות Compound RTCP. בכדי לאמת את תקינות הפקטה מתבצעות הבדיקות הבאות:

1. עבור הפקטה הראשונה בתוך ה-Compound אסור שיהיה padding bytes (אם משתמשים ב-padding אז הוא צריך להיות משומש רק בפקטה האחרונה ב-Compound) וחייב שהסוג שלה יהיה Sender report או Receiver report.
2. גרסת ה-RTP של הפקטות חייב להיות לפי התקן (שווה 2).
3. כוללת רק סוגי פקטות לגיטימיים.



4. האורך הכולל של כל הפקטות חייב להיות זהה לגודל כל הפקטה.



27. תיאור מסד הנתונים

בפרויקט זה אין שימוש במסד נתונים.

מאחר ואין צורך עבור פעילות הפרויקט באחסון מתמשך של משתמשים, היסטוריית שיחות או תצוגות ניתוח – הנתונים הרלוונטיים נטענים מקובץ קונפיגורציה סטטי, והמערכת פועלת כולה בזיכרון הראשי בזמן אמת, ללא תלות בגורם חיצוני.



28. מדריך למשתמש

המדריך למשתמש של המערכת יחולק ל-2 חלקים מרכזיים: הפעלת המערכת ושימוש במערכת.

הפעלת המערכת:

מערכת VTM מורכבת מ-2 תהליכים עצמאיים שרצים במחשב של המשתמש:

1. מנוע העיבוד (backend) - אחראי על עיבוד שמע, ניהול השיחות והפקת נתונים.
2. ממשק המשתמש (frontend) - ממשק גרפי לצפייה בשיחות ומדדי איכות.

ראשית, על המשתמש להפעיל את מנוע העיבוד של המערכת. דגשים חשובים בהפעלה:

- ודא שקובץ הקונפיגורציה קיים במערכת, ובנוסף וודא את תקינותו. כל שגיאה בקובץ עלולה לגרום לקריסת המערכת ולסגירתה.
- ודא שקיים במערכת קובץ ייעודי ללוגים לצורך ניטור מידע על פעילות המערכת.
- ודא כי אצל המחשב של המשתמש מותקנות ספריות boost ו-libspeak.

לאחר מכן, מפעילים את ממשק המשתמש של המערכת. במסך הראשי של ממשק המשתמש מופיע סטטוס החיבור אל מנוע העיבוד. יהיה ניתן להשתמש בממשק המשתמש רק אם מופיע הסטטוס "connected to backend".

שימוש במערכת:

לאחר שהמערכת פועלת, ניתן להשתמש בממשק המשתמש לצורך צפייה וניטור בפעילות השיחות המתרחשות על גבי המערכת.

המשתמש יכול לבחור שיחה כרצונו לצפייה וניטור מתוך רשימה של שיחות שמע המופיעות במסך הראשי ולהיכנס בלחיצה עליו לחלון ייעודי לו.

בחלון זה יופיע למשתמש חיווי המקור, נתוני QoS עדכניים, ובנוסף ניתן להוריד קובץ JSON של מצב השיחה.

בלחיצה על ה-Icon של המקור, המשתמש יוכל לראות את מגמת מדדי ה-QoS של המקור נבחר לאורך זמן. תצוגה זו מסייעת בזיהוי בעיות רשת ובניתוח שיחות לצורכי תחקור.

במהלך ריצה, המערכת בנוסף כותבת לוגים לקובץ ייעודי וכמו כן ל-console. הלוגים כוללים תיעוד של פעילות שוטפת, אזהרות ושגיאות. המשתמש יכול לפתוח את קובץ הלוגים או את הטרמינל לצורך ניטור מקביל בנוסף לממשק המשתמש.



29. בדיקות והערכה

במהלך פיתוח המערכת בוצעו מספר סוגים של בדיקות על המערכת, והערכת הביצועים אשר נועדו לבדוק את תקינות המערכת, ולבחון את עמידותה בתנאים משתנים והתאמתה לסטנדרטים המקובלים בתחום ה-VoIP.

Unit tests: בוצעו בדיקות ממוקדות עבור מנגנוני המערכת, כאשר כל מנגנון נבדק באמצעות מספר סוגים של קלטים, חלקם במכוון קלטים לא תקינים (למשל שליחת פקטות UDP אל מודול ה-packet parser) תוך מעקב אחר התנהגות המודול והפלט אשר יוצא ממנו. תוצאות הבדיקות הראו כי המודולים מופרדים היטב מבחינה לוגית תוך הפרדת תחומי אחריות ופועלים בצורה נכונה.

System tests: בוצע הדמיה של מספר זרמי שמע בו זמנית במערכת במטרה לבחון את פעילות המערכת תחת עומס. ההדמיה כללה שליחה של פקטות אל המערכת תוך בחינה של כלל המנגנונים במערכת וזרימת הפקטות בתוכן. המערכת לא חוותה קריסה, והשיחות נשמרו ברמה סבירה של רציפות ותגובה.

בחינת המערכת בוצעה באמצעות התוכנות GStreamer ו-FFmpeg אשר דימו עבור המערכת את נקודות הקצה. התוכנות שימשו ליצירת זרמי שמע עבור המערכת, ובאמצעות כלים אלו היה ניתן לבחון בזמן אמת את אופן פעילות המערכת מבלי ליצור תוכנה חדשנית או לקוח לצורך בחינת המערכת.



30. מסקנות

פרויקט זה מסמל עבורי לא רק השלמה של משימה הנדסית, אלא כנקודת התפתחות אישית ומקצועית שעברתי במהלך השנה. פרויקט זה הוא תוצר של חודשים רבים של למידה עצמית, ניסוי וטעיה, ובנייה הדרגתית של מערכת מורכבת תוך ביצוע התמודדות עם אתגרים טכנולוגיים בעולם התקשורת וה-VoIP.

לאורך הדרך, החל משלב הלמידה הראשוני על עקרונות רשת ופרוטוקולי VoIP עד לכתיבת קוד בשפת C++ עברתי תהליך שגרם לי לרכוש בנוסף לידיע התיאורטי במסגרת העתודה גם יכולות תכנות, ניתוח ותכנון תוכנה – ידע מעשי שיתרום לי רבות בהמשך הדרך. הפרויקט אפשר לי להעמיק ולעסוק בתחום של הכרתי לפני כן תוך מימוש מעשי של עקרונות התחום בעשיית הפרויקט.

בנוסף לידיע הטכנולוגי שרכשתי במהלך השנה, פרויקט זה היווה עבורי הזדמנות לצמיחה. פרויקט זה עזר לי לפתח יכולות חשיבה מערכתיות, ודרכו למדתי איך לפרק בעיה גדולה למשימות קטנות, איך לנהל את לוח זמנים בצורה חכמה, איך להתמודד עם חוסר ודאות, ואיך להפיק תוצרים גם שלא כלל הנתונים היה נתון בהישג יד.

הפרויקט היווה לי את קפיצת המדרגה המקצועית לה הייתי זקוק, כאשר בתחילת הפרויקט לא הייתי מיומן בכתיבת פרויקטים וקוד מעשי ואפילו לא ידעתי איך לכתוב מחלקה או איך להשתמש בפרדיגמות תכנות מתקדמות כמו `async i/o`.

לפיכך, לאור ההתקדמות המקצועית והאישית שעברתי במהלך השנה, אני גאה בתוצאה, מרוצה מהדרך בה עברתי גם אם היו בה עליות וירידות, אני מרגיש מוכן לשירות הצבאי שלי לאחר ביצוע הפרויקט שהיווה כ"ההכשרה" שלי ליחידה, ולסיכום אני גאה בעובדה שזהו הפרויקט תוכנה הראשון שלי.



31. פיתוחים עתידיים

במהלך פיתוח המערכת התמקדתי ביצירת תשתית יציבה לעיבוד וניהול תקשורת שמע בזמן אמת. עם זאת, קיימים מספר כיוונים לפיתוח עתידי אשר נועדו לשפר את ביצועי המערכת ולהרחיב את יכולותיה המבצעיות:

- **תמיכה ב-multithreading:** על אף שהנושא הוזכר כבר בשלב הצעת הפרויקט, עקב אילוצי זמן ועדכונים תכופים שנדרשו במהלך שנת הפרויקט, הוחלט לממש את המערכת בשלב זה בתור Single-threaded. מימוש עתידי של המערכת עם multithreading יאפשר פיצול עומסים בין שיחות מרובות, עיבוד מקבילי של זרמים שונים ושיפור מהותי בביצועי המערכת בתנאי עומס.
- **שדרוג מנגנון PLC:** באופן פעילותה הנוכחי, המערכת משתמשת בפתרון פשוט עבור PLC שהוא יצירת פקטות שמע של שקט. פיתוח עתידי יכלול שילוב של מנגנון PLC מתקדם יותר אשר יתבסס על אלגוריתם כגון Waveform Substitution אשר מחפשת דפוס גל קול דומה ב-frames הקודמים ומשתמשת בו לשחזור הפקטה החסרה.
- **הרחבת ממשק המשתמש:** כמו כן, יהיה ניתן להרחיב את ממשק המשתמש להצגת sound waves של השיחות המתרחשות על גבה. שילוב של פיצ'ר זה לכל שיחה יכול לספק למשתמש יכולת אינטואיטיבית לזהות פעילות קולית ולהבחין בדפוסי שמע חשודים – דבר שיכול לסייע באיתור תקלות, ניתוח ביצועים, או ניטור מבצעי.
- **הרחבת התמיכה בפרוטוקול RTCP עבור סוגים נוספים:** פיתוח עתידי אשר ניתן לבצע עבור המערכת הוא הרחבת התמיכה גם עבור RFC3611. תקן זה מציג סוג חדש של פקטת RTCP מסוג XR. הרחבת התמיכה של המערכת ל-RFC3611 מעבר לתמיכה ב-RFC3550 תוכל לגרום למערכת לקבל דיווחים מפורטים יותר מנקודות הקצה עבור מדדי הרשת והאיכות שלהם. פקטות מסוג XR יאפשרו למערכת להבין בצורה עמוקה יותר את מצב הרשת בפועל, והיא תוכל להסתמך לא רק על המדדים הבסיסיים כמו שמספקות פקטות SR/RR אלא על נתונים יותר מתקדמים, למשל jitter מצטבר, זמן אובדן פקטות, זמני RTT ונתוני MOS (Mean Opinion Score) המשמש להערכת האיכות.



32. בבליוגרפיה

למידה על VoIP:

https://www.geeksforgeeks.org/voice-over-internet-protocol-voip/?ref=header_outind

למידה על פרוטוקולים RTP/RTCP ומנגנון ה-Translator:

<https://www.rfc-editor.org/rfc/rfc3550>

למידה על מנגנון jitter buffer:

<https://trtc.io/blog/details/Jitter-and-Jitter-Buffer>

למידה על ספריות Boost:

[Version 1.85.0](#)

למידה על קודקים:

<https://www.nextiva.com/blog/voip-codecs.html>

למידה על קודק G711:

[G.711: A Complete Guide to the Audio Codec - \[Updated September 2024 \]
\(digitalgadgetwave.com\)](#)

<https://www.allaboutcircuits.com/technical-articles/an-introduction-to-companding-compressing-speech-for-transmission-across-te/>

למידה על קודק Speex ושל הספרייה libspeex:

<https://speex.org/>

למידה על QoS:

<https://www.fortinet.com/resources/cyberglossary/qos-quality-of-service>

למידה על Transcoding:

<https://www.wowza.com/blog/what-is-transcoding-and-why-its-critical-for-streaming>

למידה על PCM:

https://www.geeksforgeeks.org/analog-to-digital-conversion/?ref=header_outind

למידה על PLC:

<https://www.alango.com/technologies-plc.php>